

Chapter 5 Trees

5.1 Introduction

5.1.1 Terminology

Definition: A tree is a finite set of one or more nodes such that

(1) There is a specially designated node called **root**.

(2) The **remaining nodes** are partitioned into $n \geq 0$ disjoint sets $T_1, \dots, T_n$, where each of these sets is a **tree**. $T_1, \dots, T_n$ are called **subtrees** of the root.

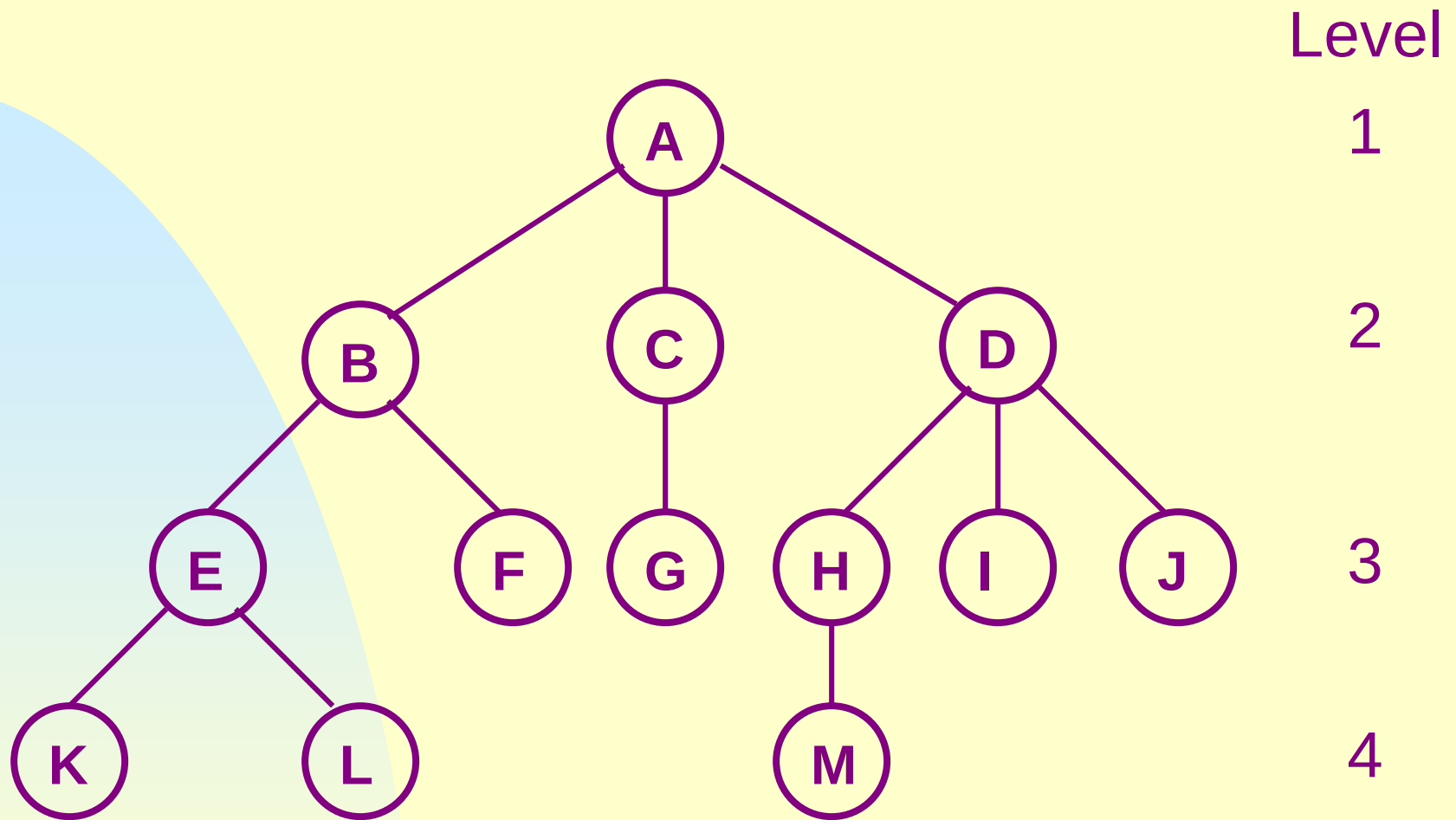


Fig. 5.2 A sample tree

Other definitions:

- ✓ A node stands for the item of information plus the branches to other nodes.
- ✓ The number of subtrees of a node is called its degree.
- ✓ Nodes with degree 0 are leaf or terminal nodes, and the others are nonterminals.
- ✓ The roots of the subtrees of a node X are the children of X, X is the parent of its children.
- ✓ Children of the same parent are said to be siblings.

- ✓ The degree of a tree is the maximum of the degree of the nodes in the tree.
- ✓ The ancestors of a node are all the nodes along the path from the root to that node.
- ✓ The level of a node is defined by letting the root be at level 1, and if a node is at level l , then its children are at level $l+1$.
- ✓ The height or depth of a tree is defined to be the maximum level of any node in the tree.

5.1.2 Representation of Trees

1. Represented by Generalized lists.

The tree of Figure 5.2 could be written as the list

(A(B(E(K,L),F),C(G),D(H(M),I,J)))

- But it is desirable to have a representation that is **specialized to trees**.
- One possibility is to represent each **tree node** by a memory node that has **fields for the data** and **pointers to the tree node's children**.

2. For a tree of degree k , we could use the node structure as follows: a tree node that has a field for data and k pointers to its children as below:

Data	Child1	Child2	...	Child k
------	--------	--------	-----	-----------

Fig.5.4 Possible node structure for a tree of degree k

However, this is very wasteful of space as Lemma 5.1 shows.

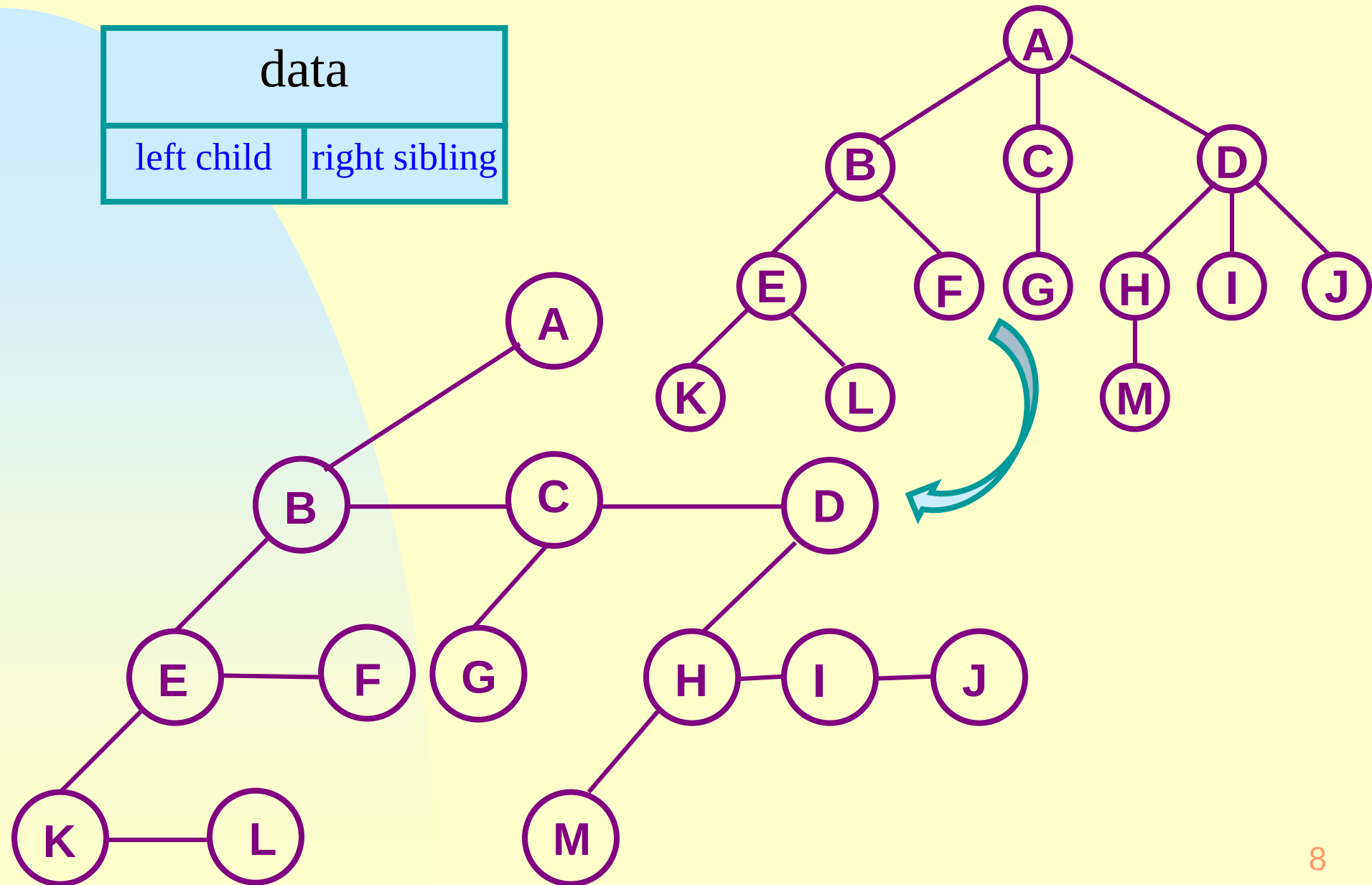
Lemma5.1: If T is a k -ary tree (i.e., a tree of degree) with n nodes, each having a fixed size as in Fig.5.4, then $n(k-1)+1$ of the nk child fields are 0, $n \geq 1$.

Proof:

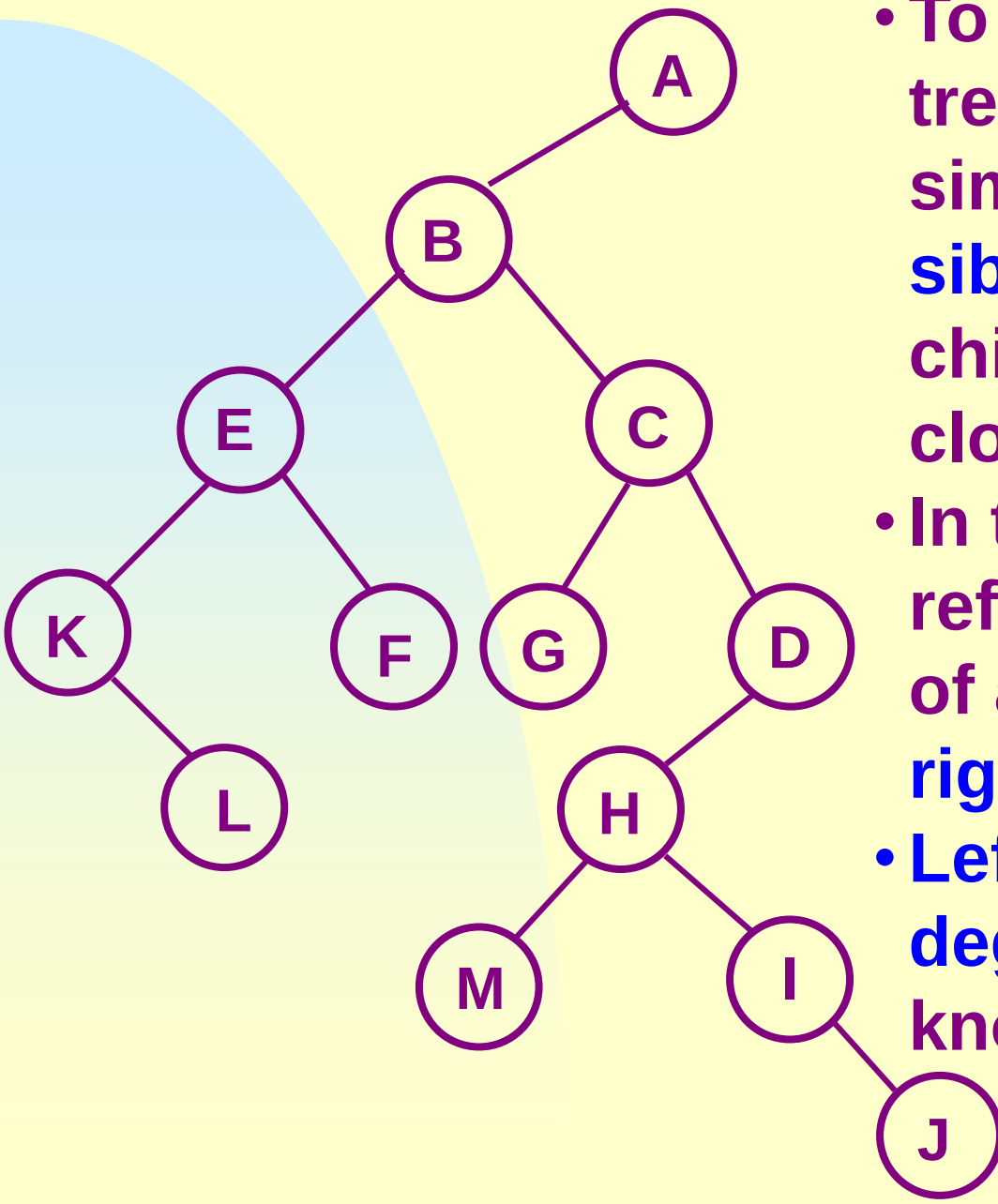
- each non-zero child field points to a node
- there is exactly one pointer to each node other than the root
- the number of non-zero child fields in an n node tree is exactly $n-1$
- Thus, the number of zero fields is $nk-(n-1)=n(k-1)+1$.

3. Left child-right sibling representation

data	
left child	right sibling



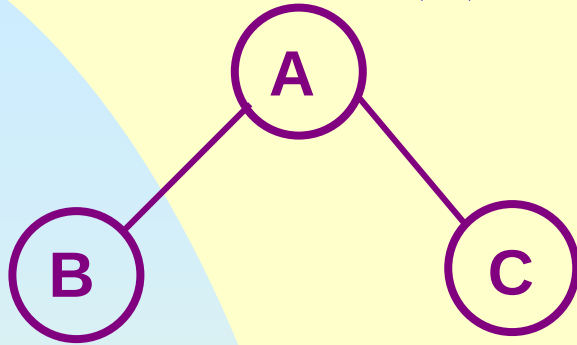
4. Representation as a Degree-Two Tree



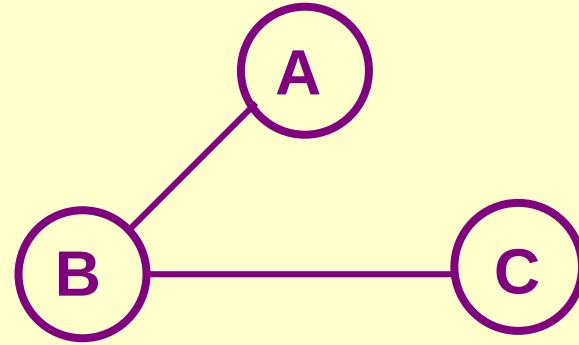
- To obtain the degree-two tree representation, we simply rotate the **right-sibling pointers** in a left child-right sibling tree clockwise by 45 degrees.
- In this representation, we refer to the two children of a node as **the left and right children**.
- **Left child-right child (or degree-two)** trees are also known as **binary trees**

An example:

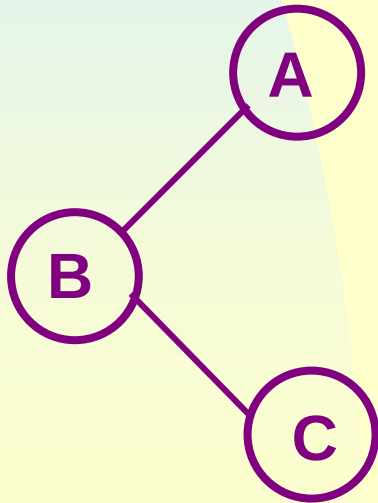
The tree in (a) is represented as (c) in the degree-two representation **not** (d).



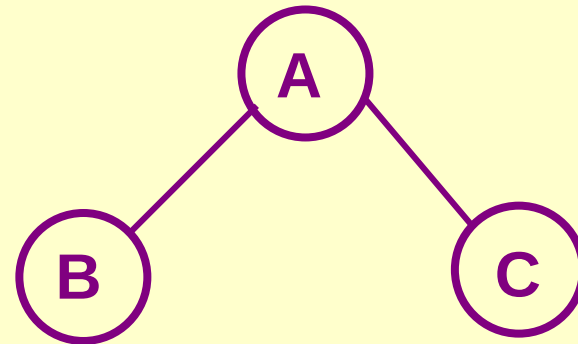
(a) tree



(b) left child-right sibling tree



(c) binary tree

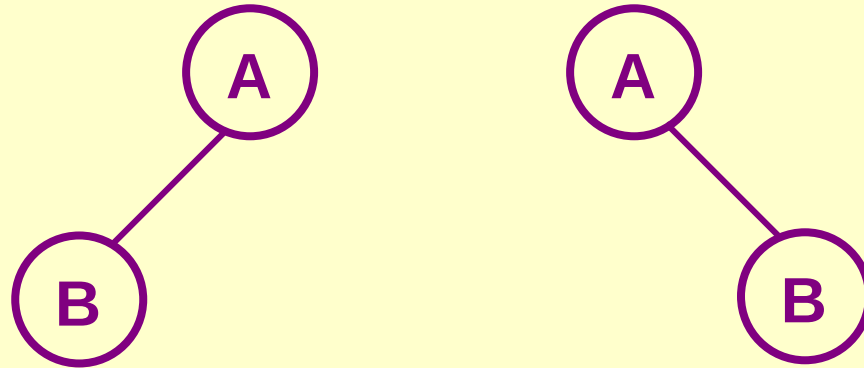


(d) a **different** binary tree

5.2 Binary Trees

5.2.1 The Abstract Data Type

- **Definition:** A binary tree is a finite set of nodes that either is empty or consists of a root and two disjoint binary trees called the left subtree and the right subtree.
- **Distinction between a binary tree and a tree:**
 - No tree has 0 node, but there is an empty binary tree.
 - In a binary tree, we distinguish between the order of the children; in a tree, we do not.



So the above **two binary trees are different**: the first has an empty right subtree while the second has an empty left subtree. But viewed as trees, they are the **same**.

- Note that, a **nonempty** binary tree's root **always** has two subtrees: the left and the right, although they may be empty.
- In a binary tree, the number of nonempty subtrees of a node is defined as its degree.

ADT 5.1

```
template <class T>
```

```
class BinaryTree
```

```
{ // Objects: A finite set of nodes either empty or consisting of  
  // a root node, left BinaryTree and right BinaryTree.
```

```
public:
```

```
  BinaryTree();
```

```
  // creates an empty binary tree
```

```
  bool IsEmpty();
```

```
  // return true iff the binary tree is empty
```

```
  BinaryTree(BinaryTree<T>& bt1, T& item,  
              BinaryTree<T>& bt2);
```

```
  // creates a binary tree whose left subtree is bt1,  
  // right subtree is bt2, and root node contain item.
```

```
  BinaryTree LeftSubtree();
```

```
  // return the left subtree of *this
```

```
T RootData();  
// return the data in the root of *this
```

```
BinaryTree RightSutree();  
// ireturn the right subtree of *this  
};
```

5.2.2 Properties of Binary Trees

Lemma 5.2 [Maximum number of nodes]:

- (1) The maximum number of nodes on **level i** of a binary tree is 2^{i-1} , $i \geq 1$.
- (2) The maximum number of nodes in a binary tree of **depth k** is $2^k - 1$, $k \geq 1$.

Proof:

Let M_i be the maximum number of nodes on level i

(1) Induction on level i .

$i=1$, only the root, $M_i = 2^{1-1} = 2^0 = 1$.

Assume $M_{i-1} = 2^{(i-1)-1}$ for $i > 1$.

Since each node has a maximum degree of 2, $M_i = 2 * M_{i-1} = 2 * 2^{i-2} = 2^{i-1}$.

(2) The maximum number of nodes in a binary tree of depth k is

$$\sum_{i=1}^k M_i = \sum_{i=1}^k 2^{i-1} = 2^k - 1.$$

Lemma 5.3 [Relation between number of leaf nodes and degree-2 nodes]:

For any nonempty binary tree T , if n_0 is the number of leaf nodes and n_2 is the number of nodes of degree 2, then $n_0 = n_2 + 1$.

Proof:

Let n_1 be the number of nodes of degree 1 and n the total number of nodes, we have

$$n = n_0 + n_1 + n_2 \quad (5.1)$$

Each node except for the root has a branch leading into it. If B is the number of branches, then $n = B + 1$. And also $B = n_1 + 2n_2$, hence

$$n = n_1 + 2n_2 + 1 \quad (5.2)$$

$$(5.1) - (5.2): \quad 0 = n_0 - n_2 - 1, \text{ i.e., } n_0 = n_2 + 1.$$

Definition: A full binary tree of depth k is a binary tree of depth k having $2^k - 1$ nodes, $k \geq 0$.

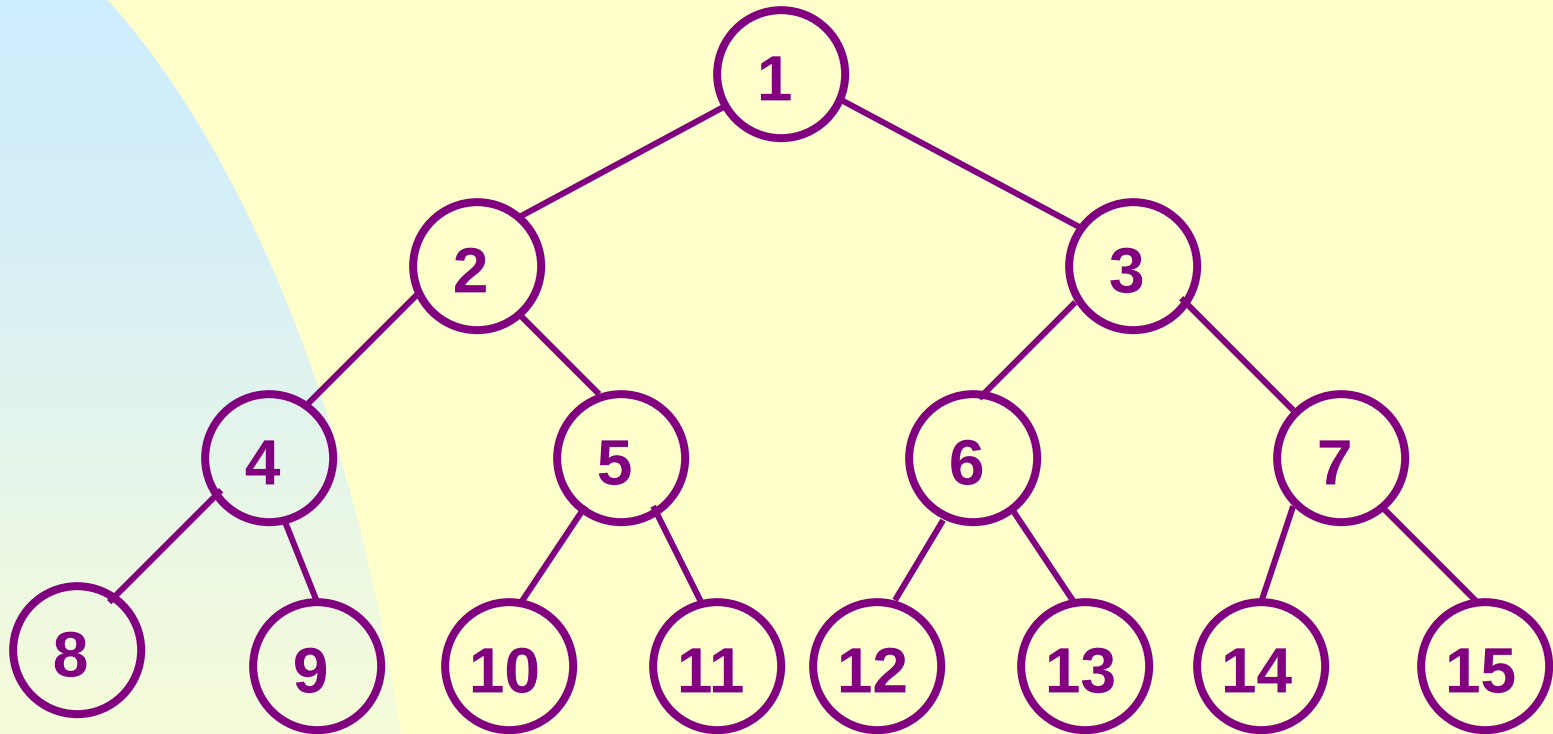


Fig. 5.11 Full binary tree of depth 4

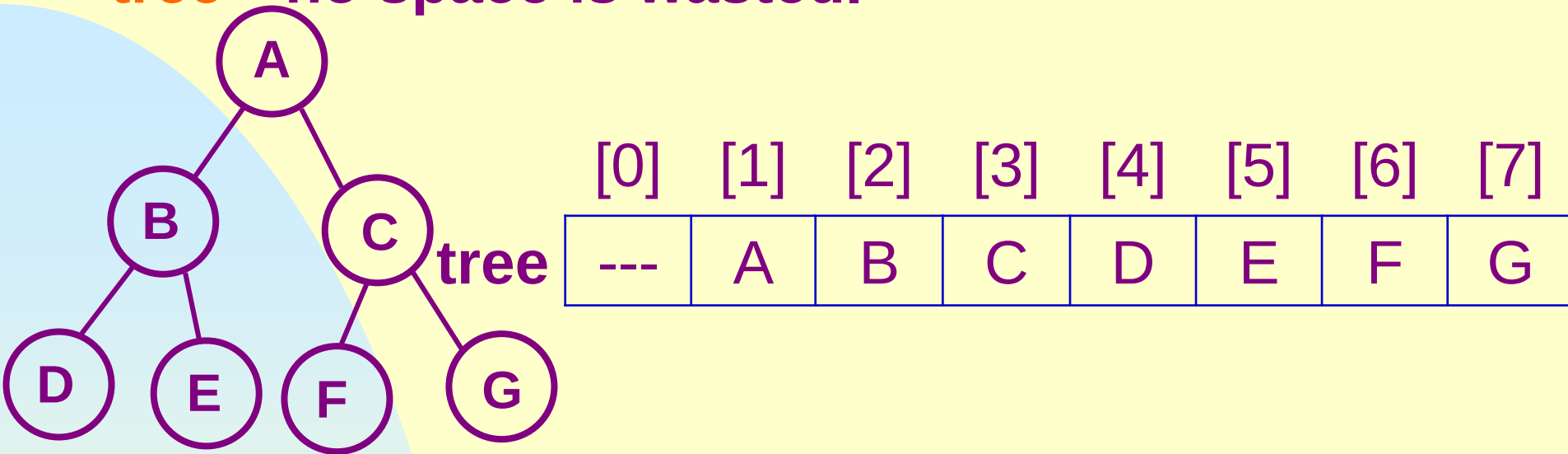
- Suppose we **number the nodes** in a full binary tree starting with the **root on level 1**, continuing with the node on **level 2**, and so on. Nodes on any level are numbered **from left to right**. This **sequentially numbering scheme** gives us the definition of a **complete binary tree**.
- **Definition:** a binary tree with n nodes and depth k is **complete** iff its nodes corresponding to the nodes numbered from **1** to n in the full binary tree of depth k .
- From Lemma 5.2, the height of a complete binary tree with n nodes is $\lceil \log_2(n+1) \rceil$.

5.2.3 Binary Trees Representations

5.2.3.1 Array Representation

- The nodes may be stored in a **one dimensional array named tree**, with the node sequentially numbered i being stored in **tree[i]**. If the C++ array is used to represent the tree, the **0th position is left empty**.

- The array representation for a **complete binary tree**---no space is wasted.



As for the array representation for a **complete binary tree**, we have:

- **Lemma 5.4:** if a complete binary tree with n nodes is represented sequentially, then for any nodes with index i , $1 \leq i \leq n$, we have:

(1) $\text{parent}(i)$ is at $\lfloor i/2 \rfloor$ if $i \neq 1$. If $i=1$, i is the root and has no parent.

(2) LeftChild(i) is at $2i$, if $2i \leq n$. If $2i > n$, i has no left child.

(3) RightChild(i) is at $2i+1$, if $2i+1 \leq n$. If $2i+1 > n$, i has no right child.

Proof: We prove (2).

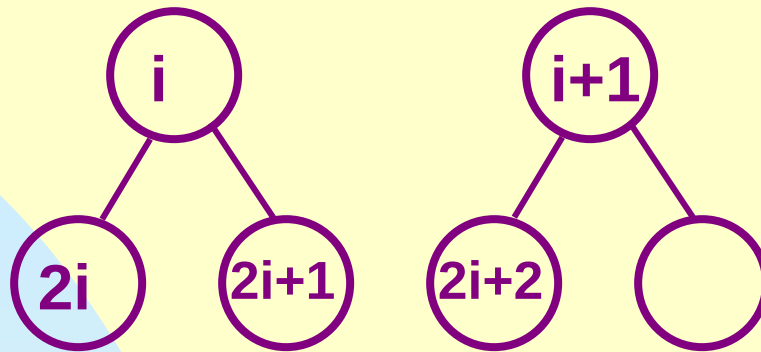
(3) follows from (2) and the nodes on the same level numbering from left to right.

(1) follows from (2) and (3).

We prove by induction on i.

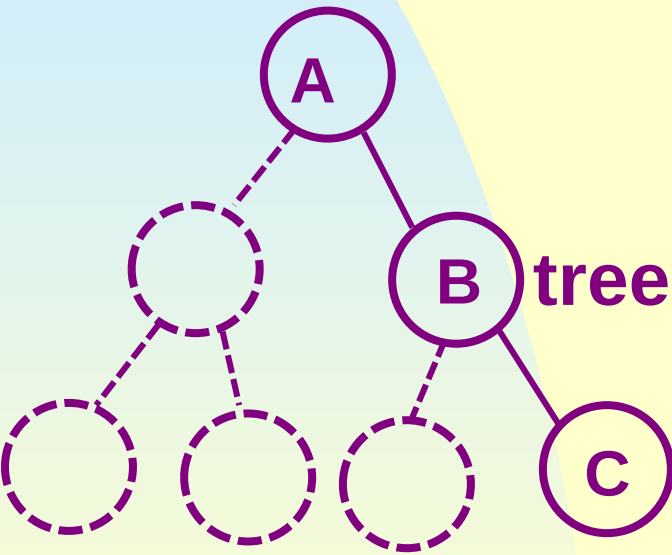
For $i=1$, clearly the left child is at 2 unless $2 > n$, in which case i has no left child.

Assume for all j , $1 \leq j \leq i$, LeftChild(j) is at $2j$.



Then the two nodes immediately preceding $\text{LeftChild}(i+1)$ are the right and left children of i . The left child is at $2i$. Hence $\text{LeftChild}(i+1)$ is at $2i+2=2(i+1)$ unless $2(i+1)>n$, in which case $i+1$ has no left child.

- The array representation for other binary trees---**space is wasteful**. Especially, for a **skewed tree** to the right of depth k , in the worst case, it will require 2^k-1 spaces of which only k will be used:



tree

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
---	A	---	B	---	---	---	C

5.2.3.2 Linked Representation

- We have known that the array representation is good for complete binary trees, but it is **wasteful of space for other binary trees** since there are a lot of unutilized space.
- In addition, the **array representation suffers from the general inadequacies of sequential representation: insertion and deletion of nodes from the middle of a tree require the movement of potentially many nodes** to reflect the change in level number of these nodes.
- These problems can be easily overcome through the use of a **linked representation**.

We use two classes **TreeNode** and **Tree** to define a tree using a linked representation.

```
template <class T> class Tree;
```

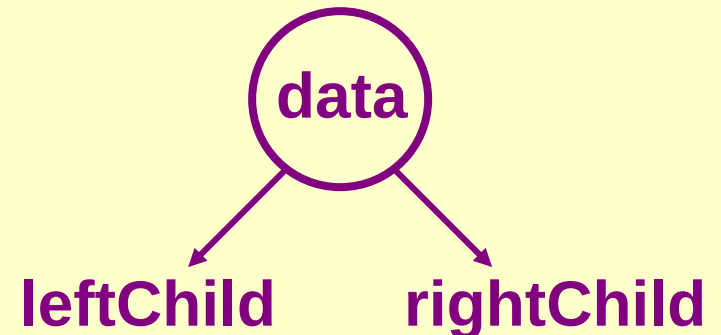
```
class TreeNode {  
friend class Tree<T>;  
public:
```

```
    TreeNode (T& e, TreeNode<T>* left, TreeNode<T>* right)  
        {data=e; leftChild=left; rightChild=right;}
```

```
private:
```

```
    T data;  
    TreeNode<T>* leftChild;  
    TreeNode<T>* rightChild;
```

```
};
```



```
template <class T>
class Tree {
public:
    // Tree operations
    ...
private:
    TreeNode<T>* root; //the access pointer of the tree
};
```

If necessary, a 4th field, **parent**, may be included in the class **TreeNode**.

5.3 Binary Tree Traversal and Tree Iterators

5.3.1 Introduction

- A traversal produces a linear order for the nodes in a tree. This linear order may be familiar and useful.
- When traversing a binary tree, we want to treat each node and its subtrees in the same fashion.
- If we let L, V, R stand for moving left, visiting the node, moving right when at a node, and adopt the convention of traversing left before right, we have 3 traversals: LVR (inorder), LRV (postorder), VLR (preorder).
- There is a natural correspondence between these traversals and producing the infix, postfix and prefix forms of an expression.

We will use the following **arithmetic expression tree** to illustrate each of these traversals.

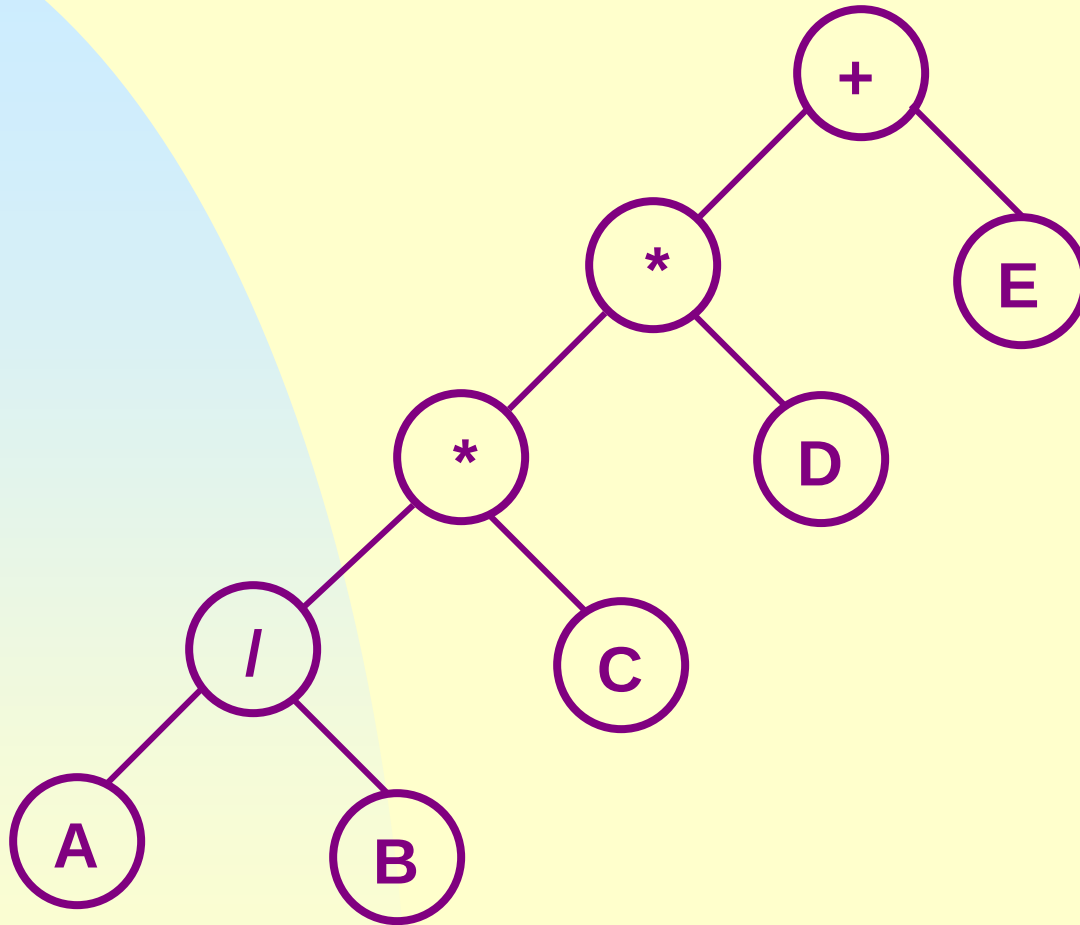


Fig. 5.16 Binary tree with arithmetic expression

5.3.2 Inorder Traversal

```
template <class T>
void Tree<T>::Inorder()
{ // Driver calls workhorse for traversal of entire tree. The driver
  // is declared as a public member function of Tree.
  Inorder(root);
}
```

```
template <class T>
void Tree<T>::Inorder (TreeNode<T>* currentNode)
{ // Workhorse traverses the subtree rooted at currentNode.
  // The workhorse is declared as a private member of Tree
  if (CurrentNode) {
    Inorder(currentNode→leftChild);
    Visit(currentNode)
    Inorder(currentNode→rightChild);
  }
}
```

Assume the Visit function has a single line of code:

```
cout <<CurrentNode→data;
```

For the tree of Fig. 5.16, the elements get output as:

$A / B * C * D + E$ (infix)

5.3.3 Preorder Traversal

```
template <class T>
void Tree<T>::Preorder()
{ // Driver.
    Preorder(root);
}
```

```
template <class T>
void Tree<T>::Preorder(TreeNode<T>* currentNode)
{ // workhorse.
    if (currentNode) {
        Visit(currentNode);
        Preorder(currentNode→leftChild);
        Preorder(currentNode→rightChild);
    }
}
```

For the tree of Fig. 5.16, the elements are output as:

+ * * / A B C D E (prefix)

5.3.4 Postorder Traversal

```
template <class T>
void Tree<T>::Postorder()
{ // Driver.
    Postorder(root);
}

template <class T>
void Tree<T>::Postorder (TreeNode<T>* currentNode)
{ // Workhorse.
    if (currentNode) {
        Postorder(currentNode→leftChild);
        Postorder(currentNode→rightChild );
        Visit(currentNode);
    }
}
```

For the tree of Fig. 5.16, the elements are output as:

$A B / C * D * E +$ (postfix)

5.3.5 Iterative Inorder Traversal

Since the tree is a container class, we would like to implement an iterator for the class Tree.

To implement a tree traversal by using iterators, we first need to implement a non-recursive tree traversal algorithm.

A nonrecursive code for inorder traversal is given in Program 5.4. This program **USES-A** template stack (“**USES-A**” refers to an new definition).

Definition: A data object of Type X USES-A data object of Type Y if a Type X object uses a Type Y object to perform a task.

This USES-A relationship is typically expressed by employing the type Y object in a member function of Type X.

The USES-A relationship is similar to IS-IMPLEMENTED-IN-TERMS-OF relationship. The difference is that the degree to which the Type Y object is used is less in the USES-A relationship than in the IS-IMPLEMENTED-IN-TERMS-OF relationship.

Program 5.4: Nonrecursive inorder traversal

```
1 template <class T>
```

```
2 void Tree<T>::NonrecInorder()
```

```
3 { // Nonrecursive inorder traversal using a stack
```

```
4   Stack<TreeNode<T>*> s; // declare and initialize a stack
```

```
5   TreeNode<T>* currentNode=root;
```

```
6   while (1) {
```

```
7     while (currentNode) { // move down leftChild
```

```
8       s.Push(currentNode); // add to stack
```

```
9       currentNode=currentNode→leftChild;
```

```
10    } //end of inner while
```

```
11    if (s.IsEmpty()) return;
```

```
12    currentNode=s.Top();
```

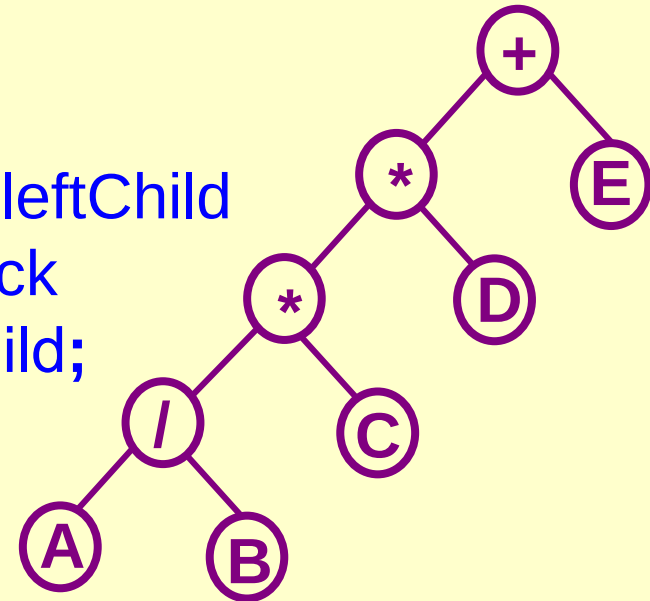
```
13    s.Pop(); // delete from stack
```

```
14    Visit(currentNode);
```

```
15    currentNode=currentNode→rightChild;
```

```
16  } //end of outer while
```

```
17 }
```



Inorder: A / B * C * D +E

Analysis of NonrecInorder:

- We assume the number of nodes in the tree is n .
- Every node is placed on the stack once, thus line 8, 9 and 11 to 15 are executed n times.
- `currentNode` will equal 0 once for every 0 link, the number of which is $n+1$, thus the time for line 7 is $O(n)$.
- Every step will be executed no more than some constant times of n , so the time complexity is $O(n)$.
- The space required for the stack is equal to the depth of the tree. This is at most n .

Now we use the function `NonrecInorder` to obtain an `inorder` iterator for a tree.

Inorder Iterator of Binary Tree

- It is obvious that each iteration of the while loop of line 6-16 in `NonrecInorder()` yields the next element in the inorder traversal of the tree.
- So, we now use the function `NonrecInorder` to obtain an **inorder iterator** for a tree.
- Rather than develop an iterator as complex as a C++ forward iterator, we consider a simple iterator that has only a **Next** function, which returns a pointer to the next element in inorder.

```
class InorderIterator { // a public nested member class of Tree
public:
```

```
    InorderIterator() {currentNode=root;};
```

```
    T* Next( );
```

```
    //Next() returns a pointer to the next elements in inorder
```

```
    //The first time to invoke Next() returns the pointer to the 1st
    element.
```

```
private:
```

```
    Stack<TreeNode<T>*> s;
```

```
    TreeNode<T>* currentNode;
```

```
};
```

Program 5.5: Definition of a simple inorder iterator class.

```
T* InorderIterator::Next()
```

```
{
```

```
    while (currentNode) {
```

```
        s.Push(currentNode);
```

```
        currentNode=currentNode→LeftChild;
```

```
    }
```

```
    if (s.IsEmpty()) return 0;
```

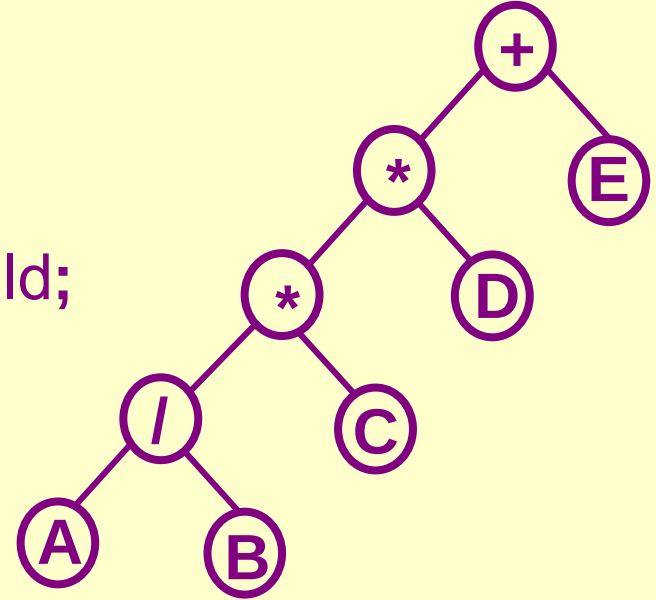
```
    currentNode=s.Top();
```

```
    T& temp=currentNode→data;
```

```
    currentNode=currentNode→rightChild;
```

```
    return &temp;
```

```
}
```



Program 5.6: Code for obtaining the next inorder element.

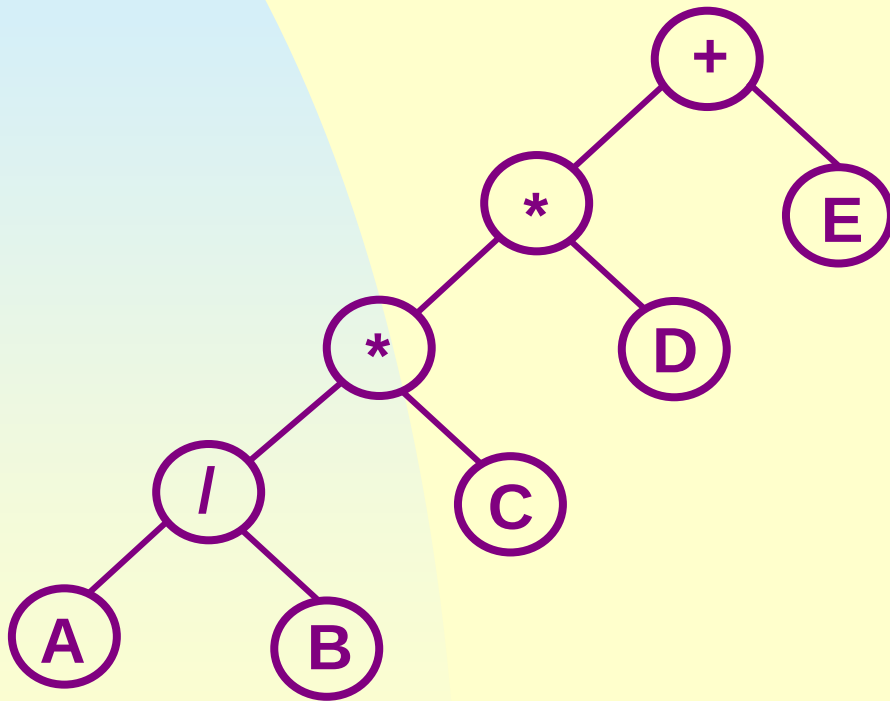
NonrecInorder function using Next()

```
1 template <class T>
2 void Tree<T>::NonrecInorder2()
3 { // Nonrecursive inorder traversal using Next()
4   InorderIterator it; // declare and initialize a iterator
5   for (T* p = it.Next(); p; p = it.Next())   Visit(*p);
6 }
```

We can assume the Visit function has a single line of code:
`cout<<*p;`

5.3.6 Level-order Traversal

Level-order traversal visits the nodes using the ordering suggested by the numbering scheme of the full binary tree nodes.



The level-order traversal of the left tree is:
+ * E * D / C A B

To implement level-order traversal, we use a **queue**.

```
template <class T>
void Tree<T>::LevelOrder()
{ // traverse the binary tree in level order
    Queue<TreeNode<T>*> q;
    TreeNode<T>* currentNode=root;
    while (currentNode) {
        Vist(currentNode);
        if (currentNode→leftChild)
            q.Push(currentNode→leftChild);
        if (currentNode→rightChild)
            q.Push(currentNode→rightChild);
        if (q.IsEmpty())return;
        currentNode=q.Front();
        q.Pop();
    }
}
```

Program 5.7: Level-order traversal of a binary tree.

5.4 Additional Binary Tree Operations

5.4.1 Copying Binary Trees

```
template <class T>
Tree<T>::Tree(const Tree<T>& s) // driver
{ // Copy constructor
    root = Copy( s.root ); //root=s.root is wrong.
}
template <class T>
TreeNode<T>* Tree<T>::Copy(TreeNode<T>* origNode)
// workhorse
{ // Return a pointer to an exact copy of the binary
  // tree rooted at origNode
  if (!origNode) return 0;
  return new TreeNode<T>(origNode→data,
                          Copy(origNode→leftChild),
                          Copy(origNode →rightChild));
}
```

5.4.2 Testing Equality

```
template <class T>
bool Tree<T>::operator==(const Tree& t) const //driver
{
    return Equal(root, t.root);
}

template <class T>
bool Tree<T>::Equal(TreeNode<T>* a, TreeNode<T>* b)
{ // Workhorse
    if ((!a) && (!b)) return true; // both a and b are 0
    return (a && b                // both a and b are non-0
            && (a->data == b->data) //data is the same
            && Equal(a->leftChild, b->leftChild) //left equal
            && Equal(a->rightChild, b->rightChild)); //right equal
}
```


5.5 (Inorder) Threaded Binary Trees

5.5.1 Threads

- Note that in the linked representation of binary tree, there are $n + 1$ 0 links, which is more than actual pointers.
- A clever way to make use of these 0 links is to replace them by pointers, called threads, to other nodes in the tree.

■ The threads are constructed using the following rules:

(1) A 0 **rightChild** field at node *p* is replaced by a pointer to the **inorder successor** of *p*.

(2) A 0 **leftChild** field at node *p* is replaced by a pointer to the **inorder predecessor** of *p*.

Example: the following is a threaded tree, in which node **E** has a predecessor thread pointing to **B** and a successor thread to **A**.

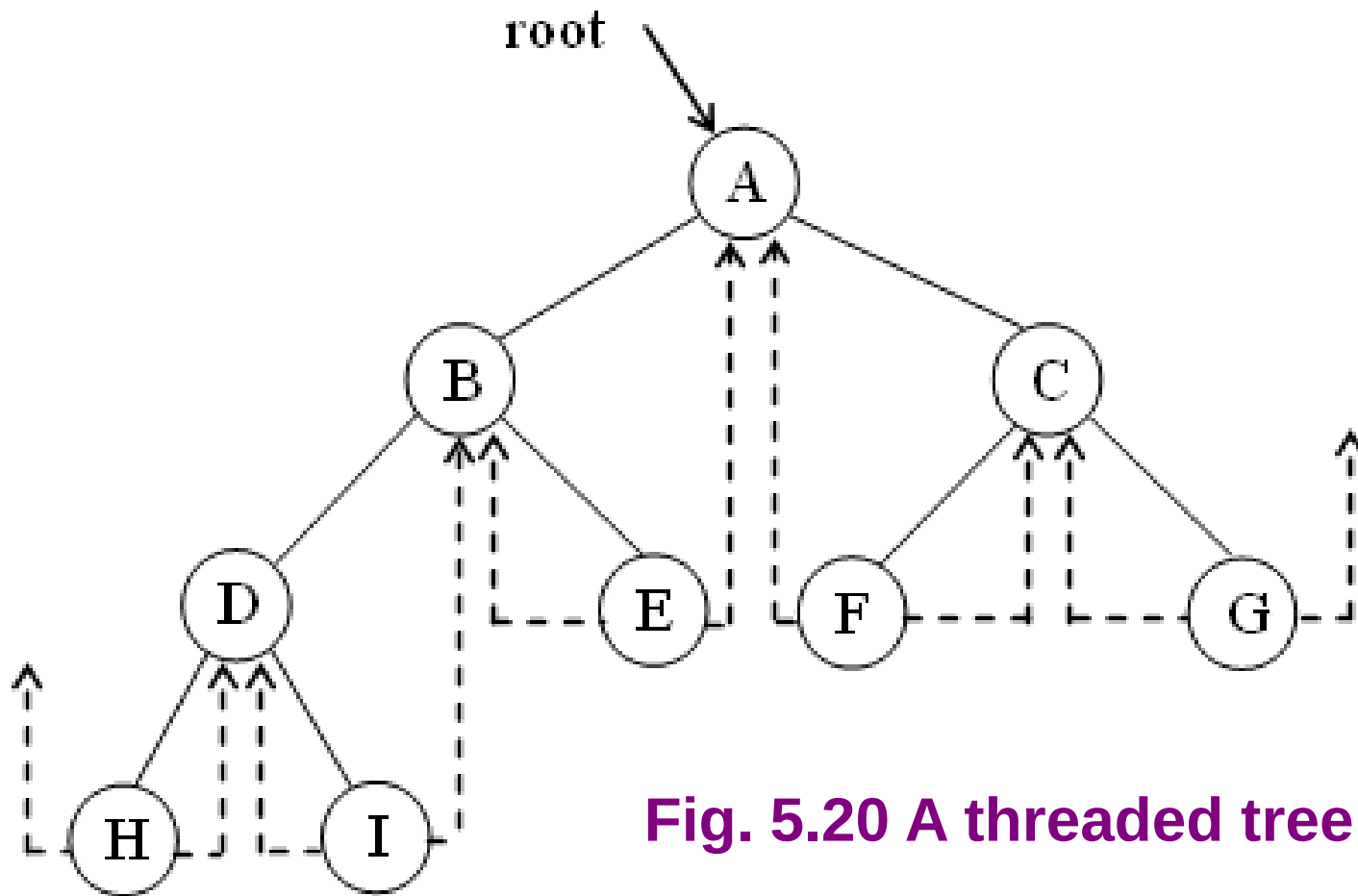


Fig. 5.20 A threaded tree

To distinguish between threads and normal pointers, add two boolean fields:

- leftThread
- rightThread

If $t \rightarrow \text{leftThread} == \text{true}$, then $t \rightarrow \text{leftChild}$ contains a thread, otherwise a pointer to left child.

If $t \rightarrow \text{rightThread} == \text{true}$, then $t \rightarrow \text{rightChild}$ contains a thread, otherwise a pointer to right child.

The representations of Threaded Binary Tree

```
template <class T>
class ThreadedNode {
friend class ThreadedTree;
private:
    bool leftThread;
    ThreadedNode * leftChild;
    T data;
    ThreadedNode * rightChild;
    bool rightThread;
```

```
};
```

leftThread	leftChild	data	rightChild	rightThread
------------	-----------	------	------------	-------------

```
template <class T>
class ThreadedTree {
public:
    ...// Tree operations
private:
    ThreadedNode *root;
};
```

- We notice that in Fig 5.20, two threads have been left dangling. One is the leftChild of the first node in inorder (node H) and the other is the rightChild of the last node in inorder (node G).
- To make no dangling threads, we assume a head node for all threaded binary trees, and let these two threads point to the head. (As shown in Fig 5.22 in the next slide)
- An empty binary tree is represented by its header as shown in Figure 5.21.

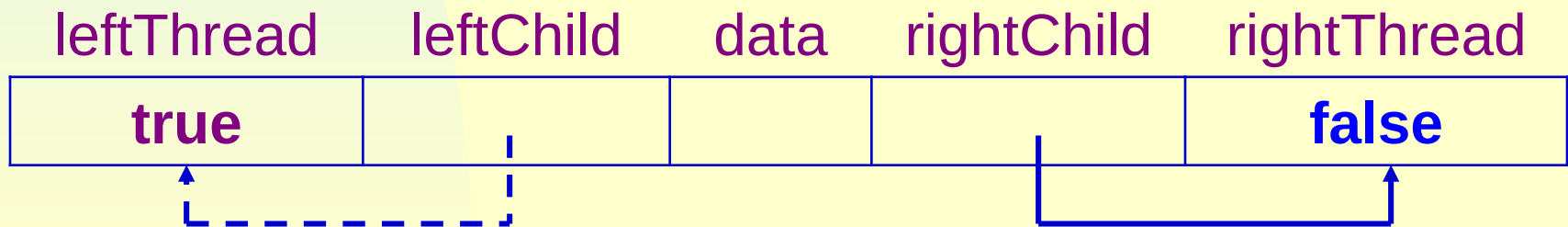


Fig.5.21 An empty threaded binary tree

- The original tree is the left subtree of the head, and the rightChild of head points to the head itself.

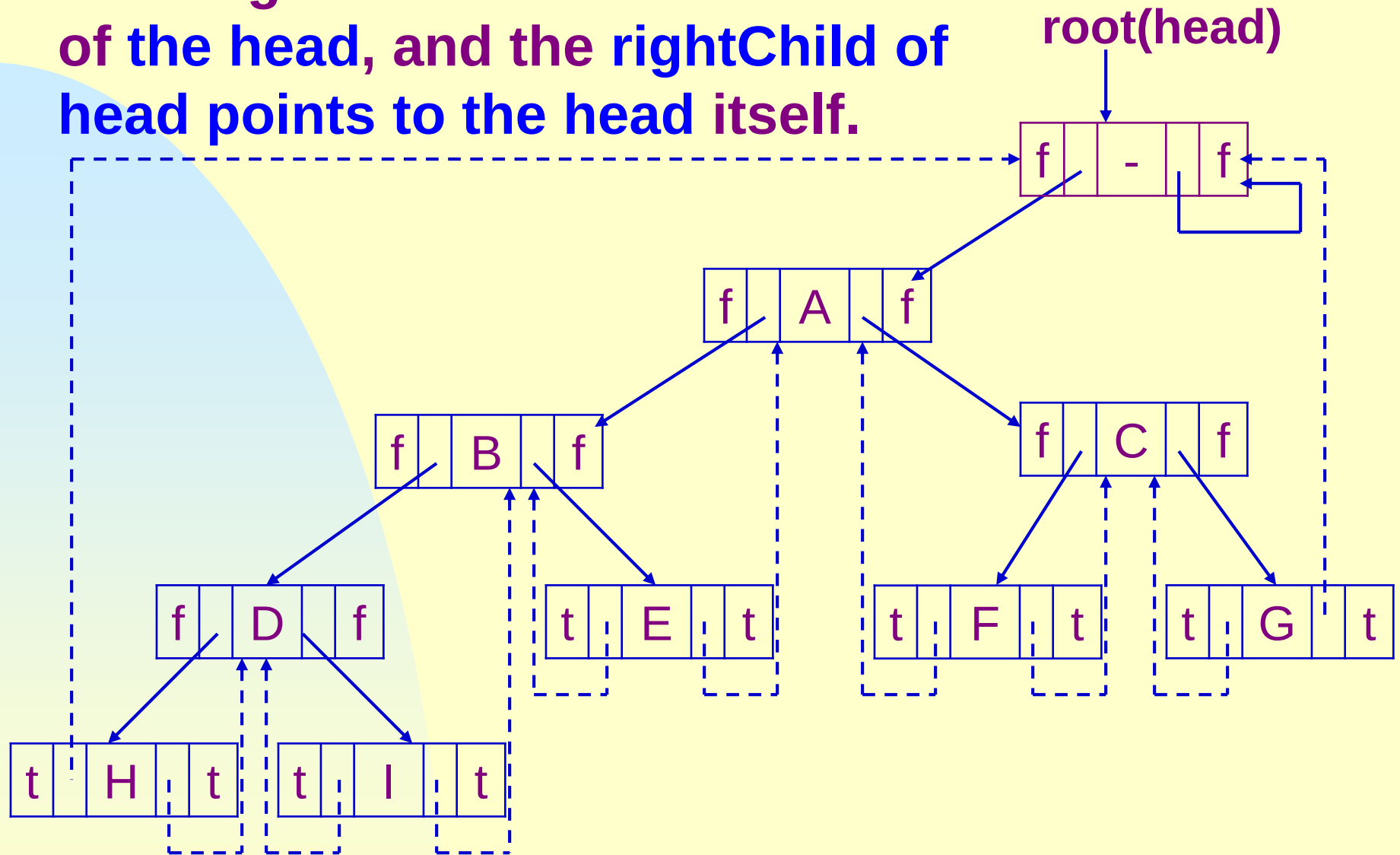


Fig.5.22 memory representation of threaded tree

➤ Thus we can see:

- The inorder predecessor of the first node of the inorder sequence of the original binary tree is the head node.
- The inorder successor of the head node is the first node of the inorder sequence of the original binary tree.
- The inorder successor of the last node of the inorder sequence of the original binary tree is the head node.
- The inorder predecessor of the head node is the last node of the inorder sequence of the original binary tree.

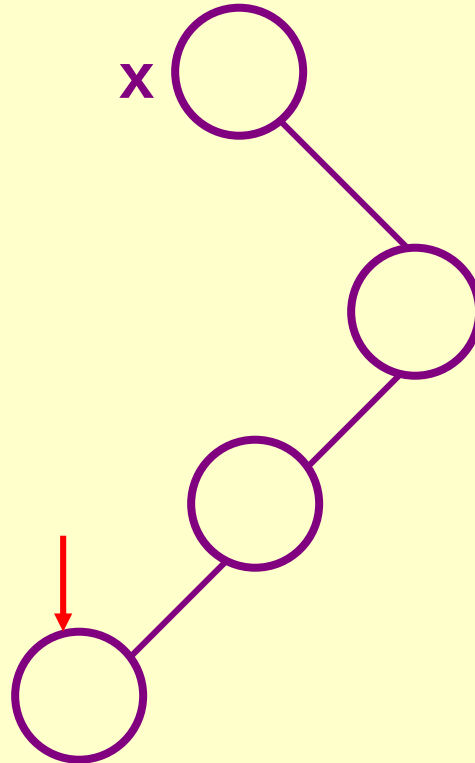
5.5.2 Inorder Traversal of a Threaded Binary Tree

- By using the threads, we can perform an inorder traversal **without making use of a stack.**

(1) If $x \rightarrow \text{rightThread} == \text{true}$, the inorder successor of x is $x \rightarrow \text{rightChild}$;

Note: the inorder successor of the node at bottom-right corner is **head**.

(2) If $x \rightarrow \text{rightThread} == \text{false}$, the inorder successor of x is obtained by following a path of `leftChild` links from the right child of x until a node with `leftThread == true` is reached.



Note: It is still right even if x points to the head node, because $\text{head} \rightarrow \text{rightThread} == \text{false}$ and $\text{head} \rightarrow \text{rightChild} == \text{head}$, the inorder successor of head is the node at bottom-left corner.

InorderIterator of a thread binary tree

Let ThreadedInorderIterator be a nested class of ThreadedTree:

```
class ThreadedInorderIterator {  
public:  
    T* Next();  
    ThreadedInorderIterator()  
        { currentNode = root; } //root is the head node  
private:  
    ThreadedNode<T>* currentNode;  
};
```

The function **Next()** that finds and return the **inorder successor** of currentNode in a threaded binary tree node without using an additional stack.

```

T* ThreadedInorderIterator::Next()
{ // Return the inorder successor of currentNode in a
  // threaded binary tree
  ThreadedNode<T>* temp=currentNode→rightChild;
  if (! currentNode→rightThread)
    while (!temp→leftThread) temp=temp→leftChild;
  currentNode=temp;
  //currentNode now points to the next
  if (currentNode==root) return 0;
  //the head node is the successor of the last node but not visited
  else return &currentNode→data;
}

```

- The head node is the right child of itself, and the original binary tree is the left subtree of the head node, so we can obtain the first node of the inorder sequence by invocation of the function Next().

Inorder traversal with ThreadedInorderiterator

- We can obtain the **inorder sequence** of nodes in the original binary tree by repeated invocations of the function **Next ()**.

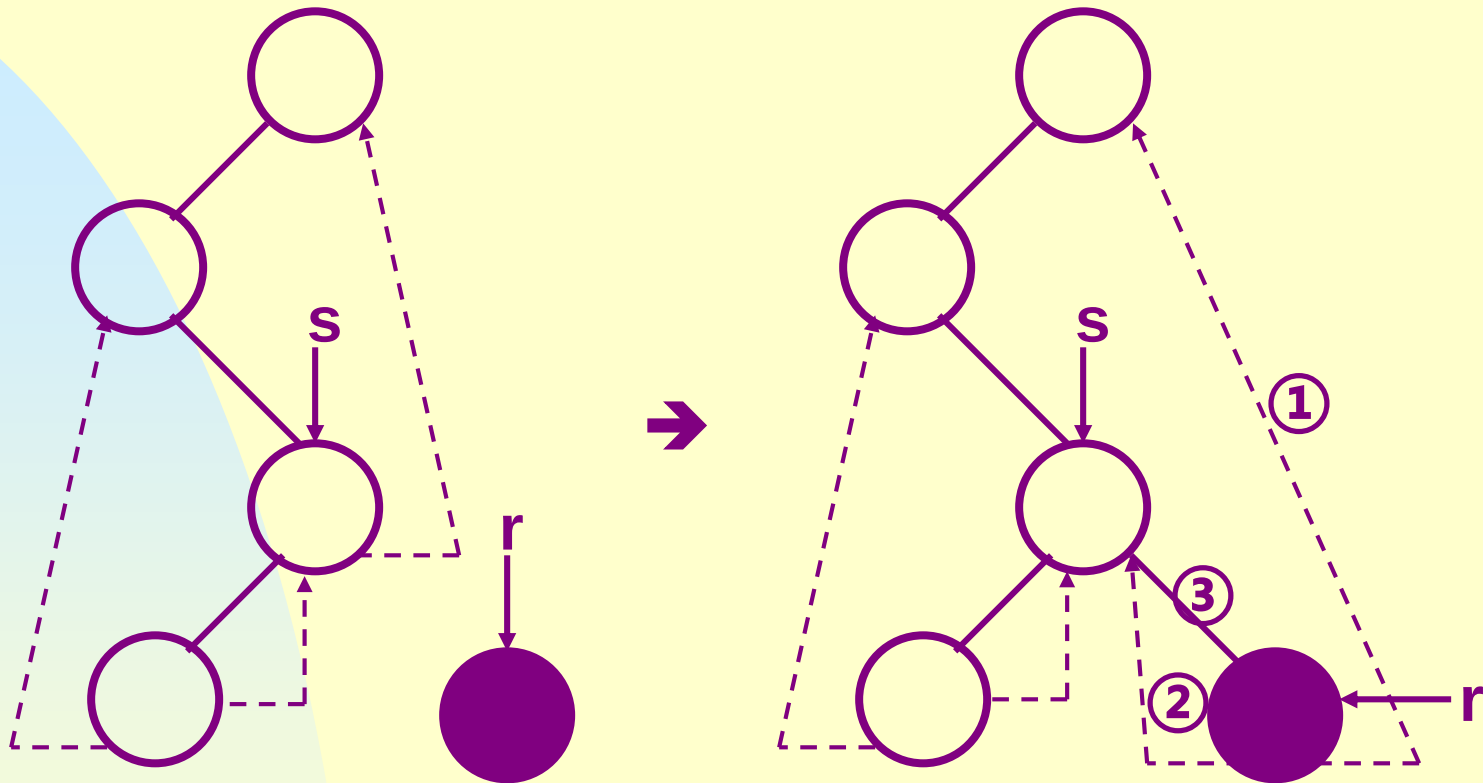
```
template <class T>
void ThreadedTree::Inorder()
{
    ThreadedInorderiterator ti;
    for (T* p = ti.Next(); p; p = ti.Next())
        Visit(*p); //when p==NULL, currentNode points to head
}
```

- For a n -node binary tree, we know the computing time of NonrecInorder is $O(n)$, so the **computing time of Inorder** is also $O(n)$, but the traversal has been done **without using a stack**.

5.5.3 Inserting a node into a Threaded Binary Tree

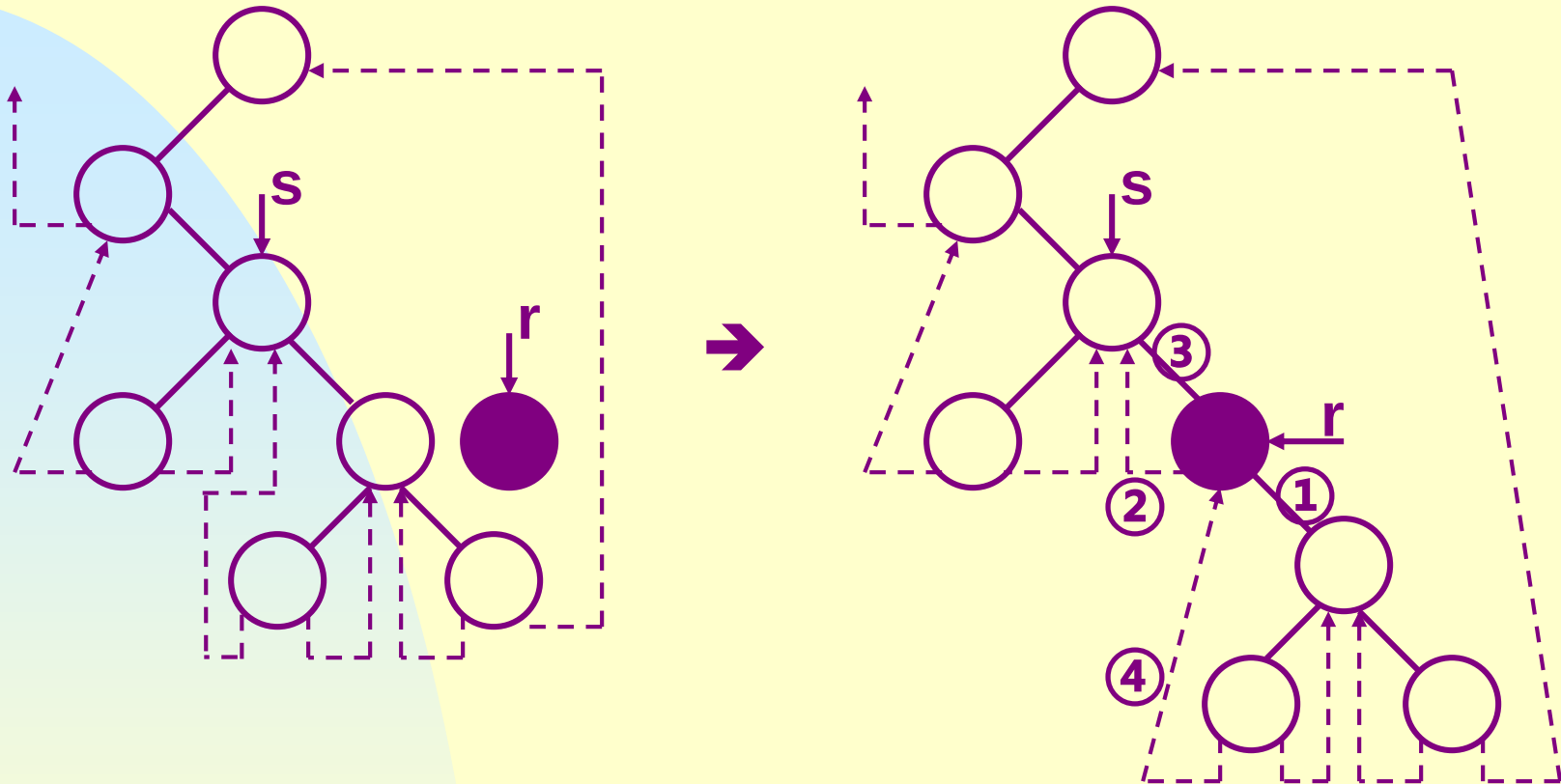
- We shall study only the case of inserting **r** as the right child of a node **s**. The case of insertion of a left child is similar.
- There are two cases for insertion:
 - (1) $s \rightarrow \text{rightThread} == \text{true}$
 - (2) $s \rightarrow \text{rightThread} == \text{false}$

(1) If $s \rightarrow \text{rightThread} == \text{true}$:



- ① $r \rightarrow \text{rightChild} = s \rightarrow \text{rightChild}$; $r \rightarrow \text{rightThread} = s \rightarrow \text{rightThread}$;
- ② $r \rightarrow \text{leftChild} = s$; $r \rightarrow \text{leftThread} = \text{true}$;
- ③ $s \rightarrow \text{rightChild} = r$; $s \rightarrow \text{rightThread} = \text{false}$;

(2) If $s \rightarrow \text{rightThread} == \text{false}$:



The original right child of s is as the new right child of r .
In both (1) and (2), actions ①, ②, ③ are the same, ④ is special for (2).

④ ThreadedNode<T>* temp=InorderSucc(r); temp→leftChild=r;
//InorderSucc(r) returns the inorder successor of r.


```

template <class T>
void ThreadedTree<T>::InsertRight(ThreadedNode<T>* s,
                                     ThreadedNode<T>* r)
{ //insert r as the right child of s
    r→rightChild=s→rightChild;           // ①
    r→rightThread=s→rightThread;         // ①
    r→leftChild=s;                       // ②
    r→leftThread=true;                   // ②
    s→rightChild=r;                      // ③
    s→rightThread=false;                 // ③
    if (! r→rightThread) { // case (2)
        ThreadedNode<T>* temp=InorderSucc(r); // ④
        temp→leftChild=r;                  // ④
    }
}

```

5.6 Heaps

5.6.1 Priority Queues

- ✓ Heaps are frequently used to implement **priority queues**. In this kind of queue, the element to be deleted is the one with **highest (or lowest) priority**.
- ✓ At any time, an element with arbitrary priority can be inserted into the priority queue.
- ✓ ADT 5.2 in the next slide specifies a **max priority queue** as a C++ abstract class MaxPQ.
- ✓ We represent MaxPQ using **pure virtual functions** (and hence make it an abstract class) because it can be realized by a number of different data structures.

ADT 5.2 A max priority queue

```
template <class T>
class MaxPQ {
public:
    virtual ~MaxPQ { }
        // virtual destructor

    virtual bool IsEmpty() const = 0;
        // return true iff the priority queue is empty

    virtual const T& Top() const = 0;
        // return reference to the max element

    virtual void Push(const T&) = 0;
        // add an element to the priority queue

    virtual void Pop() = 0;
        // delete the max element
};
```

//We know that any data structure **D** that implements a max priority queue must implement the operations specified in ADT 5.2. This is ensured by implementing **D** as a publicly derived class of **MaxPQ**.

■ The simplest way to represent a **priority queue** is as an **unordered linear list**. Regardless of whether this list is represented **sequentially** or **as a chain**:

- **IsEmpty** and **Push** is $O(1)$.
- **Top** and **Pop** is $O(n)$.

■ As we shall see shortly, when a **max heap** is used:

- **IsEmpty** and **Top** is $O(1)$.
- **Push** and **Pop** is $O(\log n)$.

5.6.2 Definition of a Max(Min) Heap

- A **max (min) tree** is a tree in which the key value in each node is no smaller (larger) than the key values in its children (if any).
- A **max (min) heap** is a **complete binary tree** that is also a **max (min) tree**.

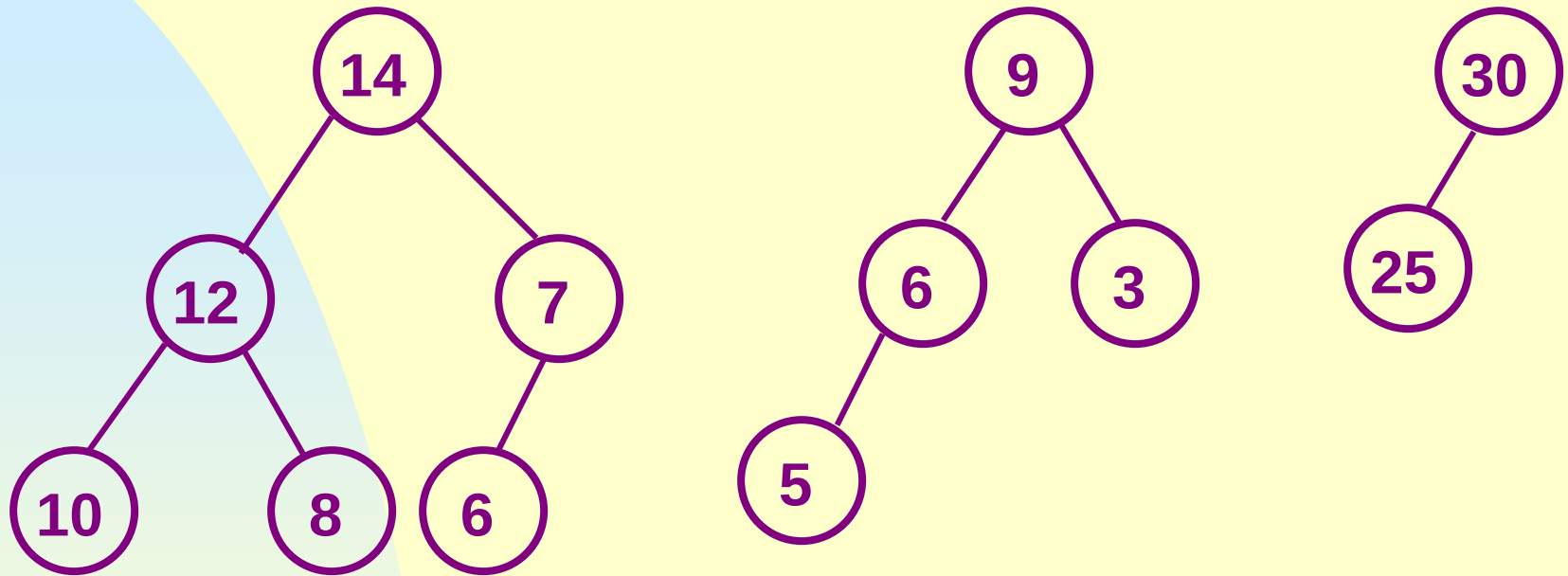


Fig. 5.24 Max heaps

- When viewed as an ADT, a max heap is very simple. The basic operations are the same as those for a max priority queue (ADT 5.2).
- Since a max heap is a complete binary tree, we represent it using an array heap.
- Thus we can get the template MaxHeap class, which derives from MaxPQ<T>, as in the next slide:

```
template <class T>
class MaxHeap: public MaxPQ <T>
{
public:
    MaxHeap (int theCapacity=10);
    bool IsEmpty () { return heapSize==0;}
    const T& Top() const;
    void Push(const T&);
    void Pop();
private:
    T* heap;           // element array
    int heapSize;      // number of elements in heap
    int capacity;      // size of the array heap
};
```



```
template <class T>
```

```
MaxHeap<T>::MaxHeap (int theCapacity=10)
```

```
{ //constructor
```

```
    if (theCapacity < 1) throw “Capacity must be >= 1”;
```

```
    capacity = theCapacity;
```

```
    heapSize = 0;
```

```
    heap = new T[capacity+1]; //heap[0] not used
```

```
}
```

```
template <class T>
```

```
Inline T& MaxHeap<T>::Top()
```

```
{
```

```
    if (IsEmpty()) throw “The heap is empty”;
```

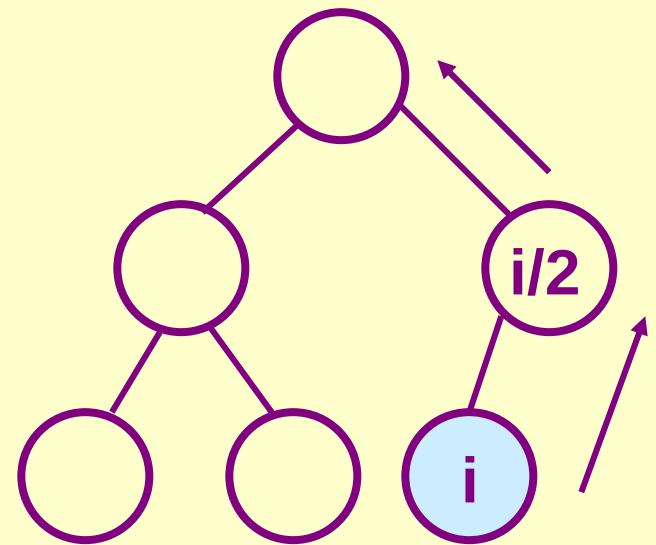
```
    return heap[1];
```

```
}
```

5.6.3 Insertion into a Max Heap

■ We use a **bubbling up process** to **insert** an element.

- ✓ Insert the new element at node $i = \text{heapSize} + 1$, and then compare its key with that of its parent. If its key is larger, it goes up to node $i/2$, and its parent (in node $i/2$) moves down to node i .
- ✓ The bubbling up process continues until the new element is in the right position.



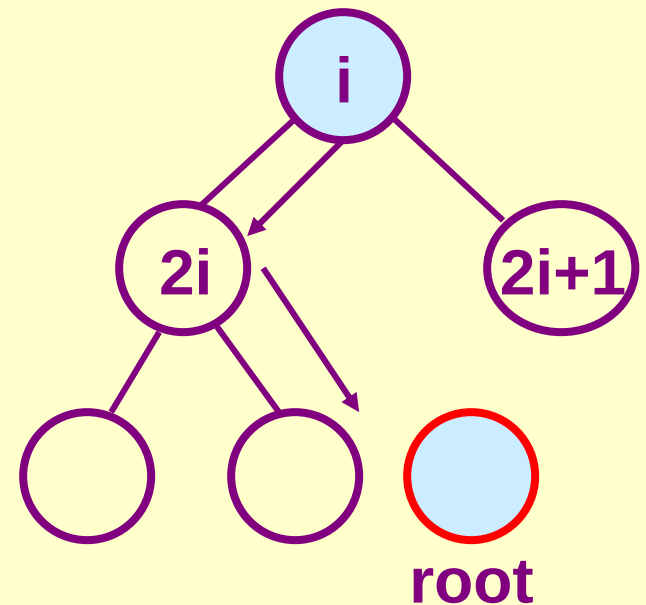
```
template <class T>
void MaxHeap<Type>::Push(const T& e)
{ // insert e into the max heap
    if (heapSize == capacity) { // double the capacity
        ChangeSize1D(heap, capacity, 2*capacity);
        capacity *= 2;
    }
    int currentNode = ++heapSize;
    while (currentNode != 1 && heap[currentNode/2] < e)
    { // bubble up
        heap[currentNode] = heap[currentNode/2];
        currentNode /= 2;
    }
    heap[currentNode] = e;
}
```

Analysis of Push:

A complete binary tree with n nodes has a height $\lceil \log_2(n+1) \rceil$, the while loop is iterated $O(\log n)$ times, and each takes $O(1)$ time, so the complexity is $O(\log n)$.

5.6.4 Deletion from a Max heap

- We Use **trickle down strategy** to delete the max element from the root (node 1).
- ✓ Swap the **root** element with the element in node heapSize **lastE**, and then heapSize--. Then, compare the key of lastE with that of its larger child. If its key is smaller, its larger child goes up, and it moves down.
- ✓ The **trickling down** process continues until lastE is in the right position.



```
template <class T>
void MaxHeap<T>::Pop()
{ // delete the max element.
    if (IsEmpty()) throw "Heap is empty. Cannot delete.";
    heap[1].~T(); // delete the max

    // remove the last element from heap
    T lastE = heap[heapSize--]; //the last node

    // trickle down
    int currentNode = 1; // root
    int child = 2;      // left child of currentNode
```

```
while (child <= heapSize)
{
    // set child to the larger child of currentNode
    if (child < heapSize && heap[child] < heap[child+1]) child++;
    // child always points to the larger child

    // can we put lastE in currentNode?
    if (lastE >= heap[child]) break; // yes

    // no
    heap[currentNode] = heap[child]; // move child up
    currentNode = child; child *= 2; // move down a level
}
heap[currentNode] = lastE;
}
```

Analysis of Pop:

The height of a heap with n nodes is $\lceil \log_2(n+1) \rceil$, the while loop is iterated $O(\log n)$ times, and each takes $O(1)$ time, so the time complexity is $O(\log n)$.

5.7 Binary Search Trees

5.7.1 Definition

- **Dictionary:** A dictionary is a collection of pairs, each pair has a key and an associated element.
- Although naturally occurring dictionaries may have several pairs with the same key, we assume here that no two pairs have the same key.

- The **pair** can be defined as:

```
template <class K, class E>
struct pair
{
    K first;
    E second;
};
```

The specification of a dictionary is:

ADT 5.3: A dictionary

```
template <class K, class E>
```

```
class Dictionary {
```

```
public:
```

```
    virtual bool IsEmpty () const = 0;
```

```
    // return true iff the dictionary is empty
```

```
    virtual pair<K,E>* Get(const K&) const = 0;
```

```
    // return pointer to the pair with specified key;
```

```
    // return 0 if no such pair
```

```
    virtual void Insert(const pair<K,E>&) = 0;
```

```
    // insert the given pair; if key is a duplicate
```

```
    // update associated element
```

```
    virtual void Delete(const K&) = 0;
```

```
    // delete pair with specified key
```

```
};
```

■ **Definition:** A binary search tree is a binary tree. It may be empty. If it not empty, satisfies the following properties:

(1) Every element has a key, and no two elements have the same key.

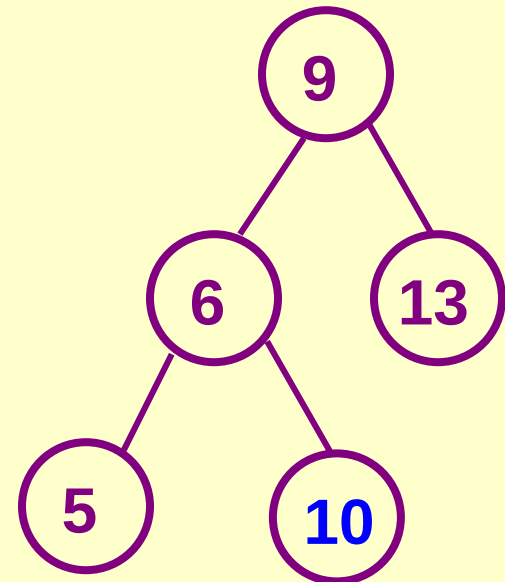
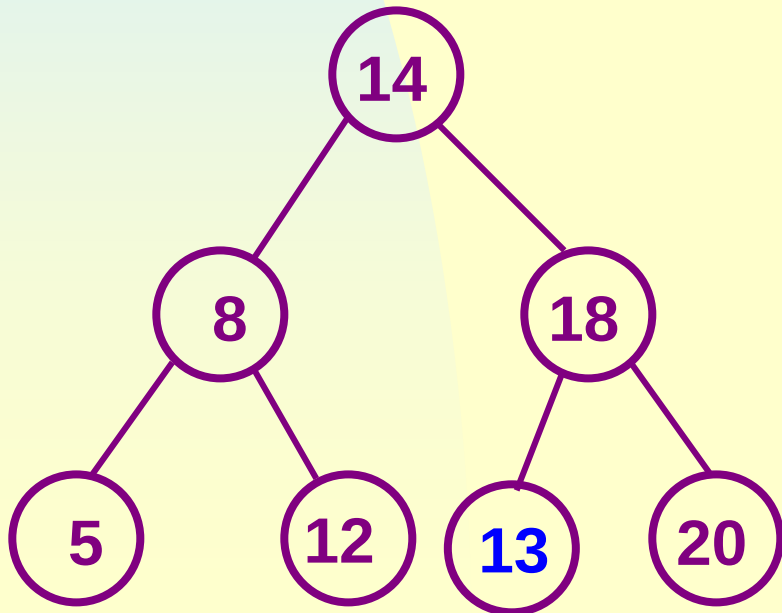
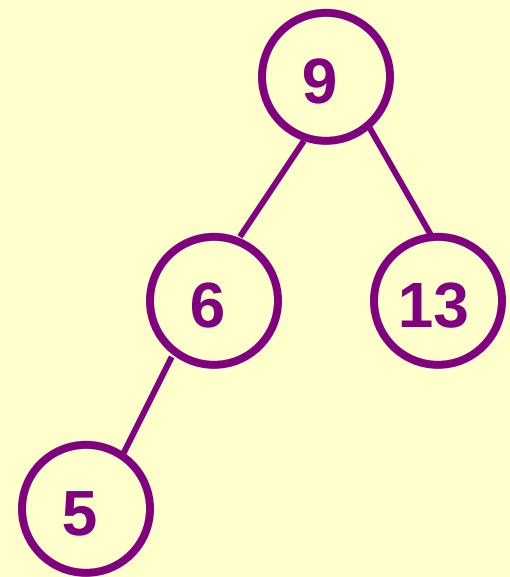
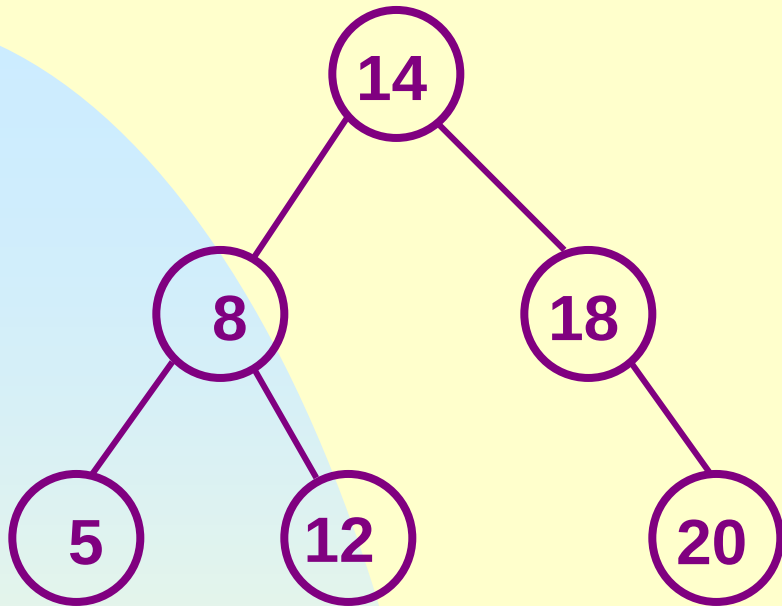
(2) The keys (if any) in the left subtree are smaller than the key in the root.

(3) The keys (if any) in the right subtree are larger than the key in the root.

(4) The left and right subtrees are also binary search trees.

■ **Note:** properties (2), (3), and (4) together imply that the keys must be distinct. So, property (1) can be replaced by: The root has a key.

Which are binary search trees?



5.7.2 Searching a Binary Search Tree

- Since the definition of a binary search tree is recursive, it is easy to describe **a recursive search method**. Suppose we search for an element with key k , we get the method:
 - ✓ If $k ==$ the key in **root**, success;
 - ✓ If $k <$ the key in **root**, search the left subtree;
 - ✓ If $k >$ the key in **root**, search the right subtree.
- Program 5.18 in the next slide gives a recursive search algorithm. This function assumes a **linked representation** for the search tree, which is assumed to be an object of the class **BST**. **BST** derives from the class `Tree<pair<K,E>>`.

Program 5.18: Recursive search of a binary search tree

```
template <class K, class E> // Driver
pair<K,E>* BST<K,E>::Get(const K& k)
{ // Search *this for a pair with key k.
    return Get(root, k);
}

template <class K, class E> // Workhorse
pair<K,E>* BST<K,E>::Get(treeNode<pair<K,E>>* p,
                        const K& k)
{
    if (!p) return 0;
    if (k < p->data.first) return Get(p->leftChild, k);
    if (k > p->data.first) return Get(p->rightChild, k);
    return &p->data;
}
```

The recursive version can be easily changed into an **iterative one** as in the following:

```
template <class K, class E> // Iterative version
pair<K,E>* BST<K,E>::Get(const K& k)
{
    TreeNode<pair<K,E>>* currentNode = root;
    while (currentNode)
        if (k < currentNode->data.first)
            currentNode = currentNode->leftChild;
        else if (k > currentNode->data.first)
            currentNode = currentNode->rightChild;
        else return &currentNode->data;
    // no matching pairs
    return 0;
}
```

Search by rank

- We define the **rank** of a node to be its **position in inorder**;
- If we wish to search by rank, each node should have an additional field **leftSize**, which is **one plus the number of elements in the left subtree of the node**.
- Program 5.20 in the next slide searches for the **rth smallest element**, where we assume that **TreeNode** has an additional field called **leftSize**.

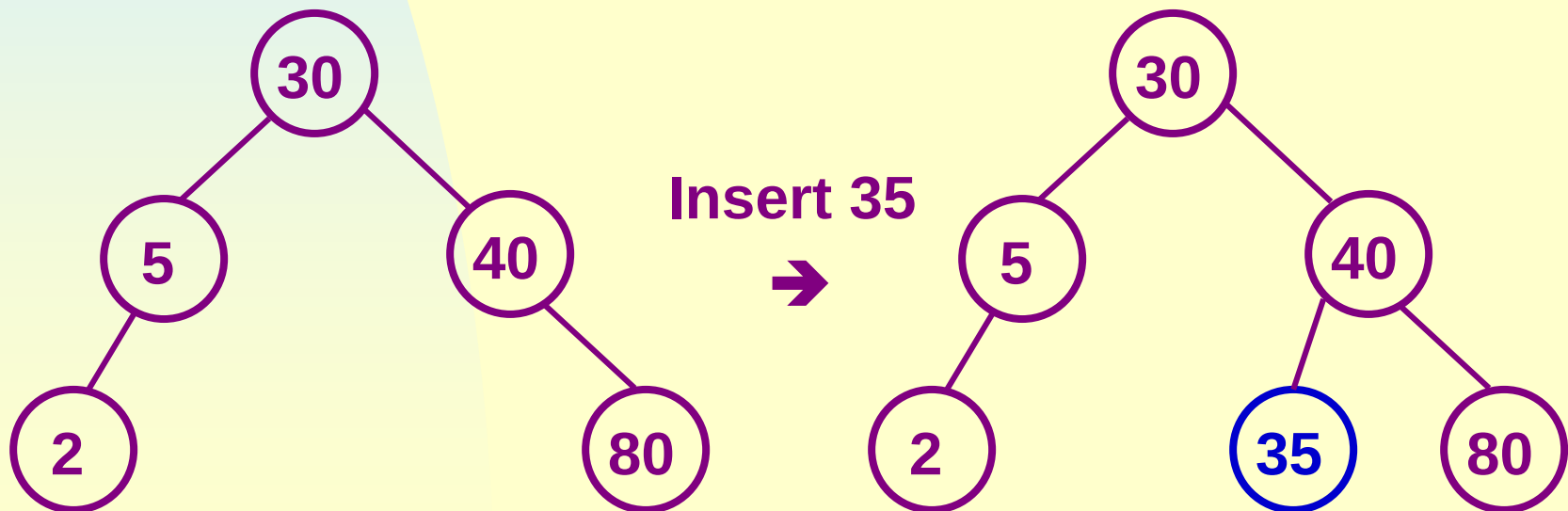
Program 5.20: Searching a binary search tree by rank

```
template <class K, class E> // search by rank
pair<K,E>* BST<K,E>::RankGet(int r)
{
    TreeNode<pair<K,E>>* currentNode = root;
    while (currentNode)
        if (r < currentNode→leftsize)
            currentNode = currentNode→leftChild;
        else if (r > currentNode→ leftsize)
        {
            r-=currentNode→ leftsize;
            currentNode = currentNode→rightChild;
        }
        else return &currentNode→data;
    return 0;
}
```

As can be seen, a binary search tree of height h can be searched by key as well as by rank in $O(h)$ time.

5.7.3 Insertion into a Binary Search Tree

- To insert a pair (k, e) , a search is carried out by k .
- ✓ If unsuccessful, then the element is inserted at the point the search terminated.
 - ✓ When the dictionary already contains a pair with key k , we simply update the element associated with this key to e .



```

template <class K, class E>
void BST<K,E>::Insert(const pair<K,E>& thePair)
{ // Insert thePair into the binary search tree
    // search for thePair.first, pp is parent of p
    TreeNode<pair<K,E>> *p=root, *pp=0;
    while (p) {
        pp=p;
        if (thePair.first < p→data.first) p=p→leftChild;
        else if (thePair.first > p→data.first) p=p→rightChild;
        else // duplicate, update associated element
            {p→data.second=thePair.second;return;}
    }
    // perform insertion
    p=new TreeNode<pair<K,E>>(thePair,0,0);
    if (root) // tree not empty
        if (thePair.first < pp→data.first) pp→leftChild=p;
        else pp→rightChild=p;
    else root=p;
}

```

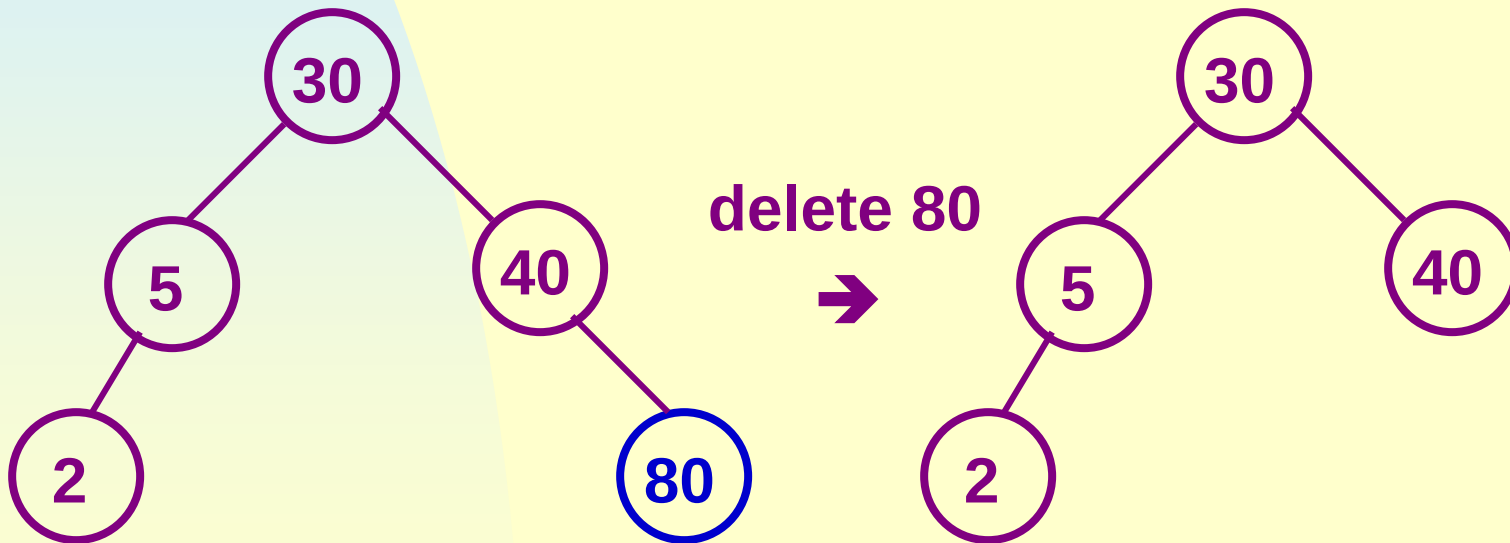
Analysis of Insert:

- ✓ The time is $O(h)$, where h is the height of the search tree.
- ✓ If a node has a **leftsize** field, then this is to be updated too.

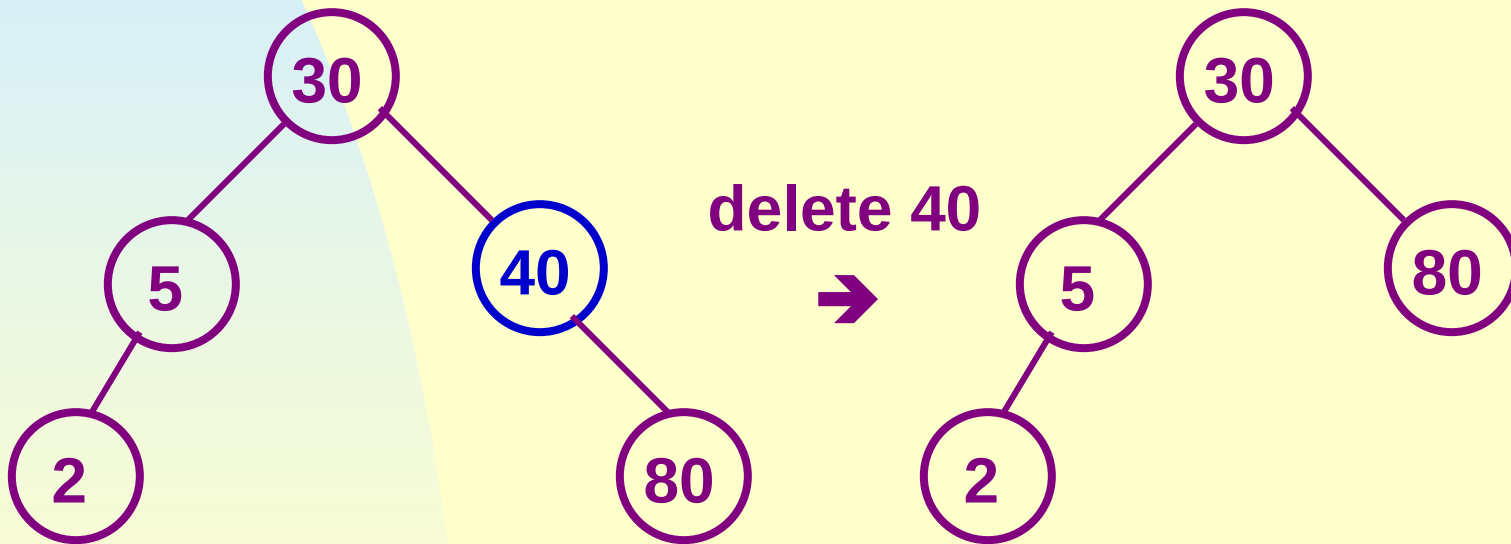
5.7.4 Deletion from a Binary Search Tree

There are 3 cases:

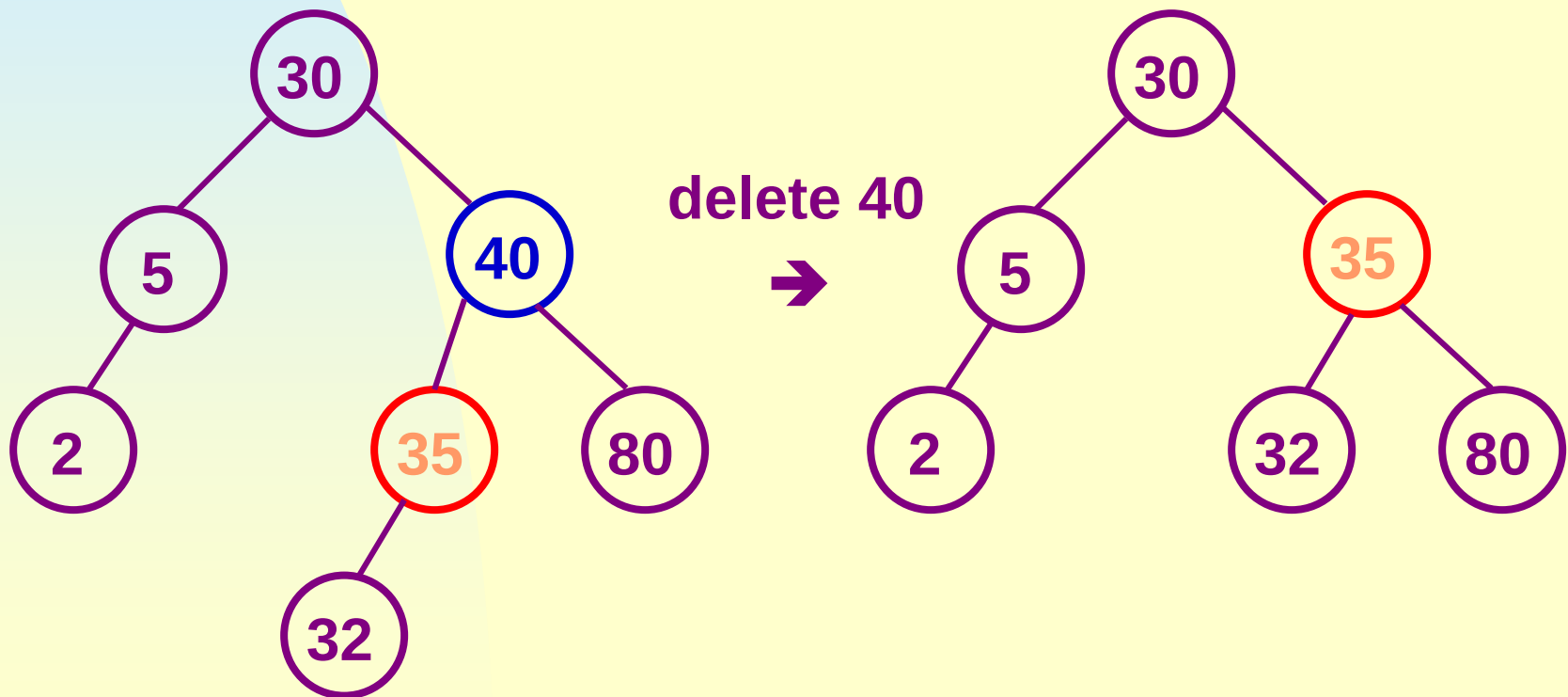
(1) deletion of a leaf: easy, set the corresponding child field of its parent to 0 and dispose the node.



(2) deletion of a nonleaf node with one child: easy, the child of the node takes the place of it and the node is disposed.



(3) deletion of a nonleaf node with two children:
the element is replaced by either the largest elements in its left subtree or the smallest element in its right subtree. Then delete that node, which has at most one child, in the subtree.



The time to delete a node is $O(h)$.

5.8 Selection Trees

5.8.1 Introduction

- Suppose we have $k(k > 2)$ ordered sequences, called **runs**, and each run consists of some records and is in **nondecreasing order** of a designated field called the **key**. They are to be merged into a single ordered sequence.
- The merging task can be accomplished by repeatedly outputting the record with the **smallest key**. It must be one of the leading records in all the k runs.
- The most direct way to find this record is to make **$k-1$ comparisons**.

- If we use the **selection tree** data structure, we can reduce the **number of comparisons** to find the smallest record.
- There are two kinds of selection trees: **winner tree** and **loser tree**.

5.8.2 Winner trees

Definition: A winner tree is a **complete binary tree** in which each node represents the **smaller (winner)** of its two children.

We can construct a winner tree by **playing of a tournament between nodes:**

- ✓ **Leaf node**---the first record in the each run.
- ✓ **Nonleaf node**---the winner of a tournament. The root represents the overall winner (the smallest key).

■ Some conventions and assumptions

- ✓ Generally, the **records** being merged are **large**, so each tree node will contain **only a pointer to the record**.
- ✓ For convenience, here we assume each node contains the **key value of a record** rather than a **pointer to the record**.
- ✓ A winner tree may be represented using the **sequential allocation scheme** for binary trees.
- ✓ Assume leaf nodes is recorded from **buffer[0]** to **buffer[k-1]**, where **k** is the number of runs. Thus, the node number of a leaf node from **buffer[i]** in the winner tree is **k+i**.

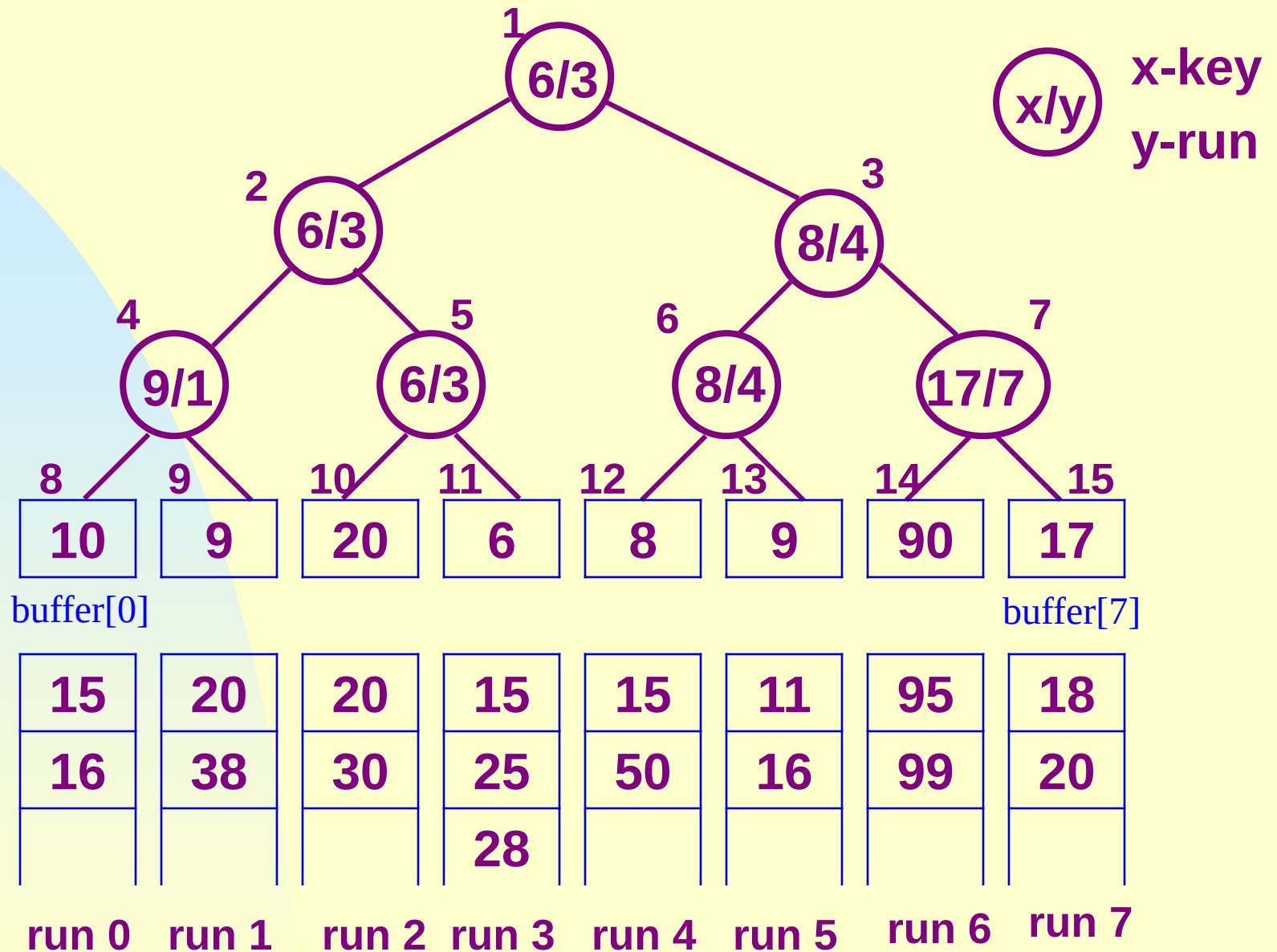


Figure 5.31: Winner tree for $k=8$

How to restructure a winner tree after the overall winner output?

- ✓ The first record which has the same run number as the overall winner enter the tree as the new leaf node.
- ✓ Then every two siblings along the path from the new entered node to the root node play the tournaments, and the result put in the parent node.
- ✓ The time to reconstruct the tree is $O(\log_2 k)$.
- ✓ The time to merge all n records is thus $O(n \log_2 k)$.

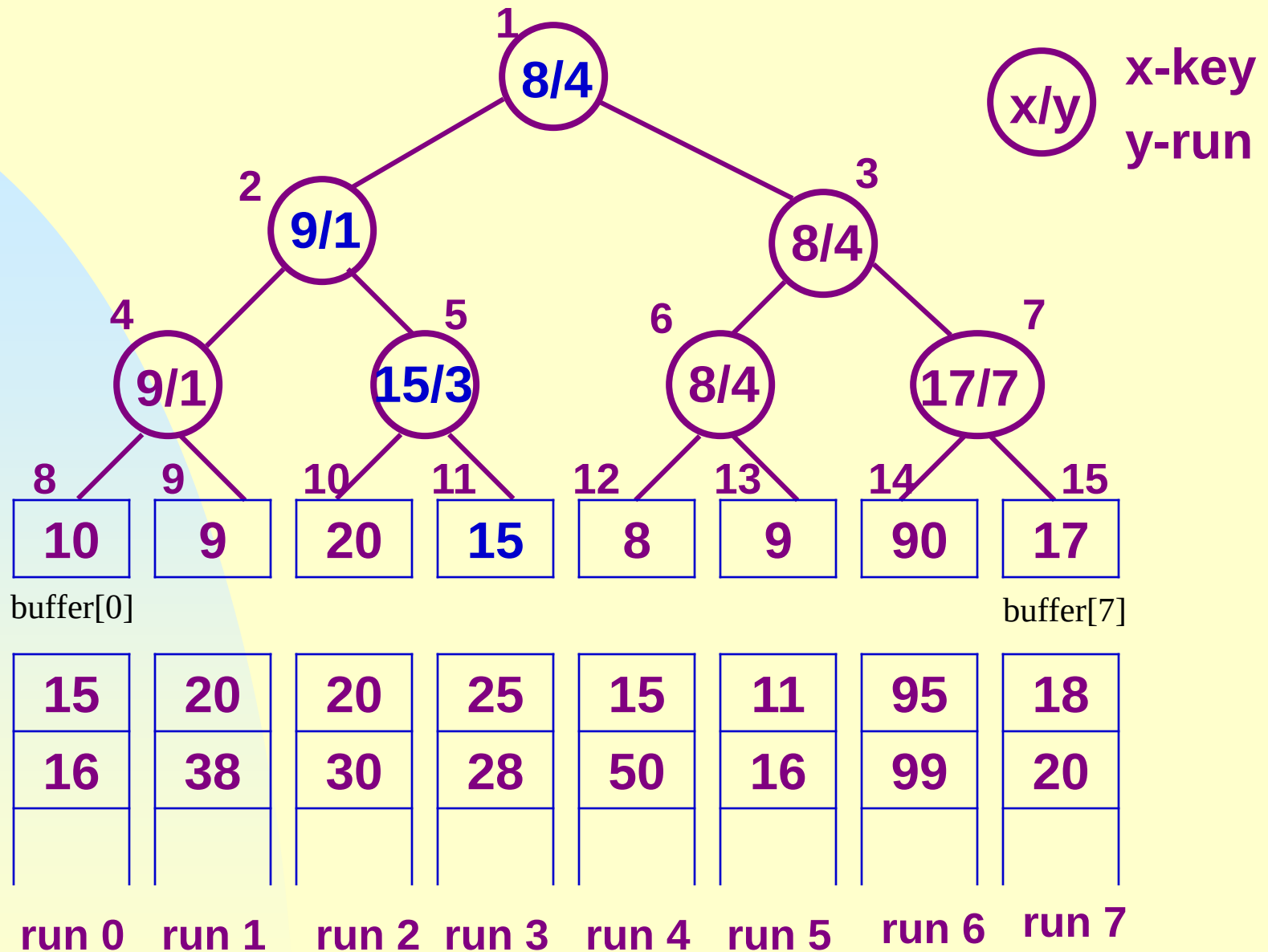


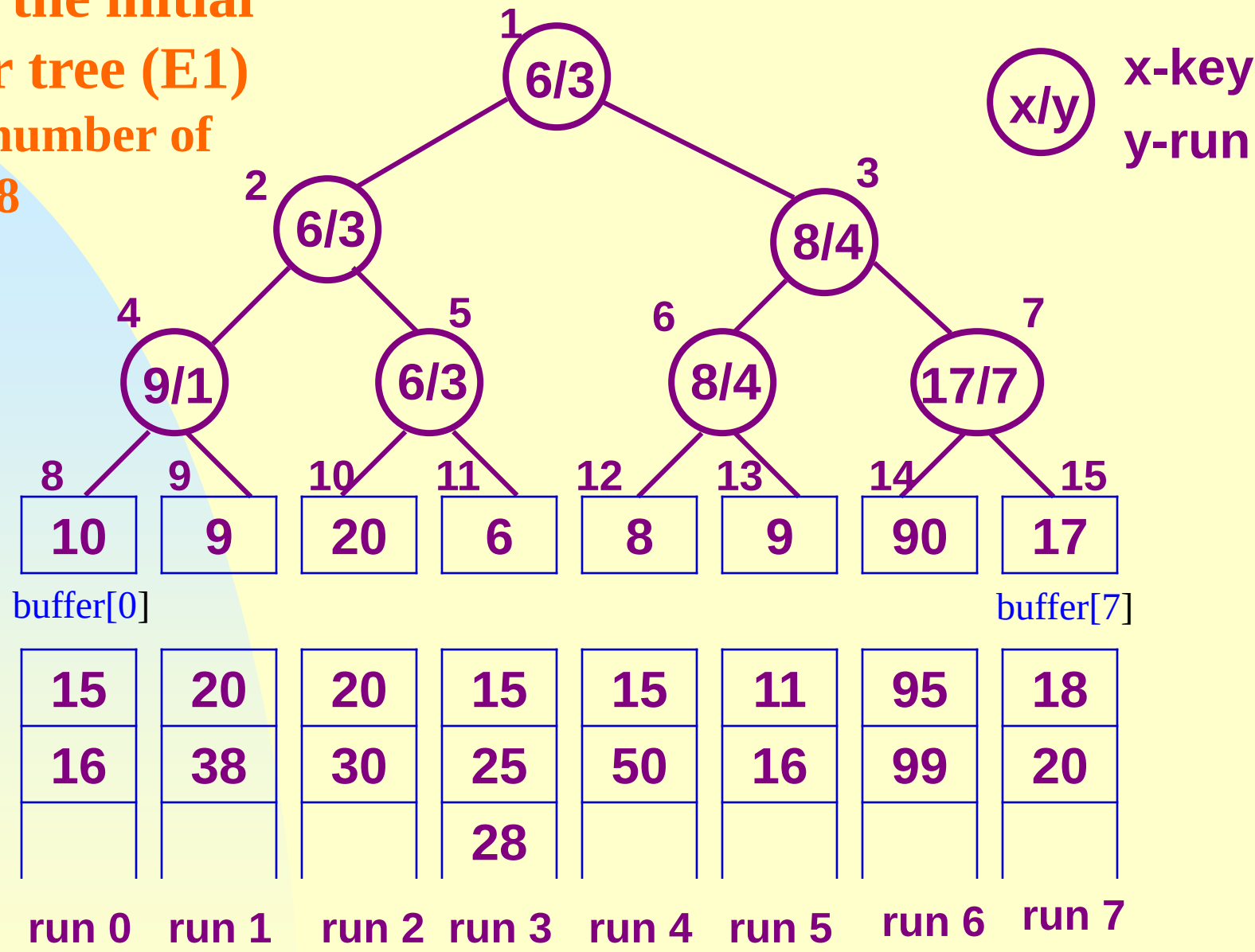
Figure 5.32: Winner tree of Figure 5.31 after one record has been output and the tree reconstructed

How to set up the initial winner tree?

- ✓ If there are k runs, we set up an array $a[2k]$ to express a complete tree with $2k-1$ nodes ($a[0]$ is not used), and k of which are leaves.
- ✓ Then from rear to front, we compare the element $a[2k-1]$ with $a[2k-2]$ to store the winner at $a[k-1]$, then compare the element $a[2k-3]$ with $a[2k-4]$ to store the winner at $a[k-2]$,... At last, we compare the element $a[3]$ with $a[2]$ to store the final winner at $a[1]$ to set up an initial winner tree.
- ✓ Obviously, there are $k-1$ times of comparison to complete the task. So the time complexity is $O(k)$.

Set up the initial winner tree (E1)

---The number of runs is 8

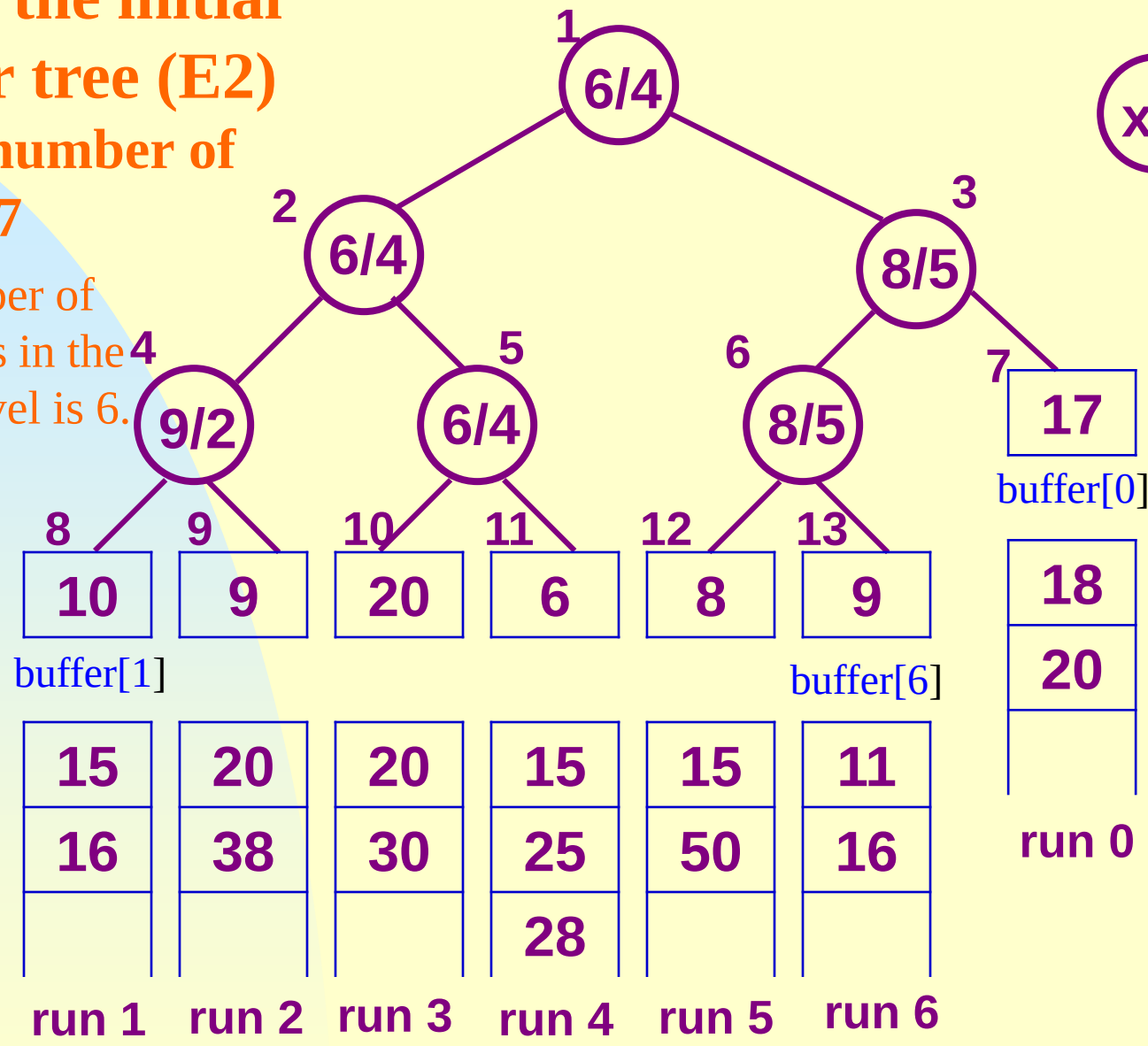


Set up the initial winner tree (E2)

---The number of runs is 7

The number of leaf nodes in the lowest level is 6.

$\bigcirc$ x-key
 $\square$ y-run



Class exercises

Question 1: A winner tree which has 500 leaf nodes, how many nodes does it have? What is the height of it? How many nodes does it have in the lowest level?

Question 2: There are 6 keys {1,2,3,4,5,6}, please construct the winner tree according to these keys.

5.8.3 Loser Trees

In the winner tree, we can see:

- ✓ After the record with the smallest key is output, and the next record is inserted into the tree, **tournaments are played between sibling nodes along the path from the inserting node to the root.**

To improve:

- ✓ In each nonleaf node, if we keep **the pointer to (or the key of) the loser** rather than the pointer to (or the key of) the winner of tournament, we can simplify the restructuring process. Thus we get a loser tree.

Definition: A selection tree in which each nonleaf node retains a pointer to the loser is called a **loser tree**.

In loser tree, an additional node, **node 0**, is added to represent the **overall winner** of the tournament.

The next slide shows a **loser tree** corresponding the previous winner tree.

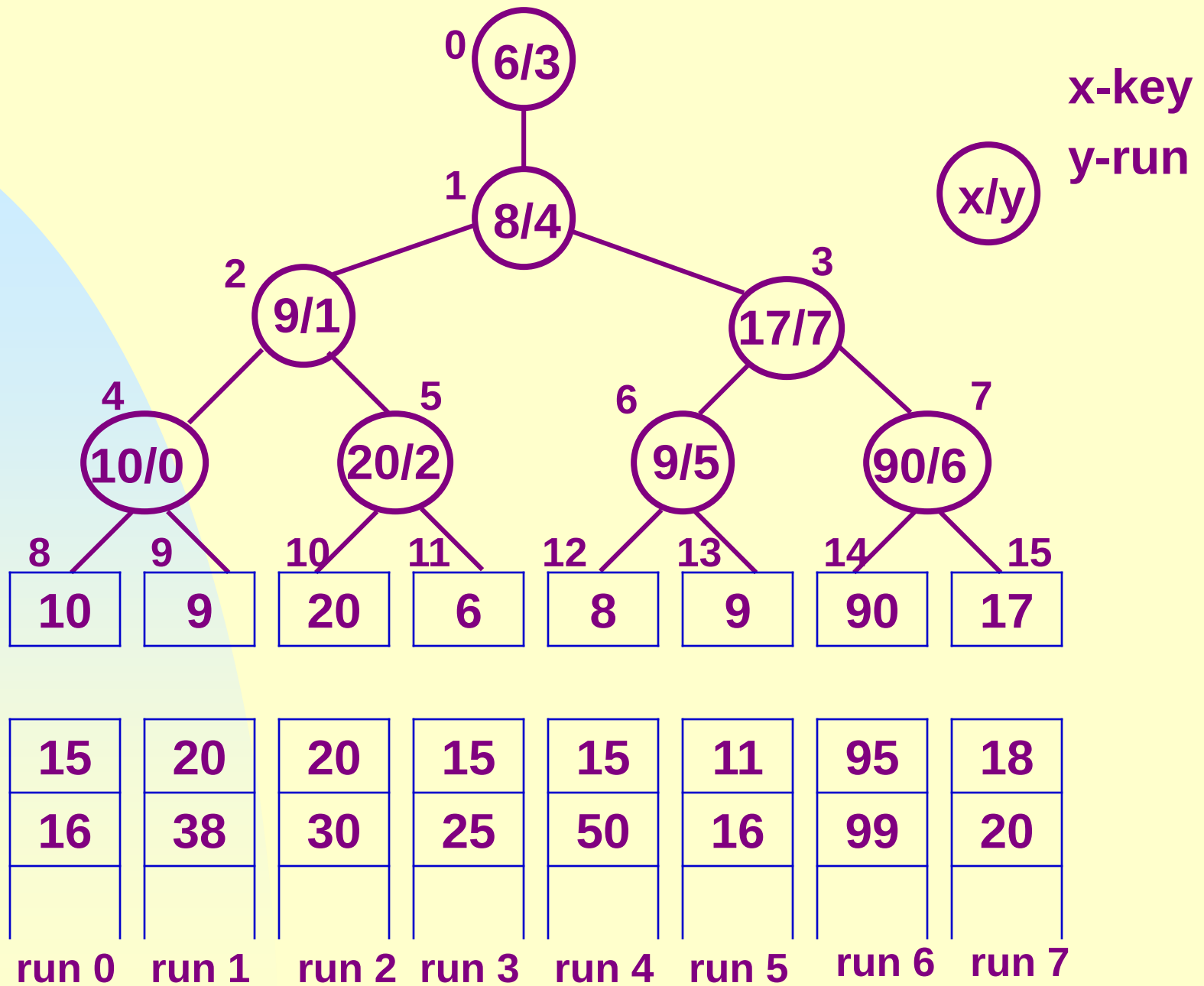
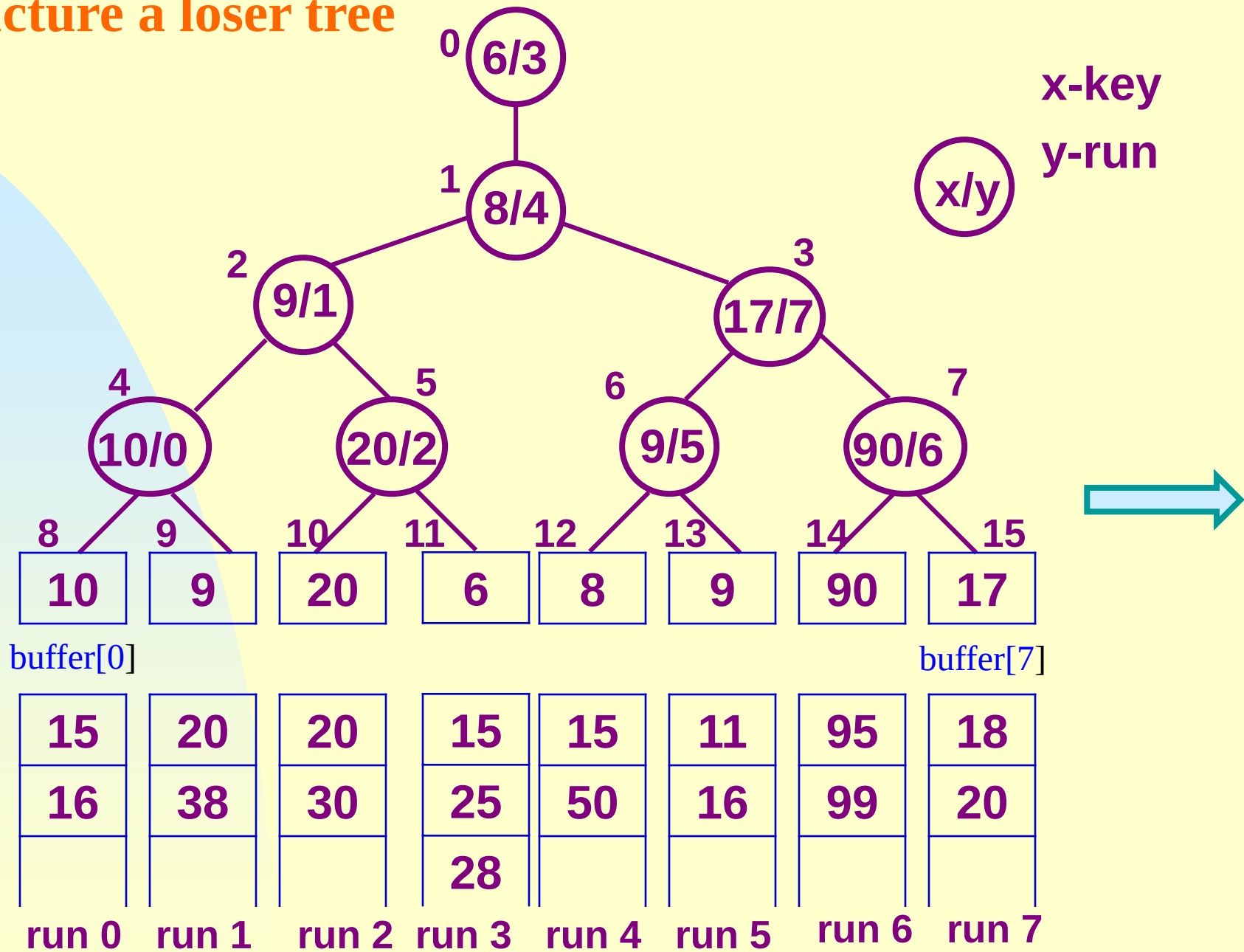


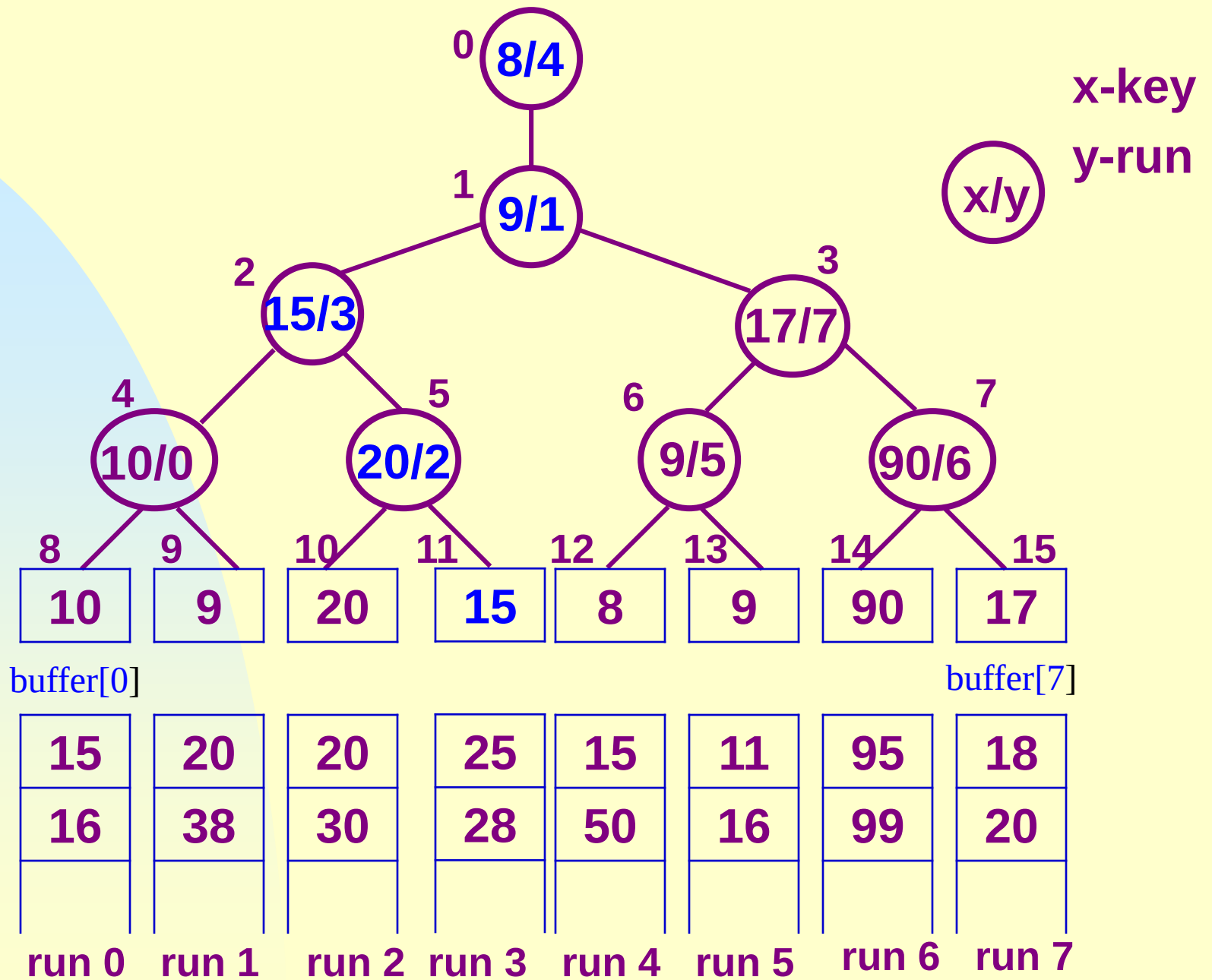
Fig. 5.33: **Loser tree** corresponding the previous winner tree

How to restructure a loser tree after the overall winner output?

- ✓ The first record which has the same run number as the overall winner enter the tree as the new leaf node.
- ✓ Then tournaments are played along the path from the new entered node to node 1.
- ✓ Note: tournament is played between parent and child nodes **not between sibling nodes**.
- ✓ The time to reconstruct the tree is $O(\log_2 k)$.
- ✓ The time to merge all n records is thus $O(n \log_2 k)$.

Restructure a loser tree





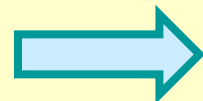
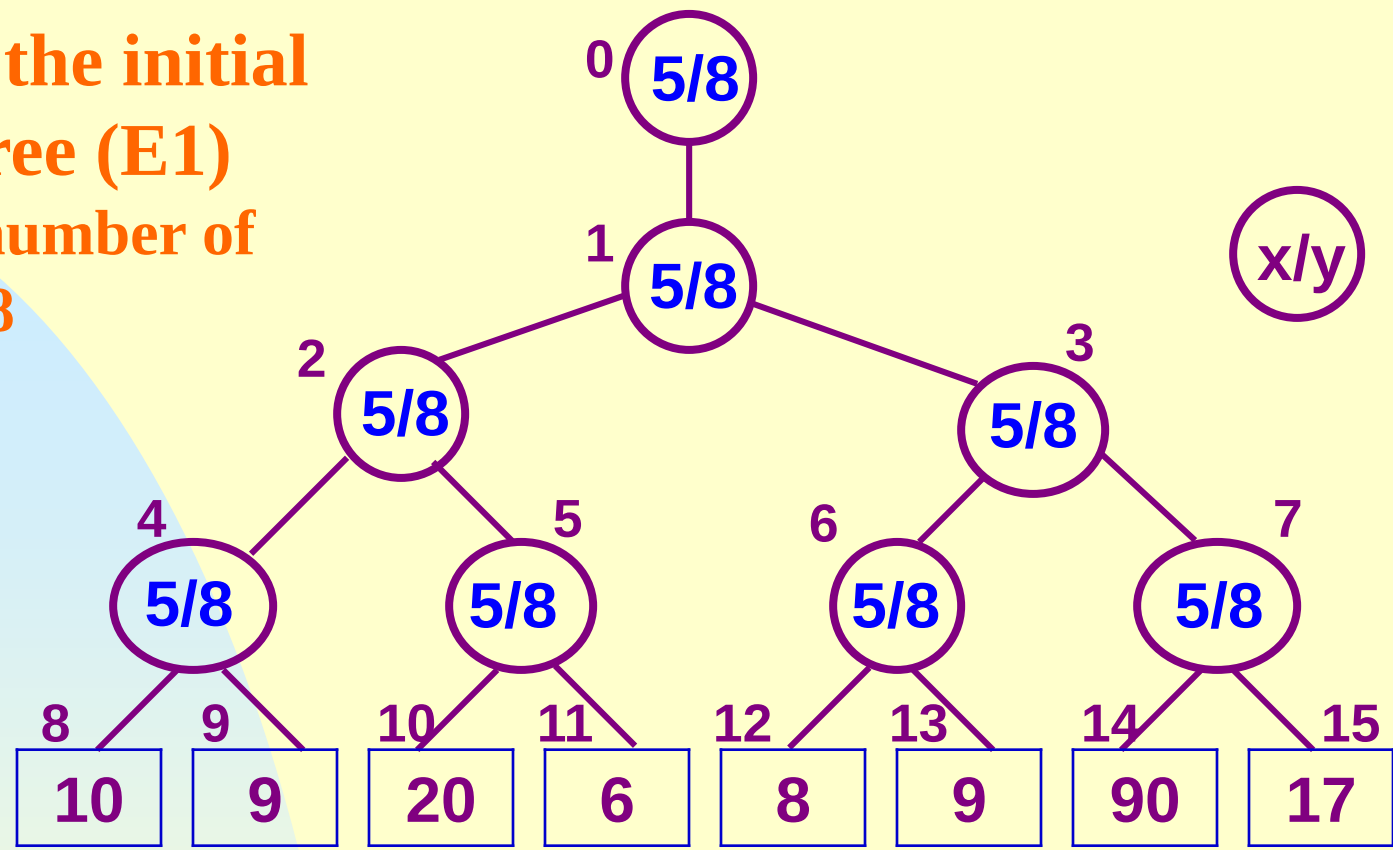
How to set up the initial loser tree?

- ✓ If there are k runs, we set up an array $a[2k]$ to express a complete tree with $2k-1$ nodes ($a[0]$ is used to keep the overall winner), and k of which are leaves.
- ✓ Next, we initialize $a[0], a[1], \dots, a[k-1]$ which are non-leaf nodes to the absolute minimal(winner) key or the run number k .
- ✓ Then we restructure the loser tree by viewing each record (leaf node) as the new entered node. That is, tournaments are played along the path from this new entered node to node 1. Once there is a node playing tournament with the one that has the absolute minimal(winner) key, the restructuring process for this new record stops.

- ✓ **Note that: the orders to process the records are irrelevant. We can process them from front to rear, or from rear to front, or with other orders.**
- ✓ **The time to initialize the non-leaf nodes is $O(k)$.**
- ✓ **The total time to set up a loser tree is also $O(k)$.**

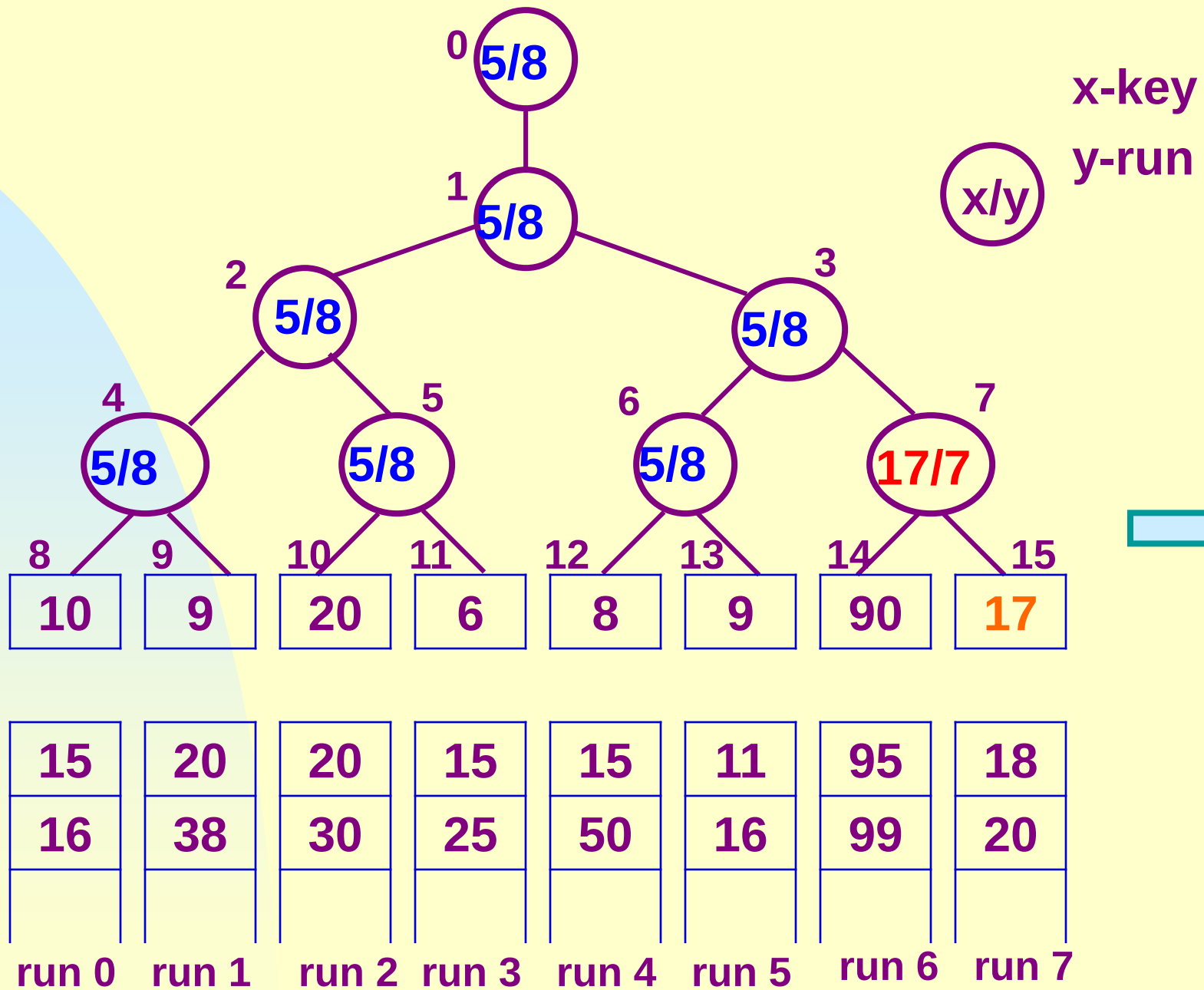
Set up the initial loser tree (E1)

---The number of runs is 8

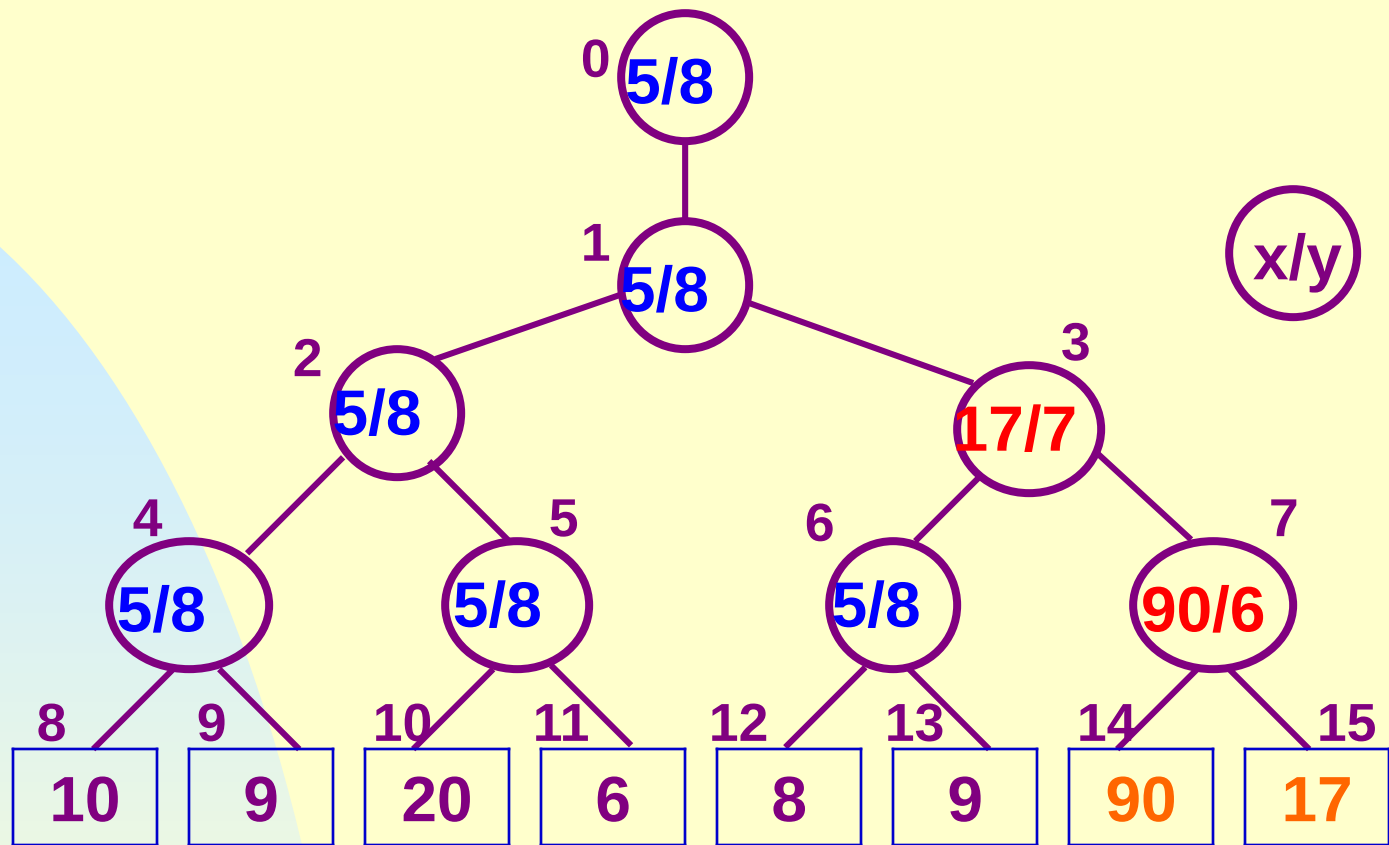


15	20	20	15	15	11	95	18
16	38	30	25	50	16	99	20
run 0	run 1	run 2	run 3	run 4	run 5	run 6	run 7

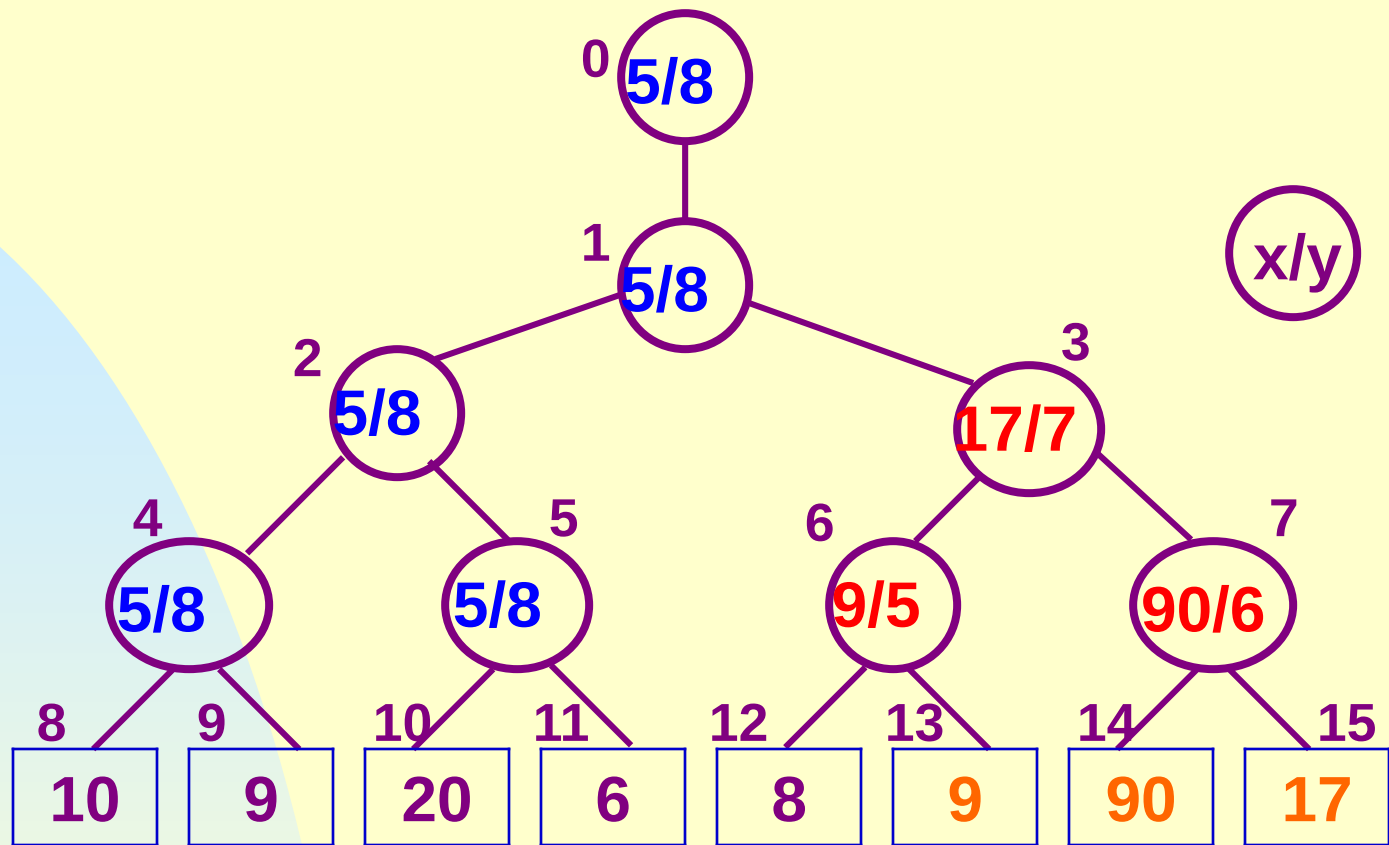
Initializing each non-leaf node to the absolute minimal key 5 or the run number 8.



Playing tournaments along the path from the leaf node to node 1.



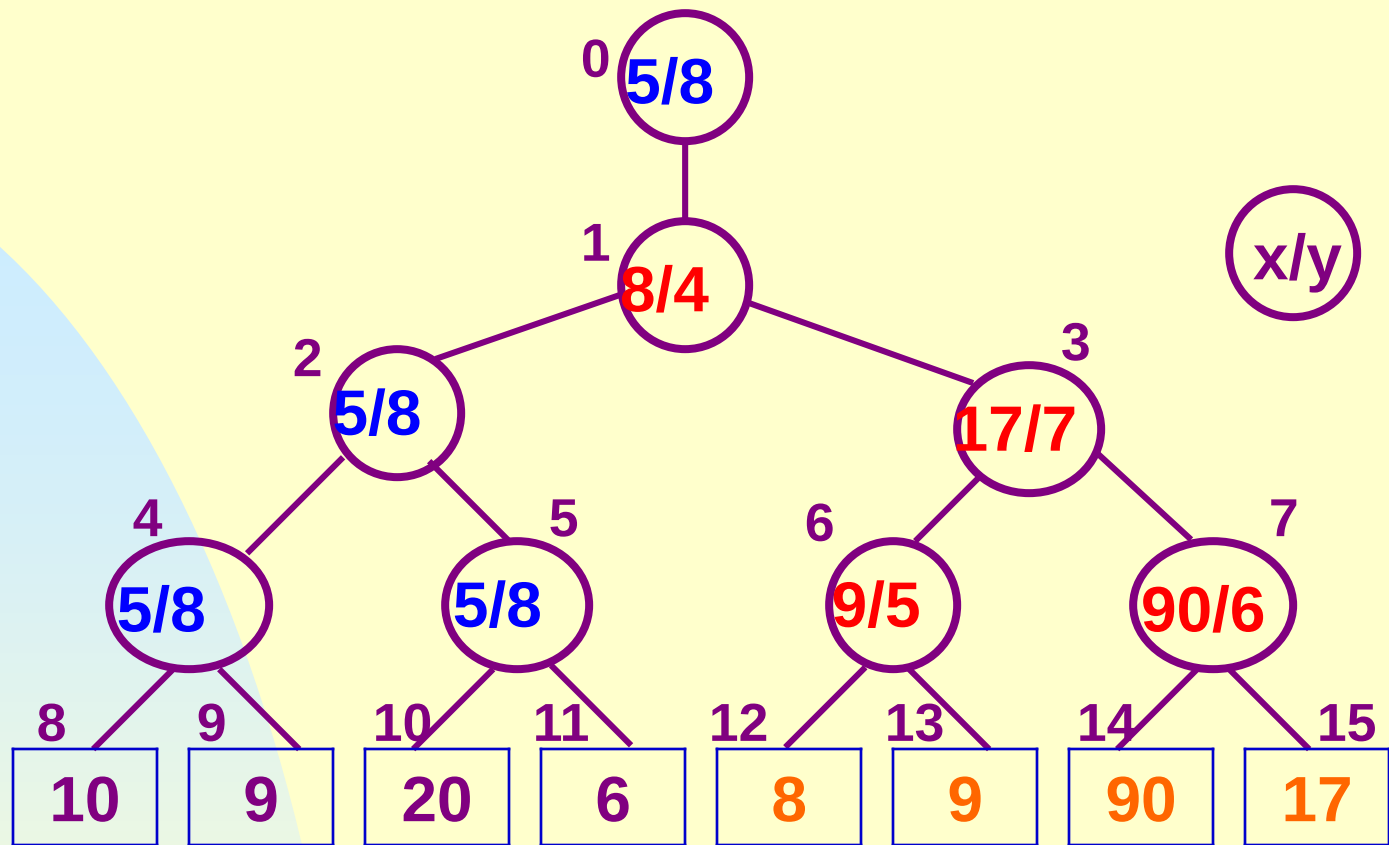
15	20	20	15	15	11	95	18
16	38	30	25	50	16	99	20
run 0	run 1	run 2	run 3	run 4	run 5	run 6	run 7



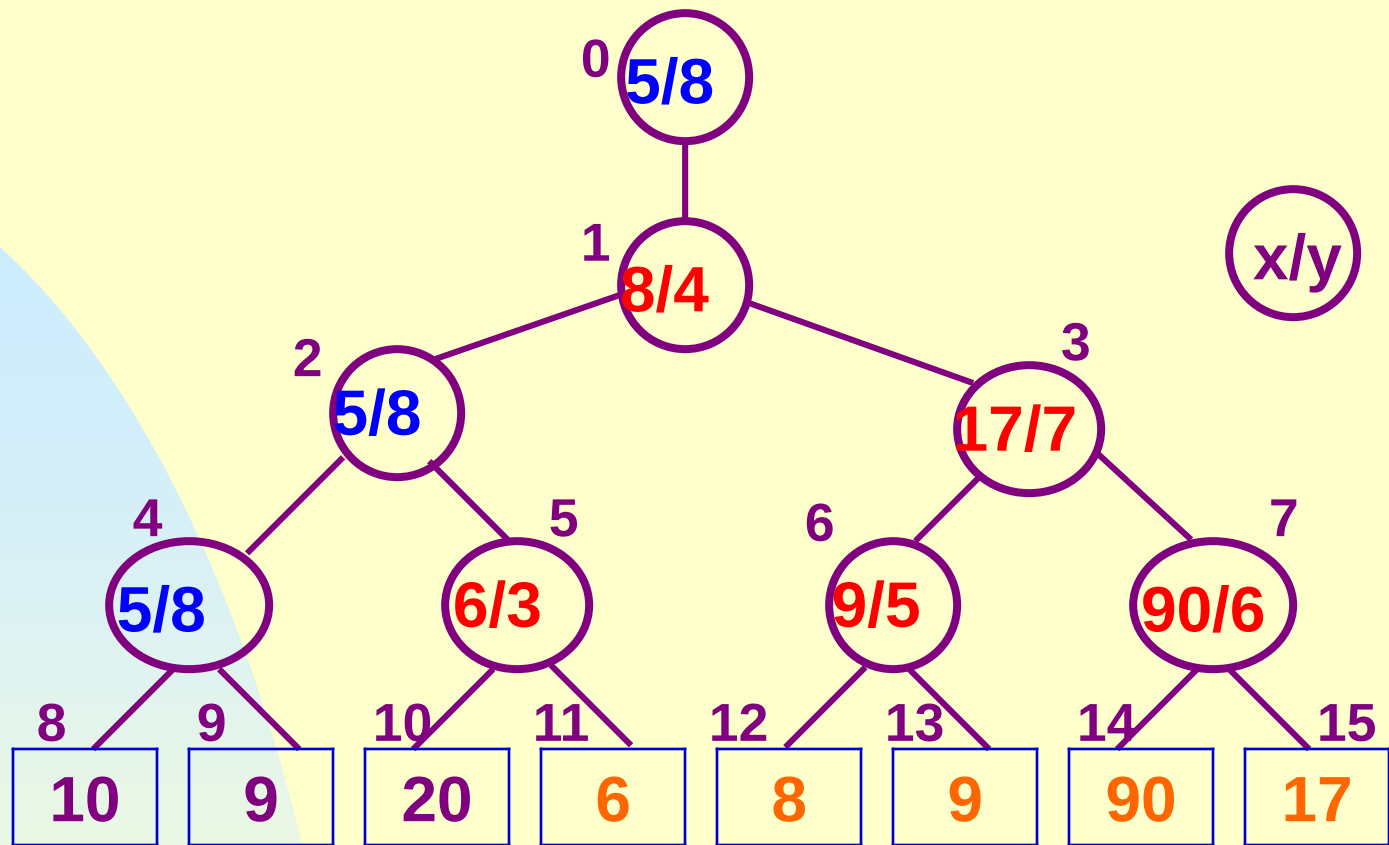
x-key
y-run



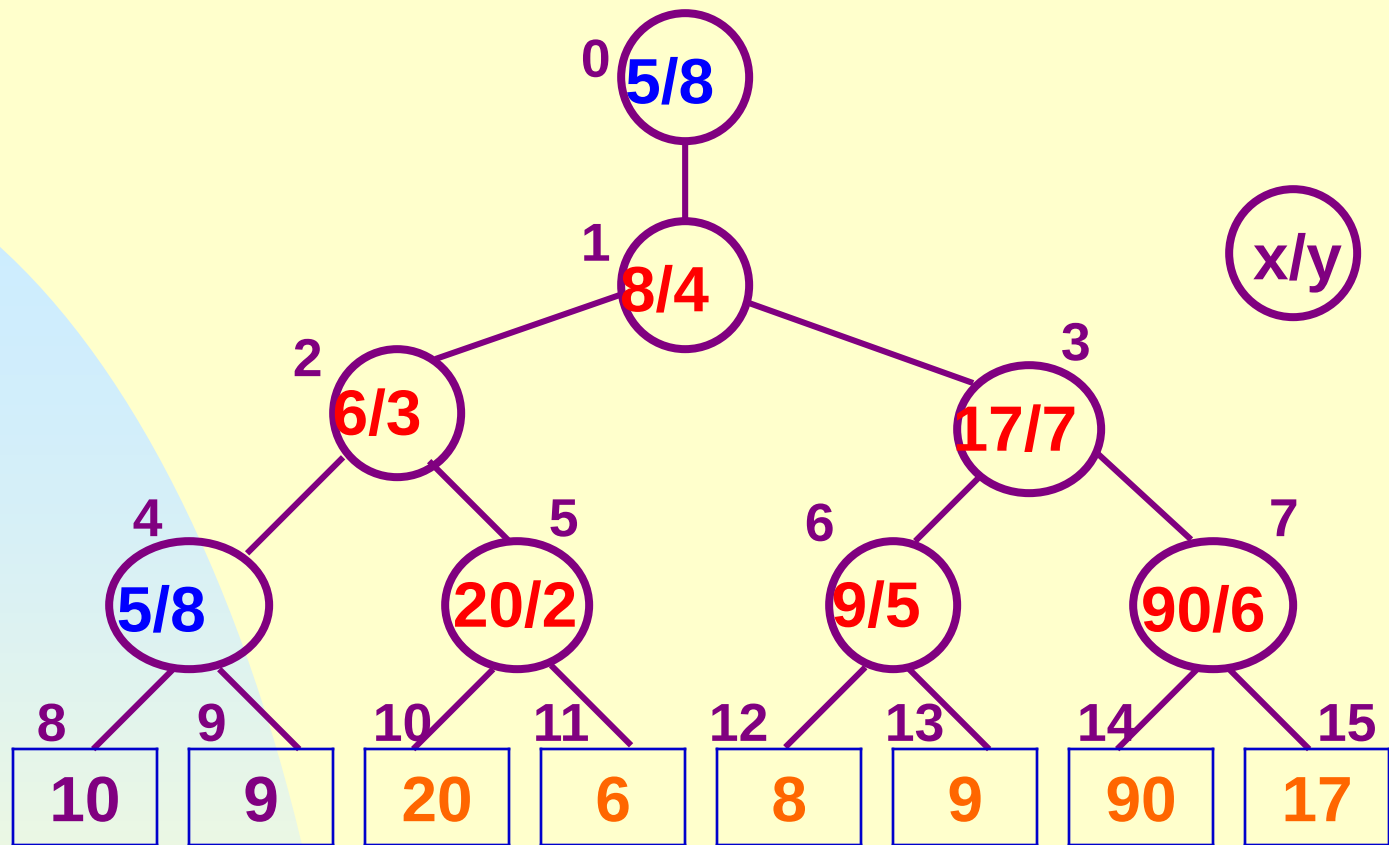
15	20	20	15	15	11	95	18
16	38	30	25	50	16	99	20
run 0	run 1	run 2	run 3	run 4	run 5	run 6	run 7



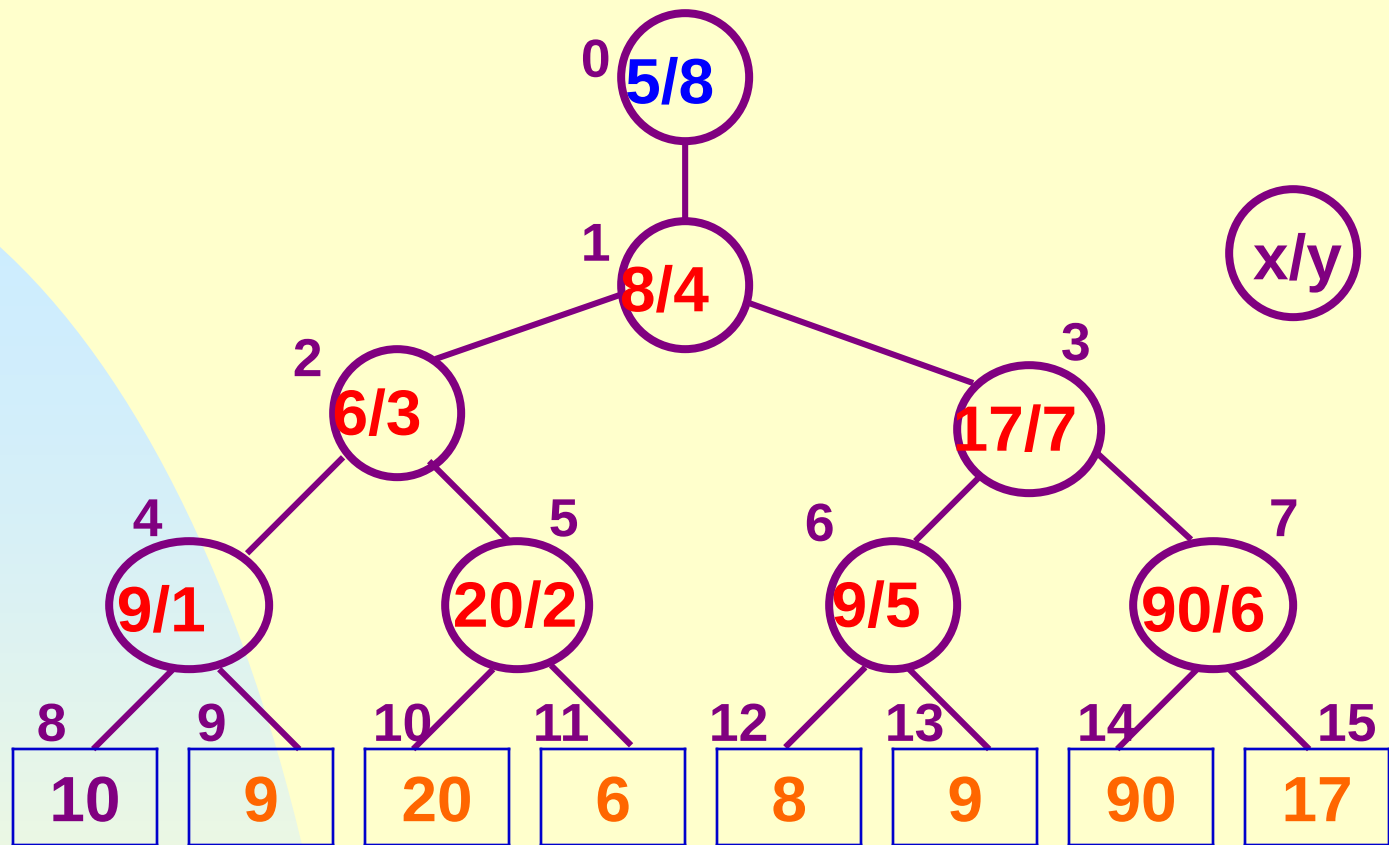
15	20	20	15	15	11	95	18
16	38	30	25	50	16	99	20
run 0	run 1	run 2	run 3	run 4	run 5	run 6	run 7



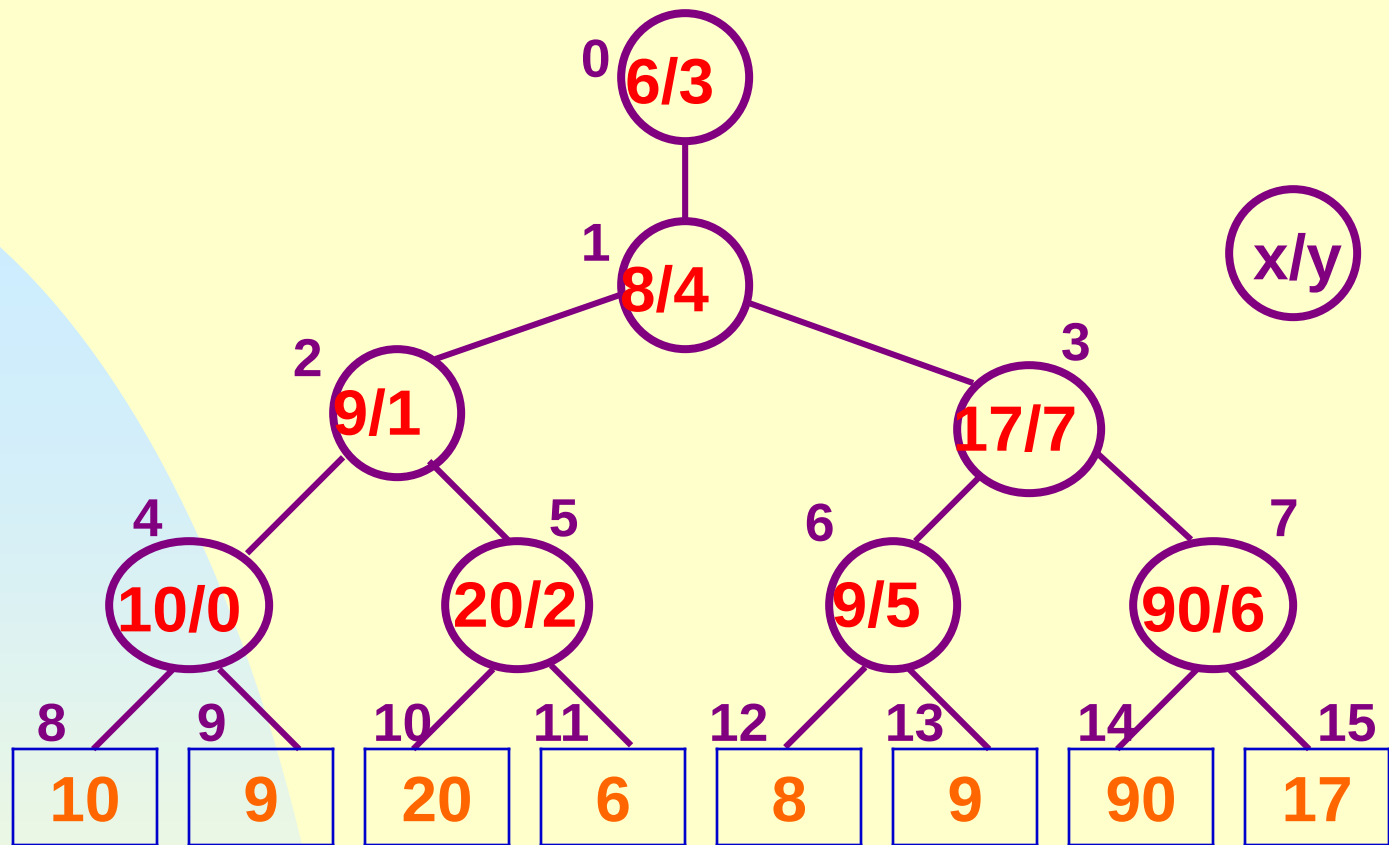
15	20	20	15	15	11	95	18
16	38	30	25	50	16	99	20
run 0	run 1	run 2	run 3	run 4	run 5	run 6	run 7



15	20	20	15	15	11	95	18
16	38	30	25	50	16	99	20
run 0	run 1	run 2	run 3	run 4	run 5	run 6	run 7



15	20	20	15	15	11	95	18
16	38	30	25	50	16	99	20
run 0	run 1	run 2	run 3	run 4	run 5	run 6	run 7



x-key
y-run



15	20	20	15	15	11	95	18
16	38	30	25	50	16	99	20
run 0	run 1	run 2	run 3	run 4	run 5	run 6	run 7

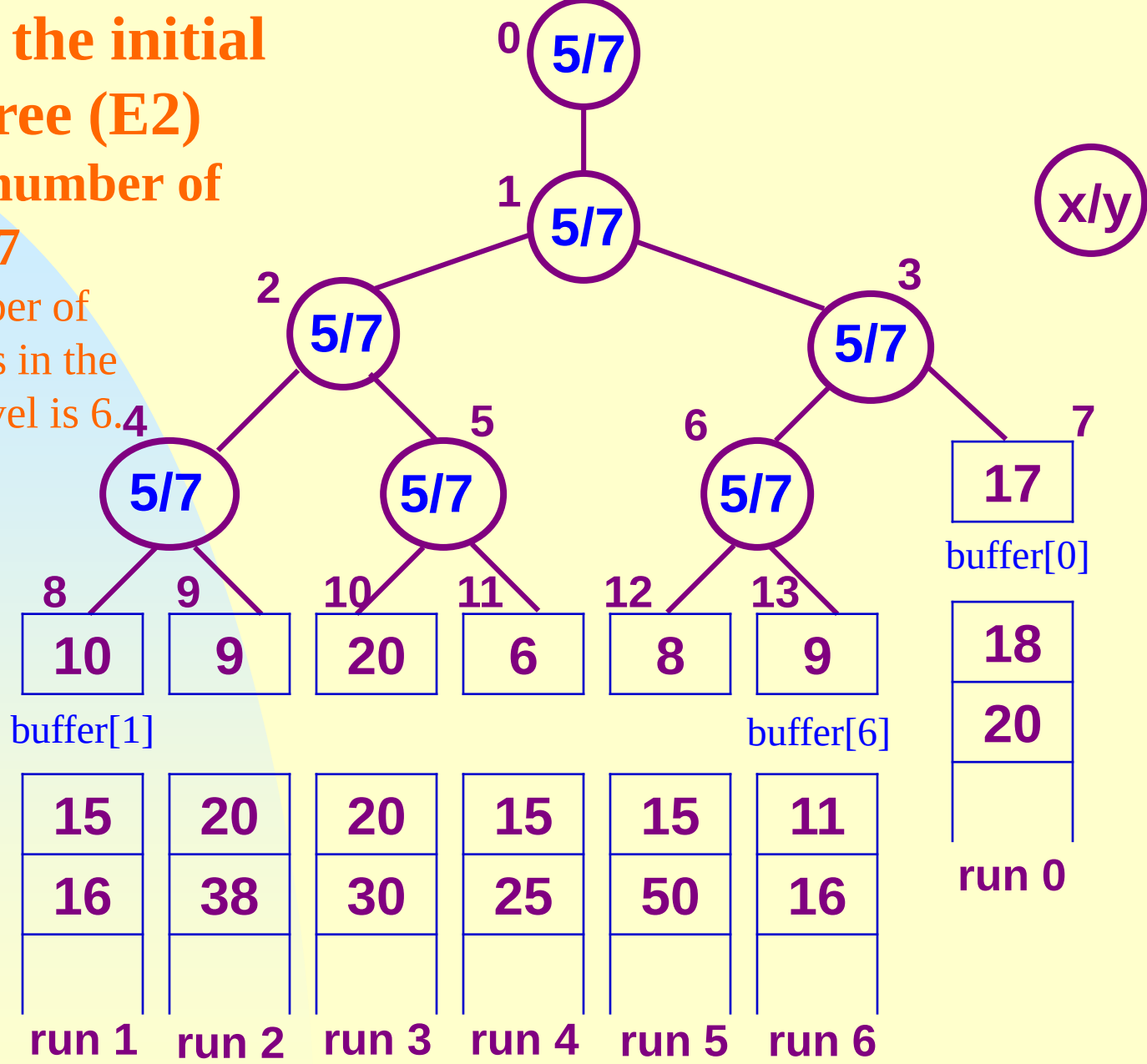
The changes of the array that represent the loser tree (E1)

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
loser tree	5	5	5	5	5	5	5	5	10	9	20	6	8	9	90	17
	5	5	5	5	5	5	5	17	10	9	20	6	8	9	90	17
	5	5	5	17	5	5	5	90	10	9	20	6	8	9	90	17
	5	5	5	17	5	5	9	90	10	9	20	6	8	9	90	17
	5	8	5	17	5	5	9	90	10	9	20	6	8	9	90	17
	5	8	5	17	5	6	9	90	10	9	20	6	8	9	90	17
	5	8	6	17	5	20	9	90	10	9	20	6	8	9	90	17
	5	8	6	17	9	20	9	90	10	9	20	6	8	9	90	17
	5	8	9	17	10	20	9	90	10	9	20	6	8	9	90	17

Set up the initial loser tree (E2)

---The number of runs is 7

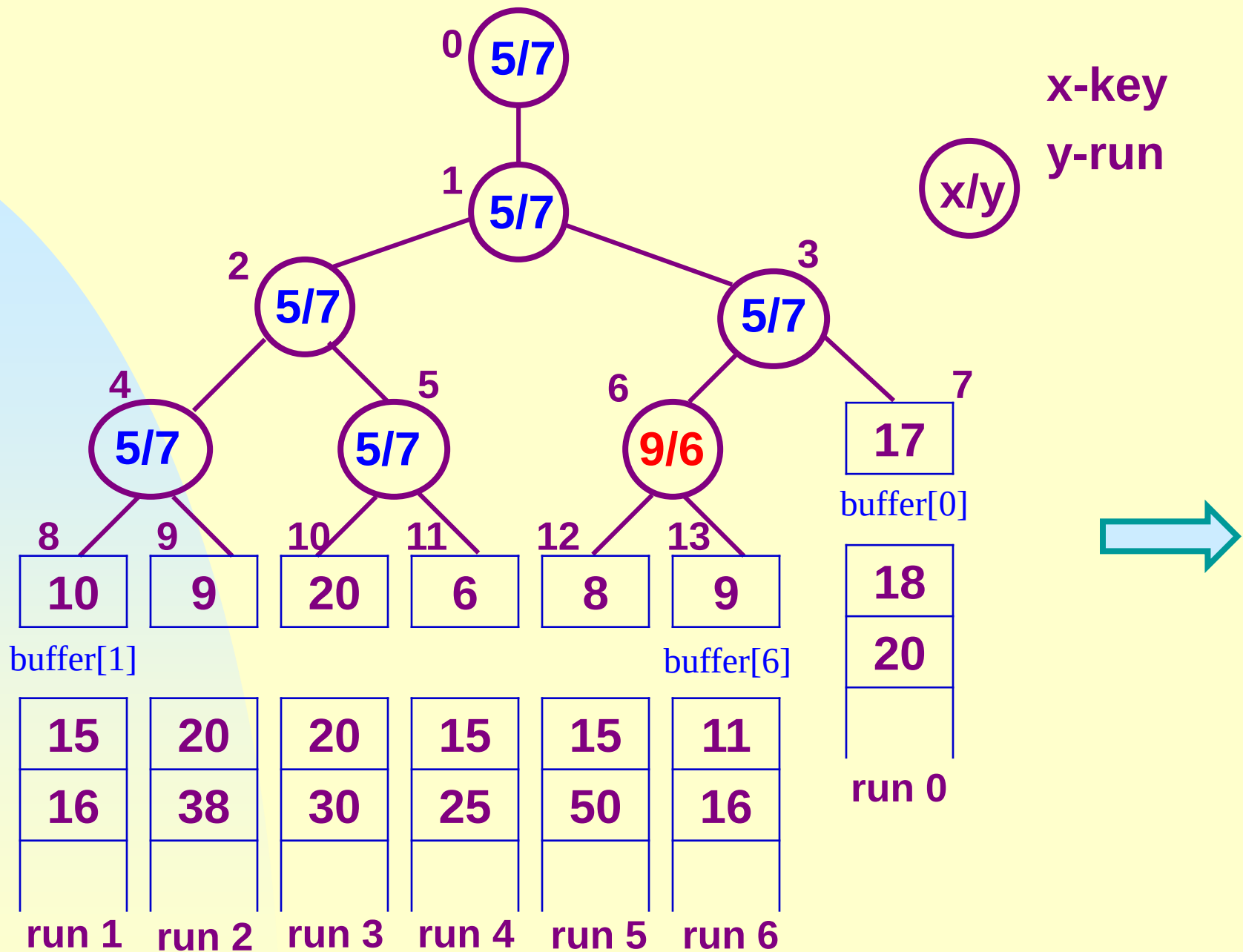
The number of leaf nodes in the lowest level is 6.4



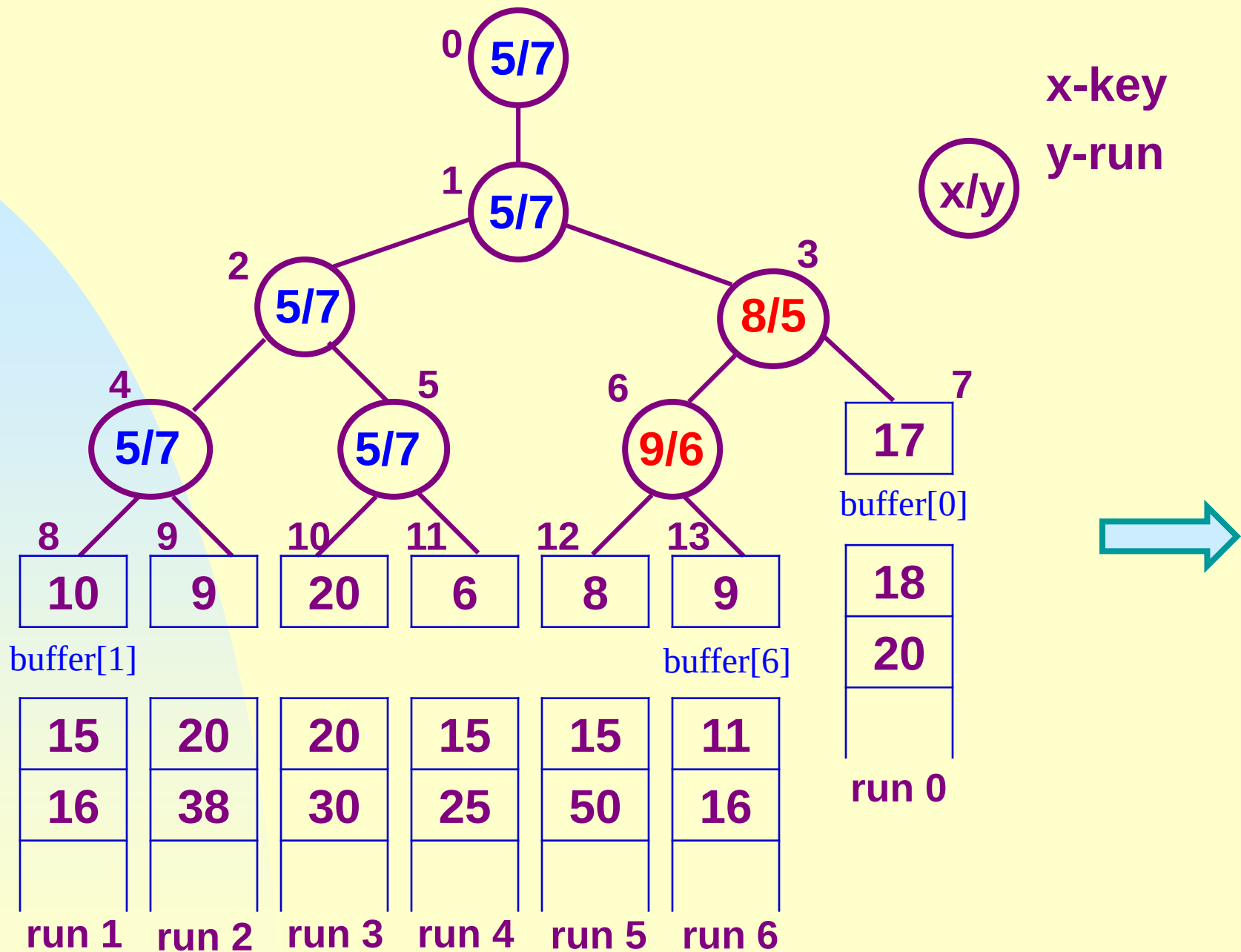
x-key
y-run

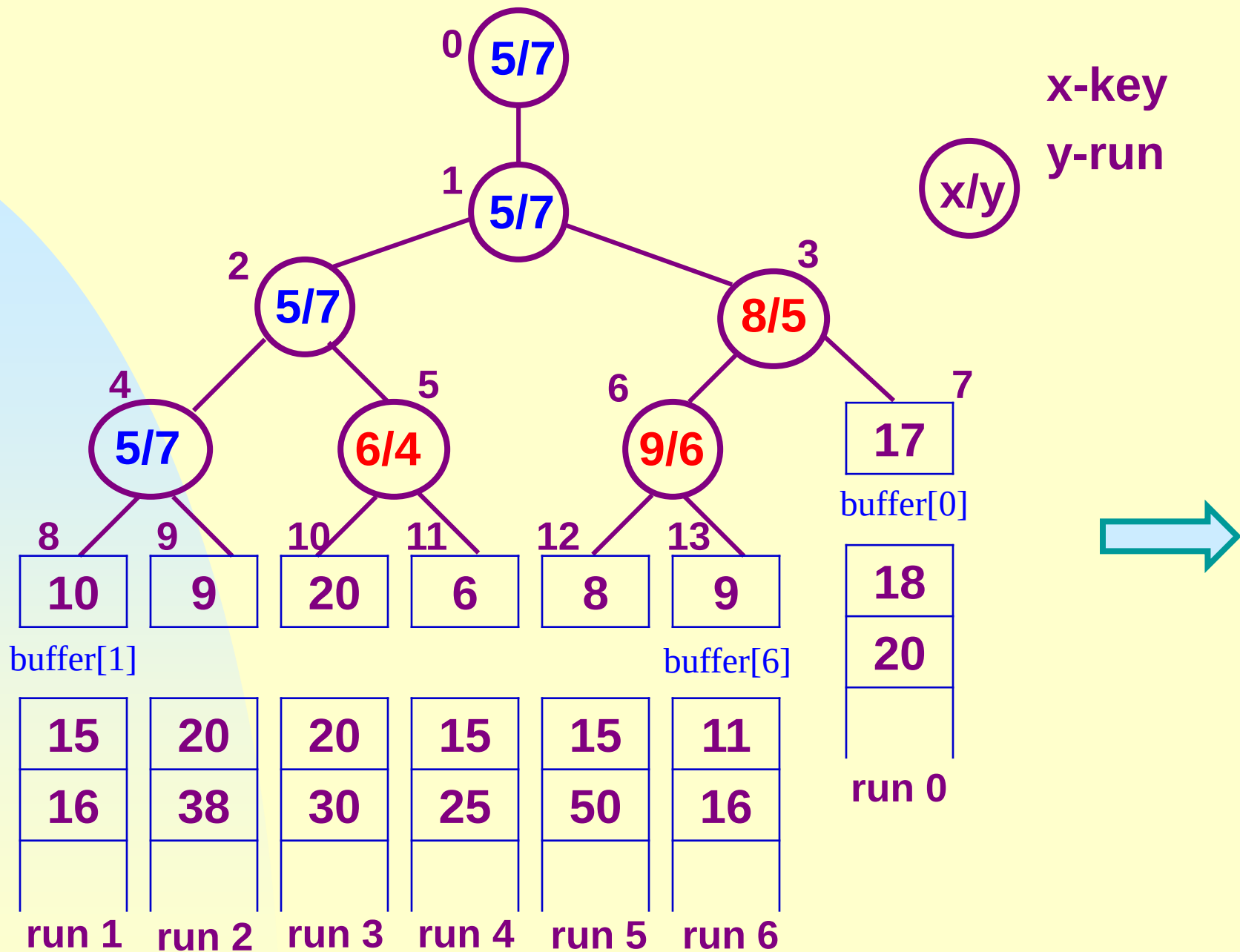


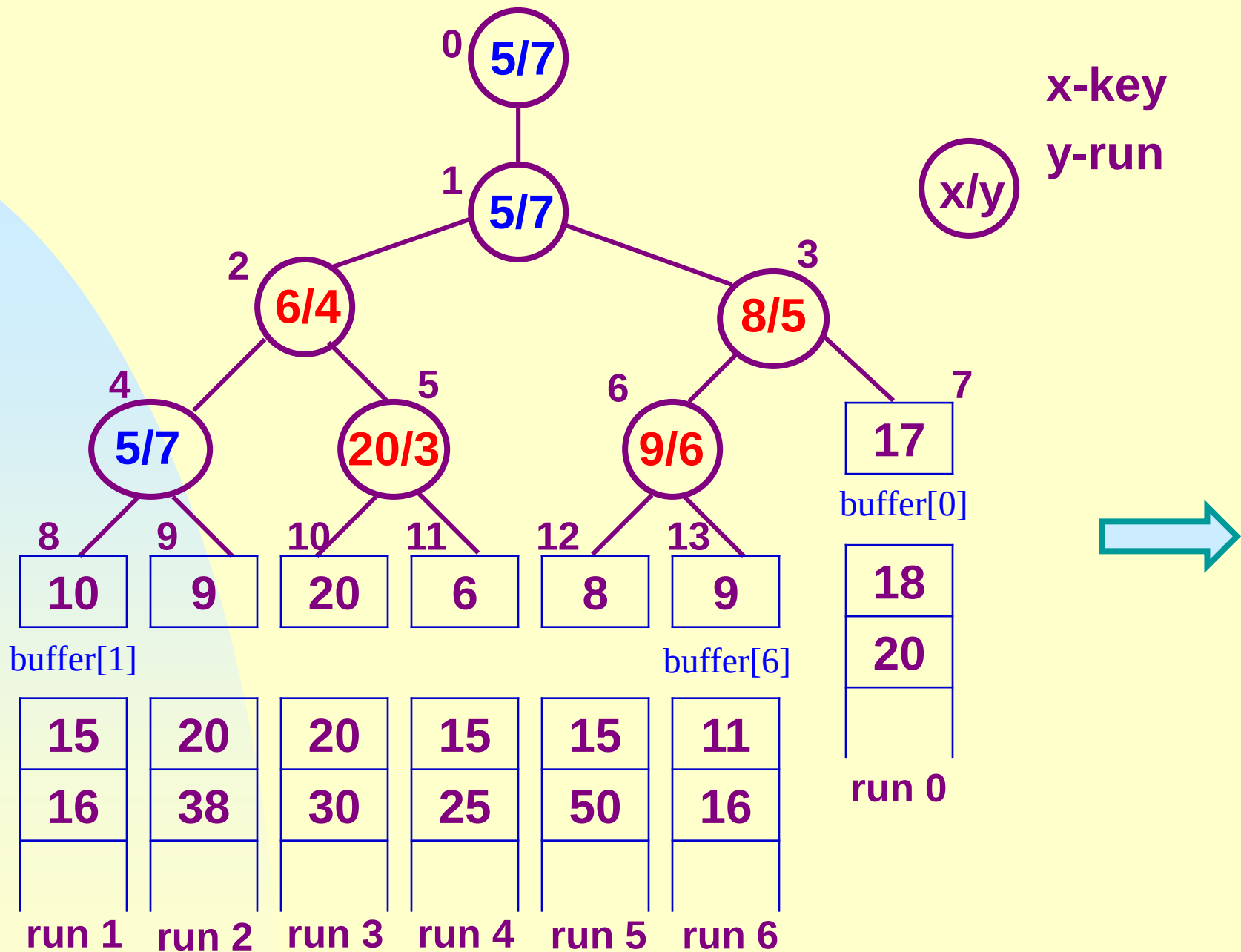
Initializing each non-leaf node to the absolute minimal key 5 or the run number 8.

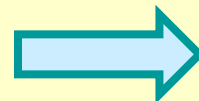
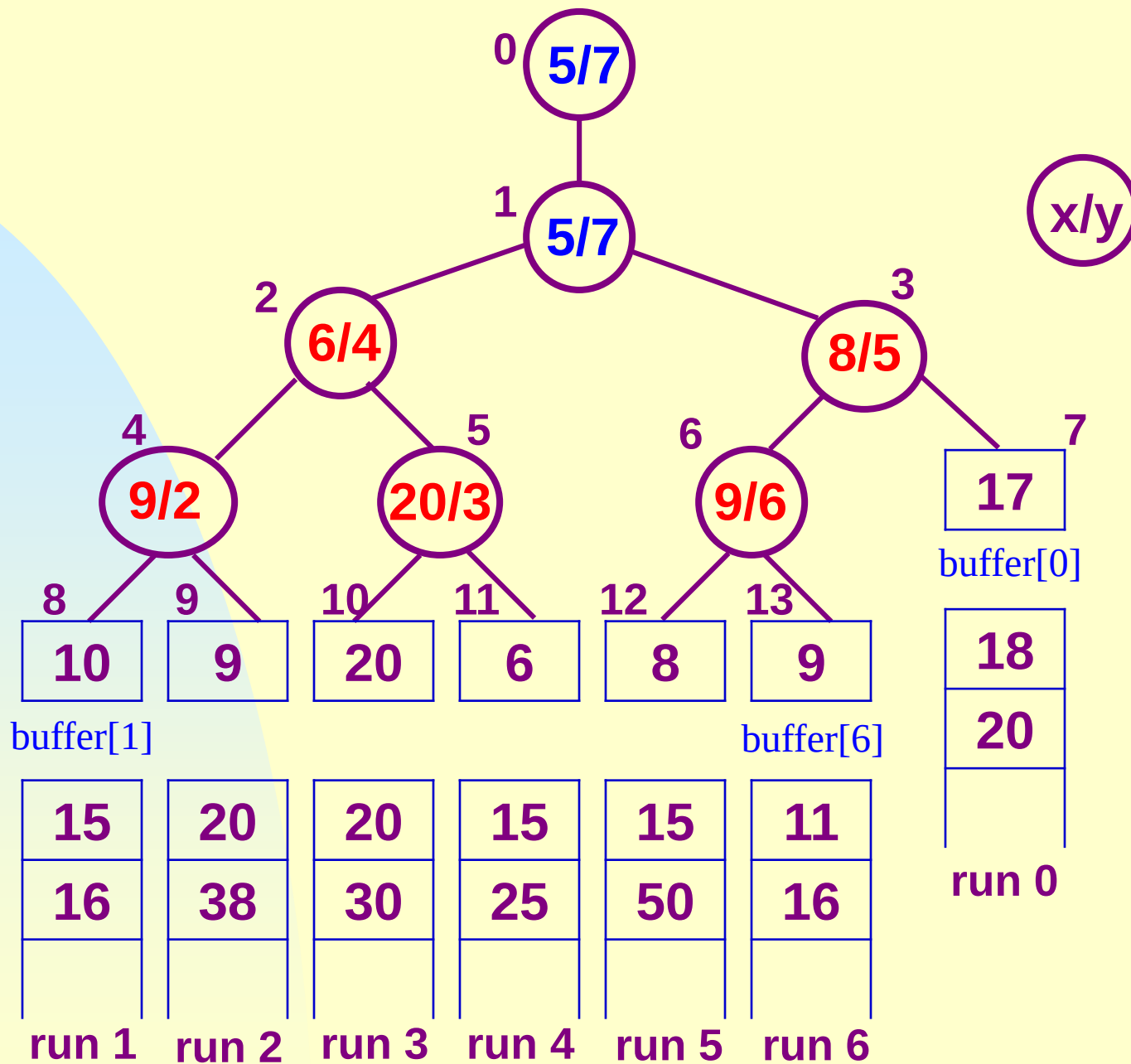


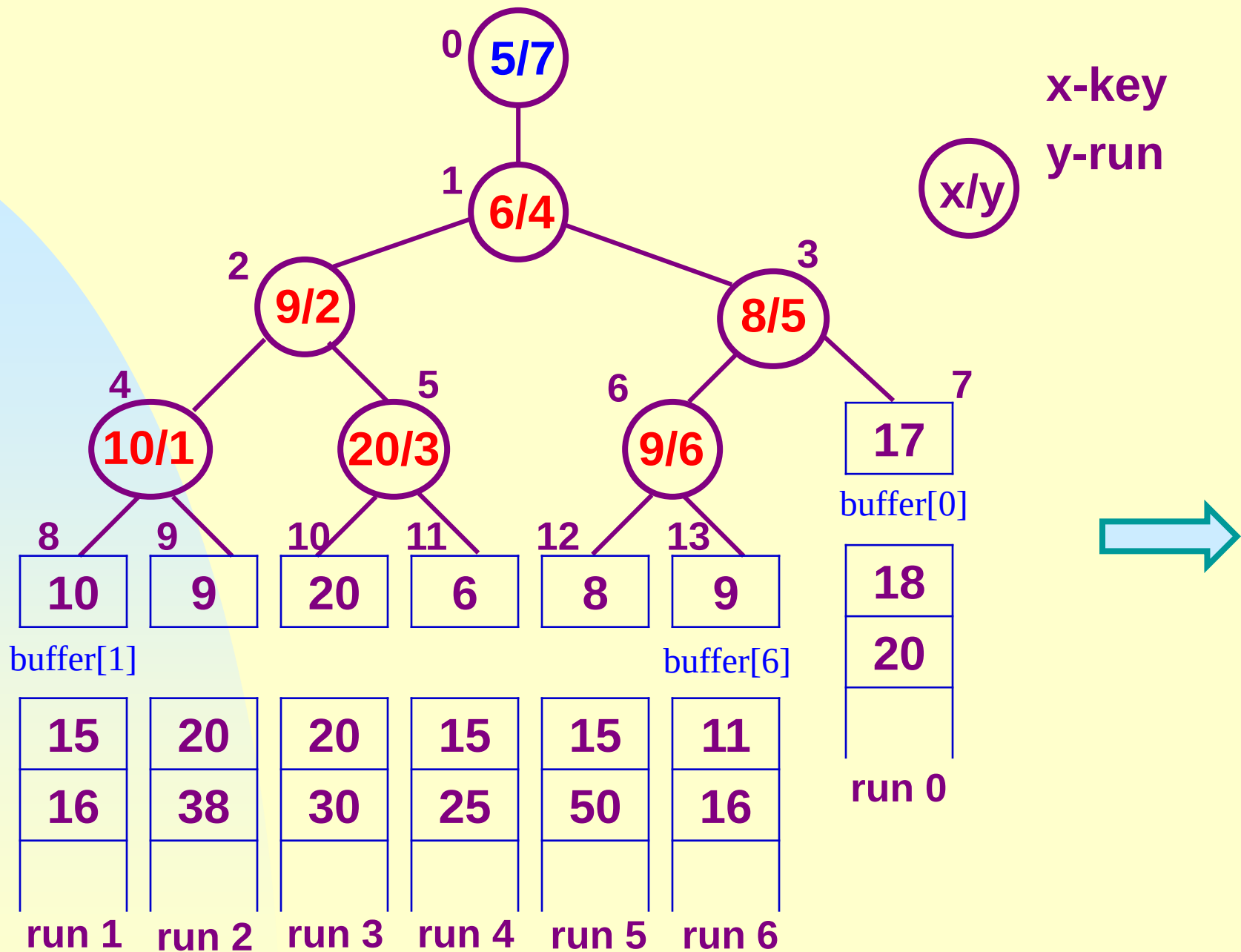
Playing tournaments along the path from the leaf node to node 1.

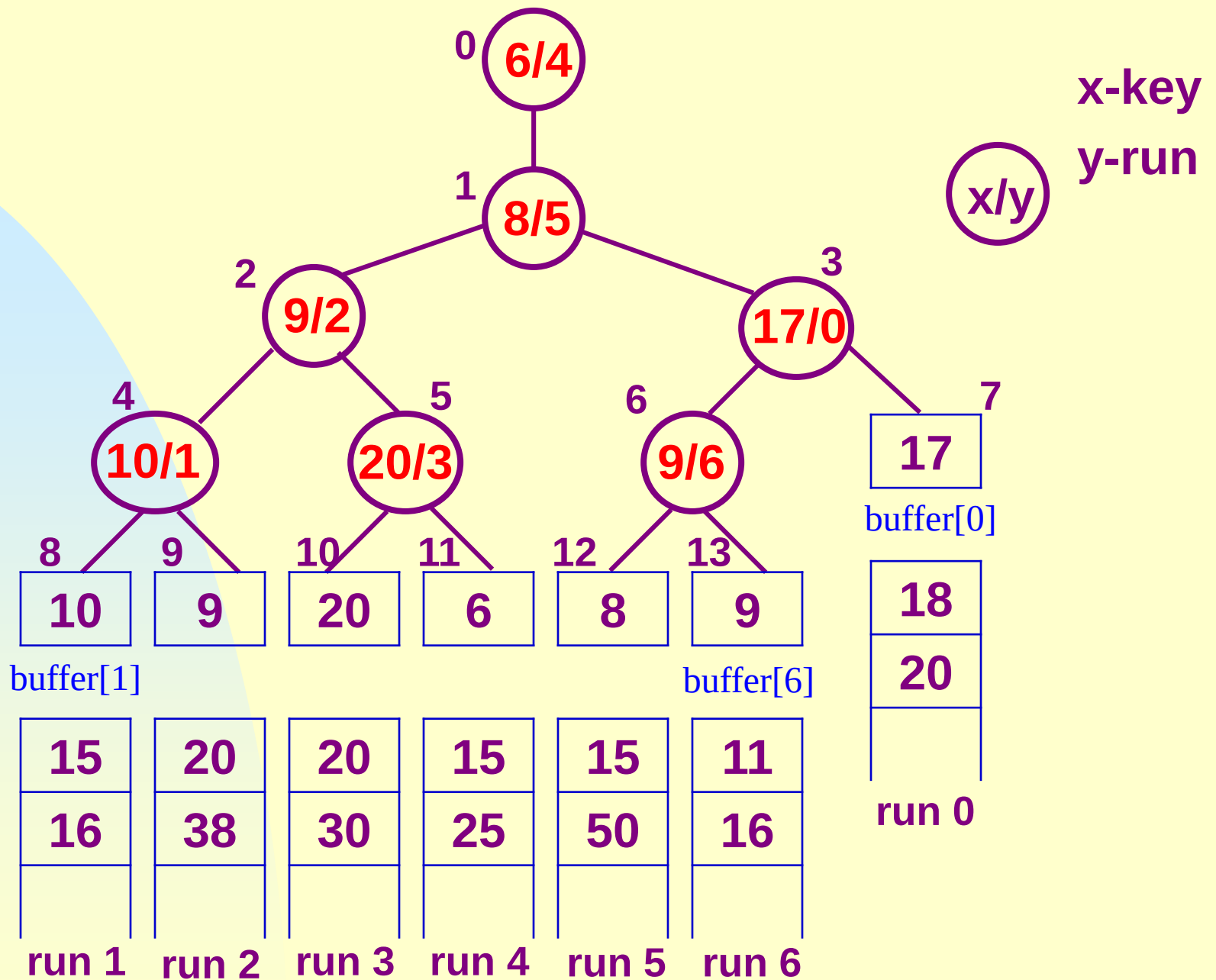












Class exercise

Question: There are 6 keys {1,2,3,4,5,6}, please construct the winner tree according to these keys.

5.9 Forests

Definition: A **forest** is a set of $n \geq 0$ **disjoint trees**.

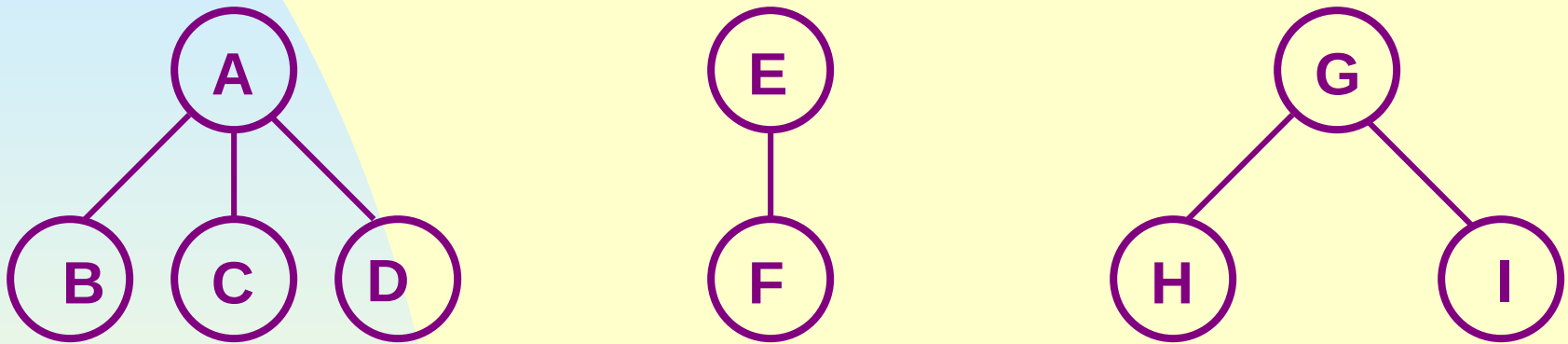


Fig. 5.34: Three-tree forest

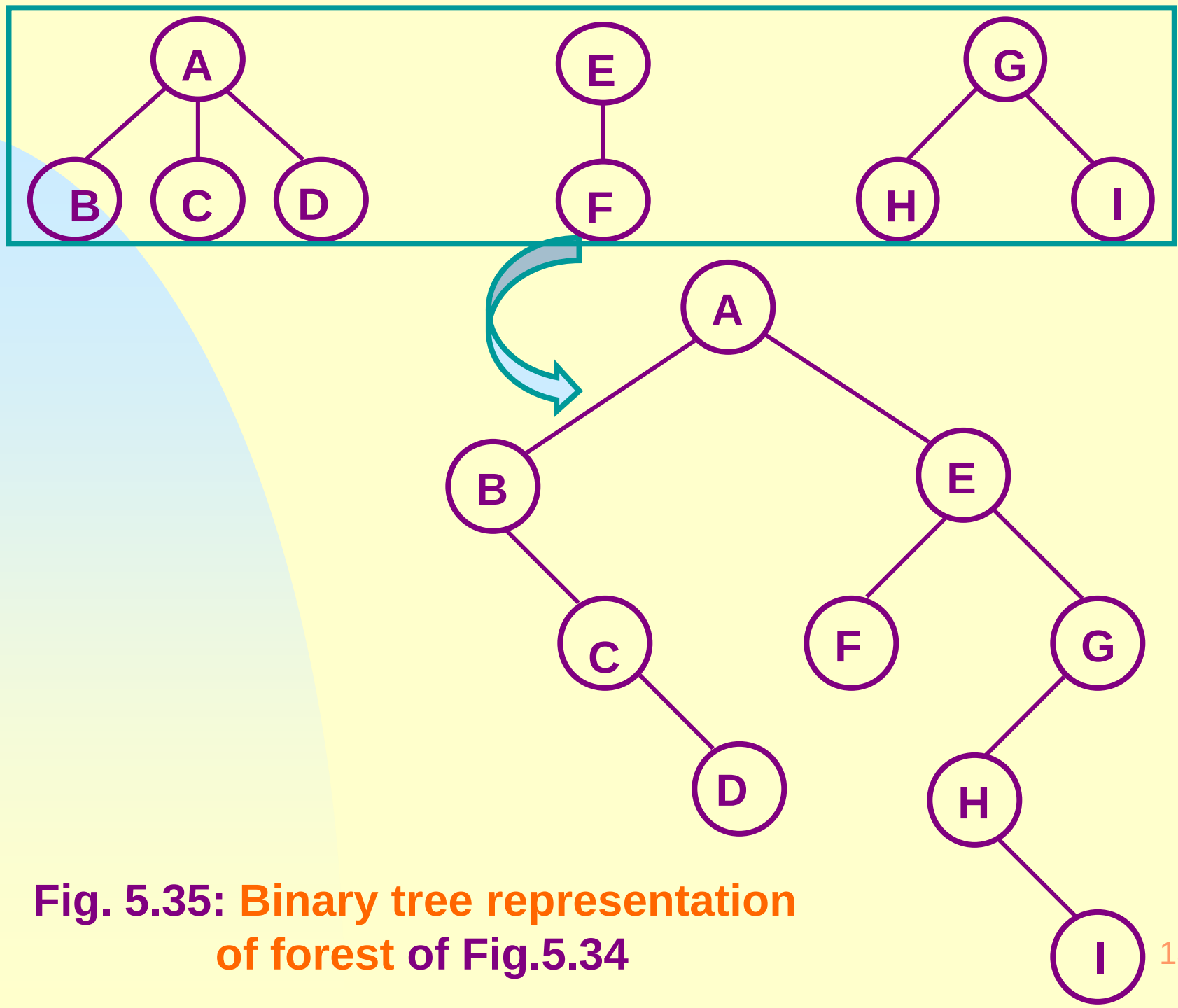
5.9.1 Transforming a Forest into a Binary Tree

Definition: If $T_1, \dots, T_n$ is a forest of trees, then the binary tree corresponding to this forest, denoted by $B(T_1, \dots, T_n)$,

(1) is empty if $n=0$

(2) has the root equal to the root(T_1); has the left subtree equal to $B(T_{11}, \dots, T_{1m})$, where $T_{11}, \dots, T_{1m}$ are the subtrees of root(T_1); and has right subtree $B(T_2, \dots, T_n)$.

Thinking of binary list representation for a tree.



**Fig. 5.35: Binary tree representation
of forest of Fig.5.34**

5.9.2 Forest Traversals

Let F be a forest and T be the corresponding binary tree of the forest.

■ Visiting the nodes of F in forest preorder is defined as:

(1) If F is empty then return.

(2) Visit the root of the first tree of F .

(3) Traverse the subtrees of the first tree in forest preorder.

(4) Traverse the remaining trees of F in forest preorder.

■ Preorder traversal on T is the same as Preorder traversal on F .

■ Visiting the nodes of F in forest inorder is defined as:

(1) If F is empty then return.

(2) Traverse the subtrees of the first tree in forest inorder.

(3) Visit the root of the first tree of F .

(4) Traverse the remaining trees of F in forest inorder.

■ Inorder traversal on T is the same as Inorder traversal on F .

- Visiting the nodes of F in forest postorder is defined as:
 - (1) If F is empty then return.
 - (2) Traverse the subtrees of the first tree in forest postorder.
 - (3) Traverse the remaining trees of F in forest postorder.
 - (4) Visit the root of the first tree of F .
- For postorder traversal, there is no natural analog between a forest and its corresponding binary tree.

- In a level-order traversal of F , nodes are visited by level, beginning with the roots of each trees in F . Within each level, nodes are visited from left to right.
- The level-order traversal of a forest and that of its associated binary tree do not necessarily yield the same result.

5.10 Representation of disjoint sets

5.10.1 Introduction

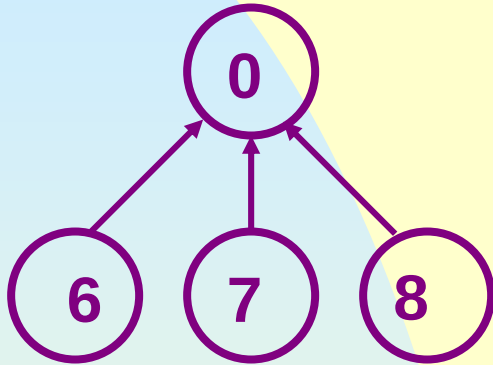
In this section, we study the use of trees in the representation of sets.

Assume:

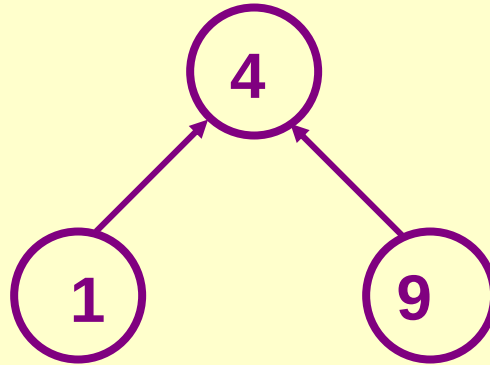
- ✓ Elements of the sets are the numbers 0, 1, 2, ..., n-1 (might be thought as indices).
- ✓ For any two sets S_i and S_j , if $i \neq j$, then $S_i \cap S_j = \emptyset$.

■ Each set can be represented by **a tree**:

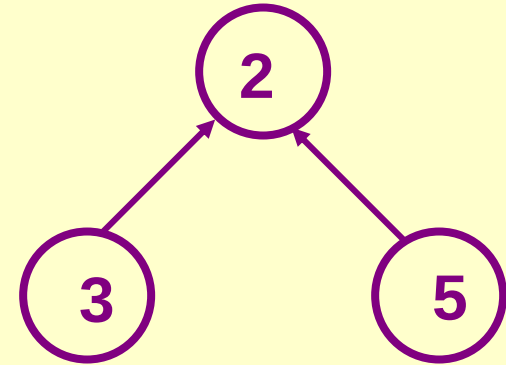
$S_1 = \{0, 6, 7, 8\}$



$S_2 = \{1, 4, 9\}$



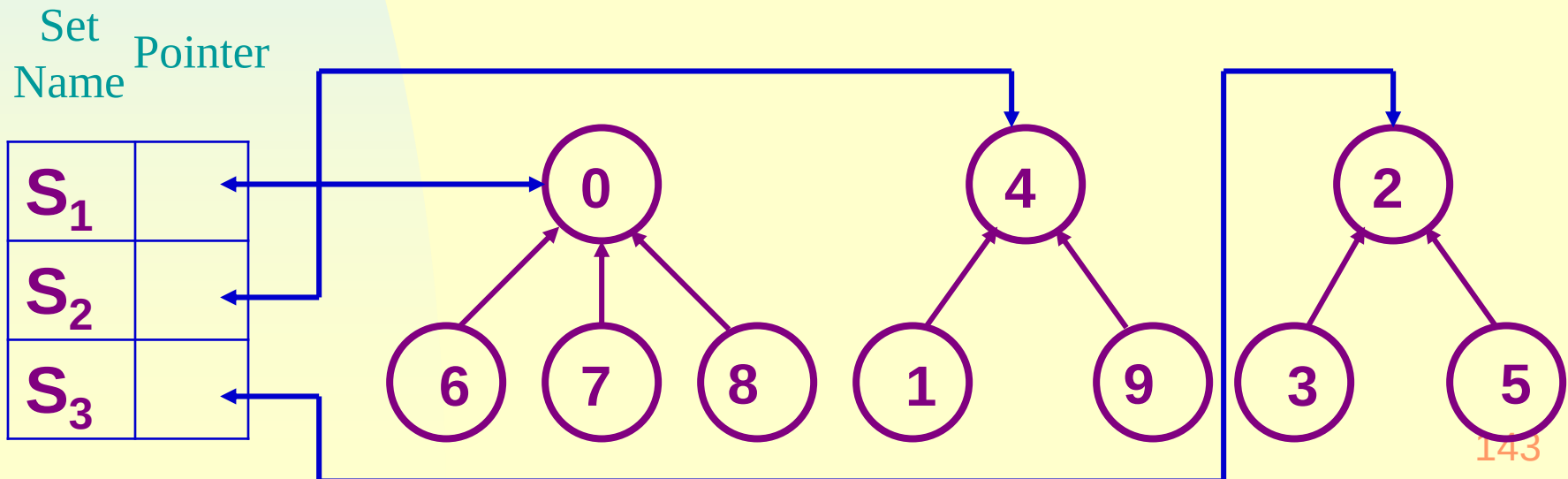
$S_3 = \{2, 3, 5\}$



■ Note: The nodes are linked **from the children to the parent**, rather than the usual method of linking from the parent to the children.

■ Convention:

- ✓ In the following discussion, we **ignore the actual set names**, just **identify sets by the roots** of the trees representing them.
- ✓ Because we can easily keep a **one-to-one mapping** between the set name and the root of the corresponding tree.



■ Since the set elements are numbered $0, 1, \dots, n-1$, we represent the tree nodes using an array **parent[n]**.

- The **parent[i]** gives the parent pointer of node i (element i).
- Notice that root nodes have a parent of **-1**.

■ For example, the array representation of S_1 , S_2 , and S_3 is as follows:

i	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
parent	-1	4	-1	2	-1	2	0	0	0	4

■ We will consider the operations:

(1) Disjoint set **union** $S_i \cup S_j$.

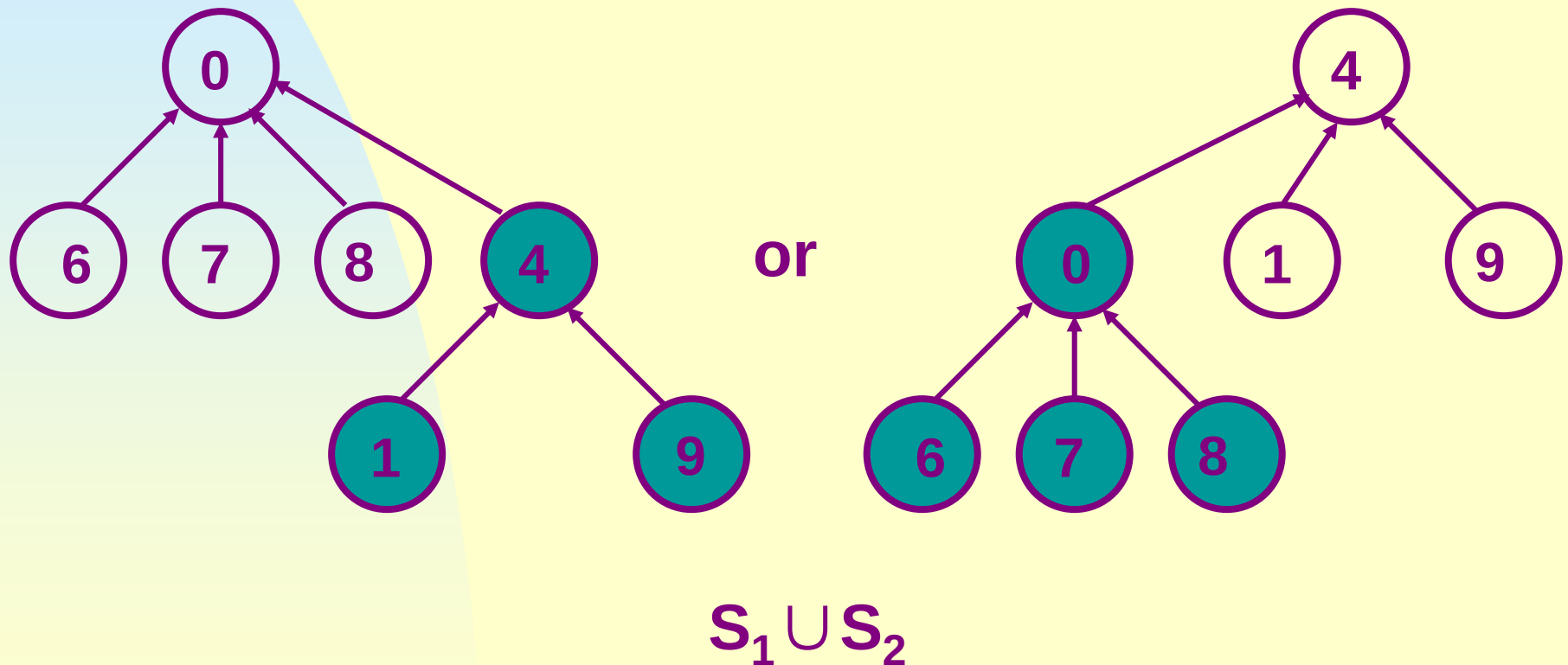
(2) **Find(i)**---find the set containing i .

Class definition and constructor for Sets

```
class Sets {  
public:  
    // Set operations  
    ...  
private:  
    int *parent;  
    int n; // number of set elements  
};  
  
Sets::Sets (int numberOfElements)  
{  
    if (numberOfElements < 2) throw "Must have at least 2  
elements.";  
    n=numberOfElements;  
    parent=new int[n];  
    fill(parent, parent+n, -1);  
}
```

5.10.2 Union and Find Operations

Suppose we wish to do $S_1 \cup S_2$, we simply **make one of the trees a subtree of the other**:



The simple function for union is as follows:

```
void Sets::SimpleUnion (int i, int j)
{ // Union sets with roots i and j,  $i \neq j$ 
  parent[i] = j; //tree i is as the subtree of tree j
}
```

To determine the root of the tree containing element is easy.

The simple function for find is as follows:

```
int Sets::SimpleFind (int i)
{ //find the root of the tree containing element i.
  while (parent[i] >= 0) i = parent[i];
  return i;
}
```

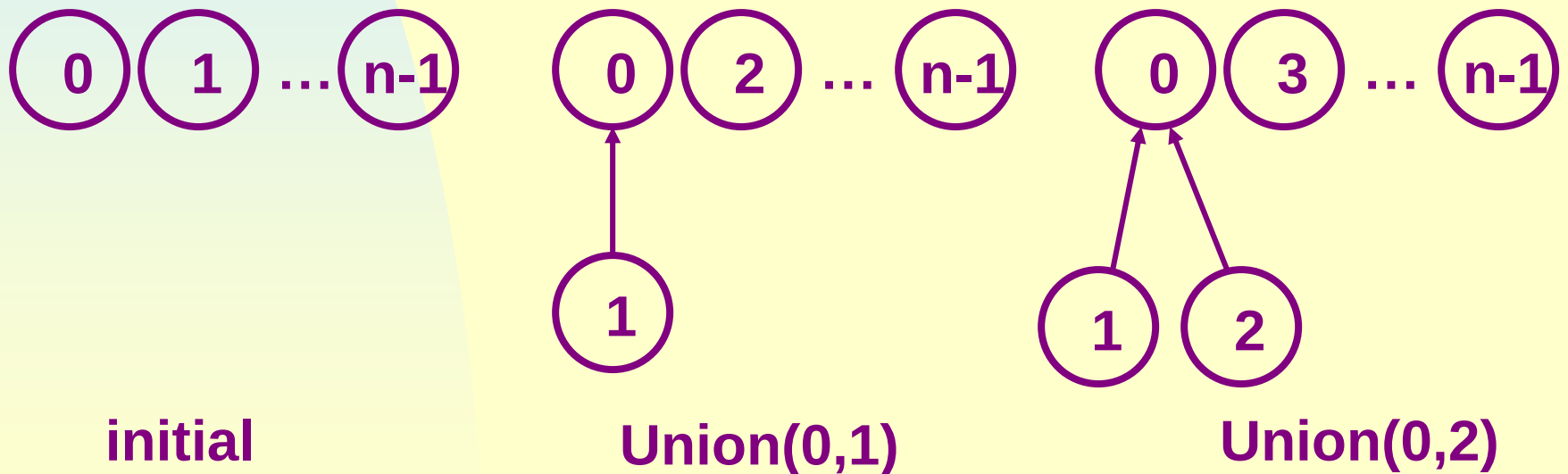
Analysis of SimpleUnion and SimpleFind:

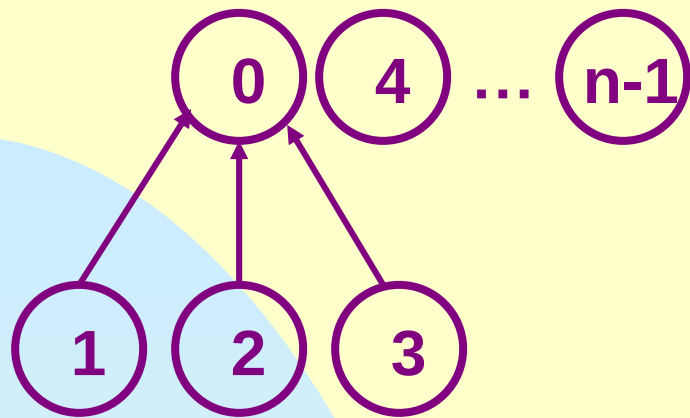
Given $S_i = \{i\}$, $0 \leq i < n$, we do $\text{Union}(0,1)$, $\text{Union}(1,2)$, ..., $\text{Union}(n-2,n-1)$, $\text{Find}(0)$, $\text{Find}(1)$, ..., $\text{Find}(n-1)$ one by one, and get the following tree:



- ✓ **$n-1$ unions take $O(n)$.**
- ✓ **The time to process a find for an element at level i of a tree is $O(i)$, so n finds take $O(\sum_{i=1}^n i) = O(n^2)$.**
- ✓ **Performance is not very good.**
- ✓ **To improve: avoid creating degenerate trees.**

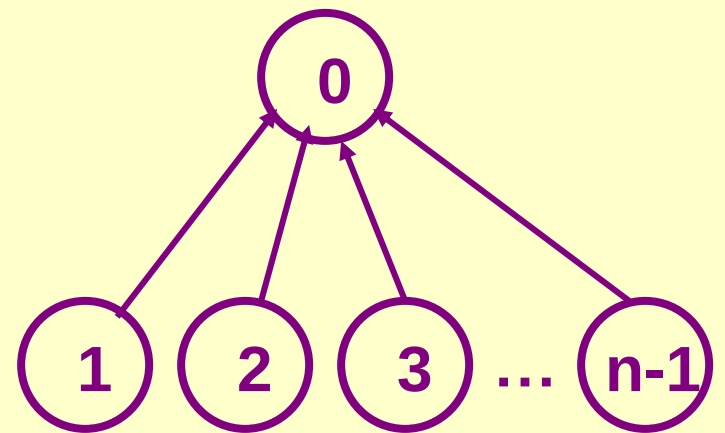
- **Definition [Weighting rule for Union(i,j)]:** If the number of nodes in the tree with root i is less than that in the tree with root j , then make j the parent of i ; otherwise make i the parent of j .
- When we use the weighting rule to perform the sequence of set unions given before, we get:





Union(0,3)

....



Union(0,n-1)

- To implement the weighting rule, we need to know **how many nodes** are there in every tree. To do this, we maintain a **count** field in the root of every tree.
- Since all nodes other than the roots of trees **have a non-negative number** in the parent field, we can maintain the count in the parent field of the roots as a **negative number**.

```
void Sets::WeightedUnion (int i, int j)
{ // Union sets with roots i and j,  $i \neq j$ , using the weighting rule
  // parent[i] < 0 and parent[j] < 0
  int temp = parent[i] + parent[j];
  if (parent[i] > parent[j]) { // i has fewer nodes than j
    parent[i] = j;
    parent[j] = temp;
  }
  else { // j has fewer nodes or
    // i and j have the same number of nodes
    parent[j] = i;
    parent[i] = temp;
  }
}
```

Analysis of WeightedUnion and SimpleFind:

- ✓ The time of a WeightedUnion is still $O(1)$.
- ✓ The maximum time to perform a find after a sequence of weighted unions is determined by Lemma 5.5.

Lemma 5.5: Assume that we start with a forest of trees, each having one node. Let T be a tree with m nodes created as a result of a sequence of weighted unions. The height of T is no more than $\lfloor \log_2 m \rfloor + 1$.

Proof by induction:

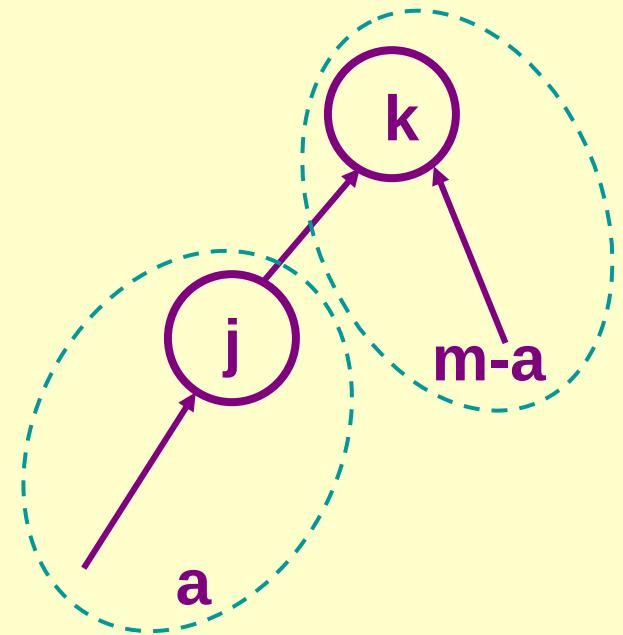
$m = 1$, it is true.

Assume it is true for all trees with **$i \leq m-1$** nodes.

For **$i = m$** , let **T** be a tree with **m** nodes created by **WeightedUnion**.

Consider the **last union** performed, **Union(k, j)**.

Let a be the number of nodes in tree j and $m-a$ that in tree k . Without loss of generality, assume $1 \leq a \leq m/2$. Then the height of T is either the same as that of k or is $1 +$ that of j .



$$m-a \geq m/2 \geq a$$

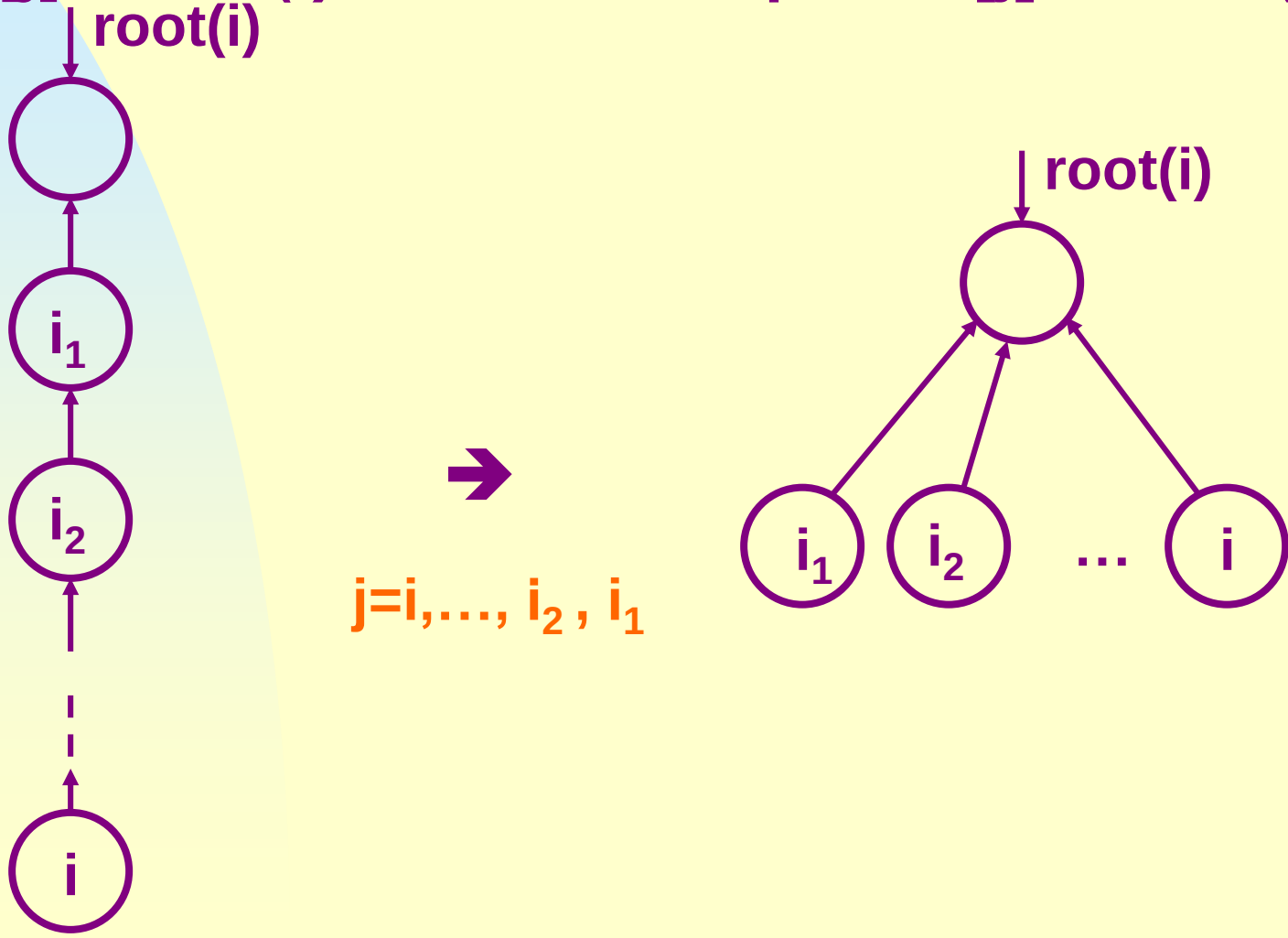
- ✓ If the former is the case, the height of $T \leq \lfloor \log_2 (m-a) \rfloor + 1 \leq \lfloor \log_2 m \rfloor + 1$.
- ✓ If the latter is the case, the height of $T \leq \lfloor \log_2 a \rfloor + 1 + 1 \leq \lfloor \log_2 m/2 \rfloor + 2 = \lfloor \log_2 m \rfloor + 1$.

Inductive assumption

Thus, the time to process a “find” is at most $O(\log n)$ in a tree of n nodes.

Further improvement in the find algorithm

Definition [Collapsing rule] : If j (including i) is a node on the path from i to its root ($\text{root}(i)$), and $\text{parent}[j] \neq \text{root}(i)$, then we set $\text{parent}[j]$ to $\text{root}(i)$.



```
int Sets::CollapsingFind (int i )
{ // Find the root of tree containing element i. Use the
  // collapsing rule to collapse all nodes from i to the root.
  int r;
  for (r = i; parent[r] >= 0; r = parent[r]); // find r, the root of i
  while ( i != r ) {
    int s = parent[i];
    parent[i] = r; //r is set to i's parent
    i = s;
  }
  return r;
}
```

Analysis of WeightedUnion and CollapsingFind:

- ✓ Use of the collapsing rule **roughly doubles the time** for an individual find. However, it reduces the worst-case time over a sequence of finds.
- ✓ The worst-case complexity of processing a sequence of unions and finds using **WeightedUnion and CollapsingFind** is stated in Lemma 5.6.
- ✓ **Lemma 5.6 [Tarjan and Van Leeuwen]:** Assume that we **start with a forest of n trees, each having one node**. Let $T(f, u)$ be the maximum time required to process any intermixed sequence of f finds and u unions. Assume that $u \geq n/2$, then $k_1(n+f \alpha(f+n, n)) \leq T(f, u) \leq k_2(n+f \alpha(f+n, n))$ for some positive constants k_1 and k_2 .

- ✓ The requirement that $u \geq n/2$ in lemma 5.6 is really not significant, as when $u < n/2$, some elements are involved in no union operation. These elements remain in singleton sets throughout the sequence of union and find operations and can be eliminated from consideration, as find operations that involve these elements can be done in $O(1)$ time each.
- ✓ This lemma makes use of a function $\alpha(p,q)$ that is related to a functional inverse of Ackermann's function $A(i,j)$.

✓ Ackermann's function $A(i,j)$ is defined as:

$$A(1, j) = 2^j \quad \text{for } j \geq 1$$

$$A(i, 1) = A(i-1, 2) \quad \text{for } i \geq 2$$

$$A(i, j) = A(i-1, A(i, j-1)) \quad \text{for } i, j \geq 2$$

✓ $\alpha(p,q) = \min \{z \geq 1 \mid A(z, \lfloor p/q \rfloor) > \log_2 q\}, p \geq q \geq 1$

✓ The function $A(i,j)$ is a very rapidly growing function. Consequently, $\alpha(p,q)$ growing very slowly as p and q are increased.

$$A(1, 2) = 2^2$$

$$A(3,1) = A(2, 2) = A(1, A(2, 1)) = A(1, A(1, 2))$$

$$= 2^{2^2} = 16$$

$$A(4,1) = A(3, 2) = A(2, A(3, 1)) = A(2, 16)$$

$$= A(1, A(2, 15)) = 2^{A(2, 15)}$$

$$= 2^{2^{A(2, 14)}} = \dots = 2^{2^{2^{\dots^{2^{16}}}}} \quad (\text{very big !})$$

Since $A(4,1)$ is a very large number and in our application q will be the number of set elements n and p will be $n+f$ (f is the number of finds), $\alpha(p,q) \leq 4$ for all practical purposes.

- ✓ **Note:** Even though the function $\alpha(f+n, n)$ grows very slowly, $T(f, u)$ is not linear in the number of unions and finds.
- ✓ The **space** requirements are one node for each element.

5.10.3 Application to Equivalent Classes

equivalence classes $\Leftrightarrow$ disjoint sets

✓ Initially, $\text{parent}[i] = -1$, $0 \leq i \leq n-1$.

✓ To process **an equivalence pair**, $i \equiv j$:

Let $x = \text{find}(i)$, $y = \text{find}(j)$ --- 2 finds

If $x \neq y$ then $\text{union}(x, y)$

If $x = y$ then nothing to do --- at most 1 union

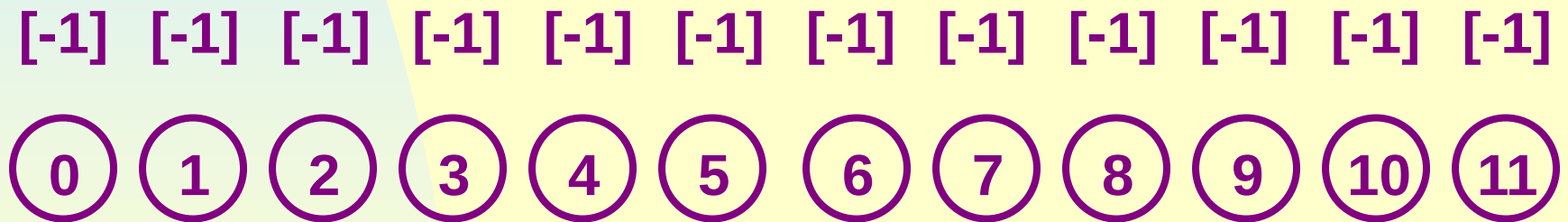
✓ Thus if we have **n elements** and **m equivalence pairs**, we need **$2m$ finds** and at most **$\min\{n-1, m\}$ unions**. The total time is **$O(n+2m \alpha(n+2m, n))$** .

m may be greater than n-1

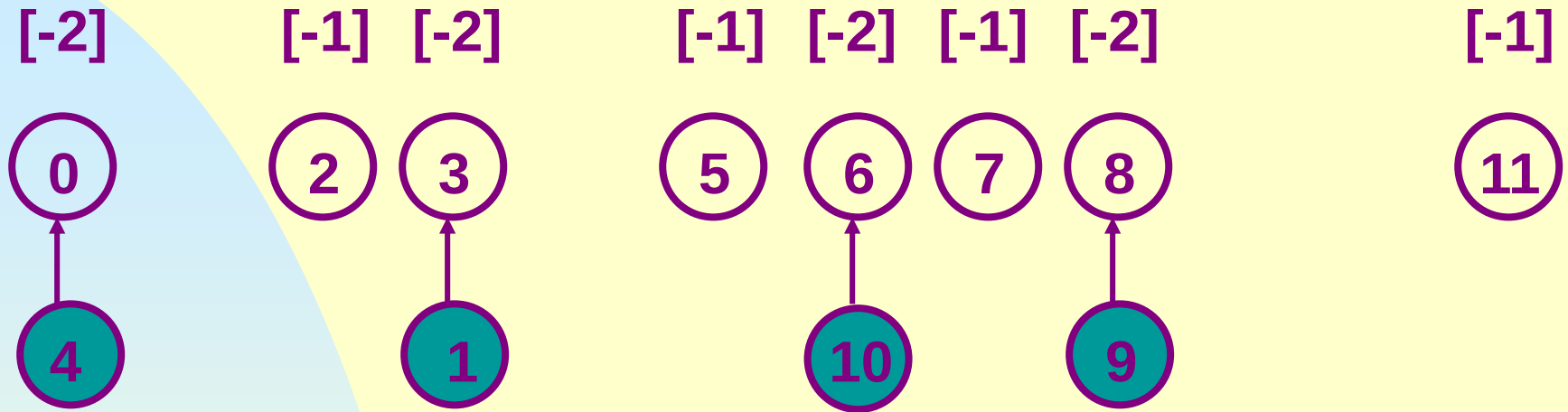
Example:

$n = 12$, process equivalence pairs:

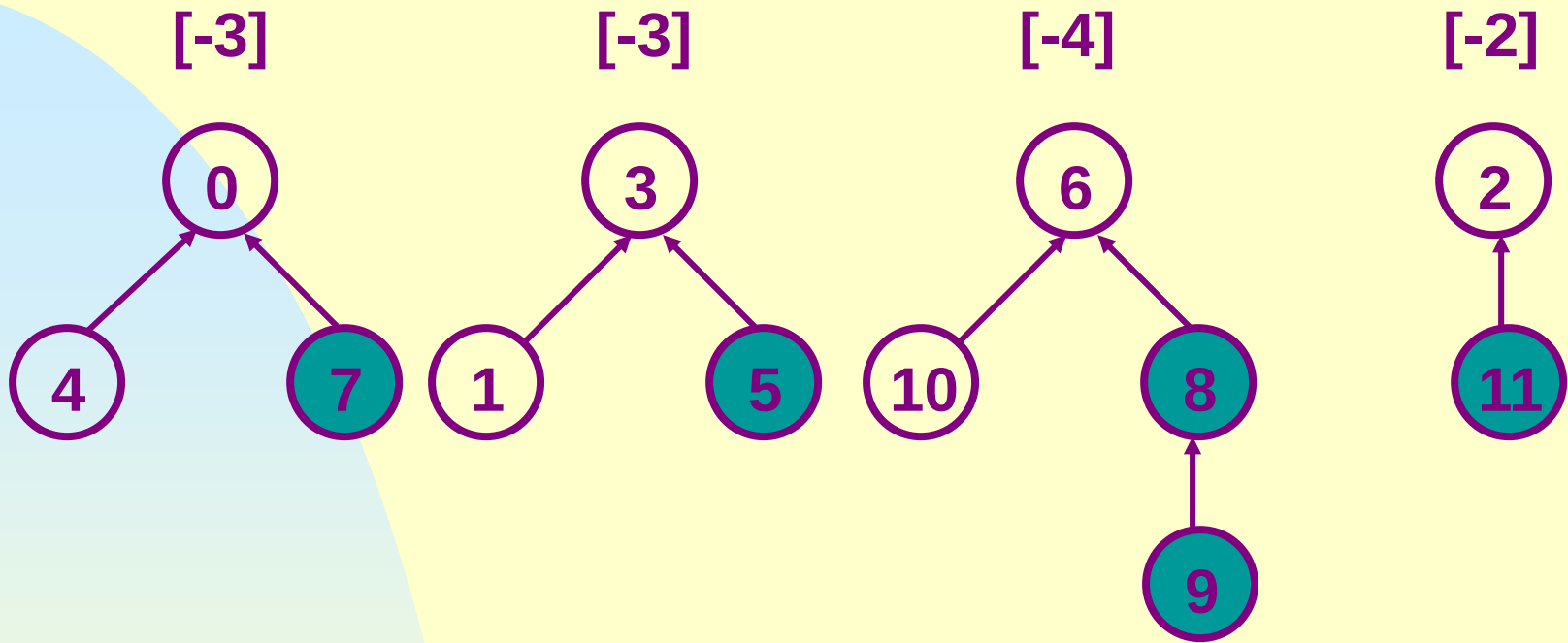
$0 \equiv 4, 3 \equiv 1, 6 \equiv 10, 8 \equiv 9, 7 \equiv 4, 6 \equiv 8, 3 \equiv 5, 2 \equiv 11,$
 $11 \equiv 0$



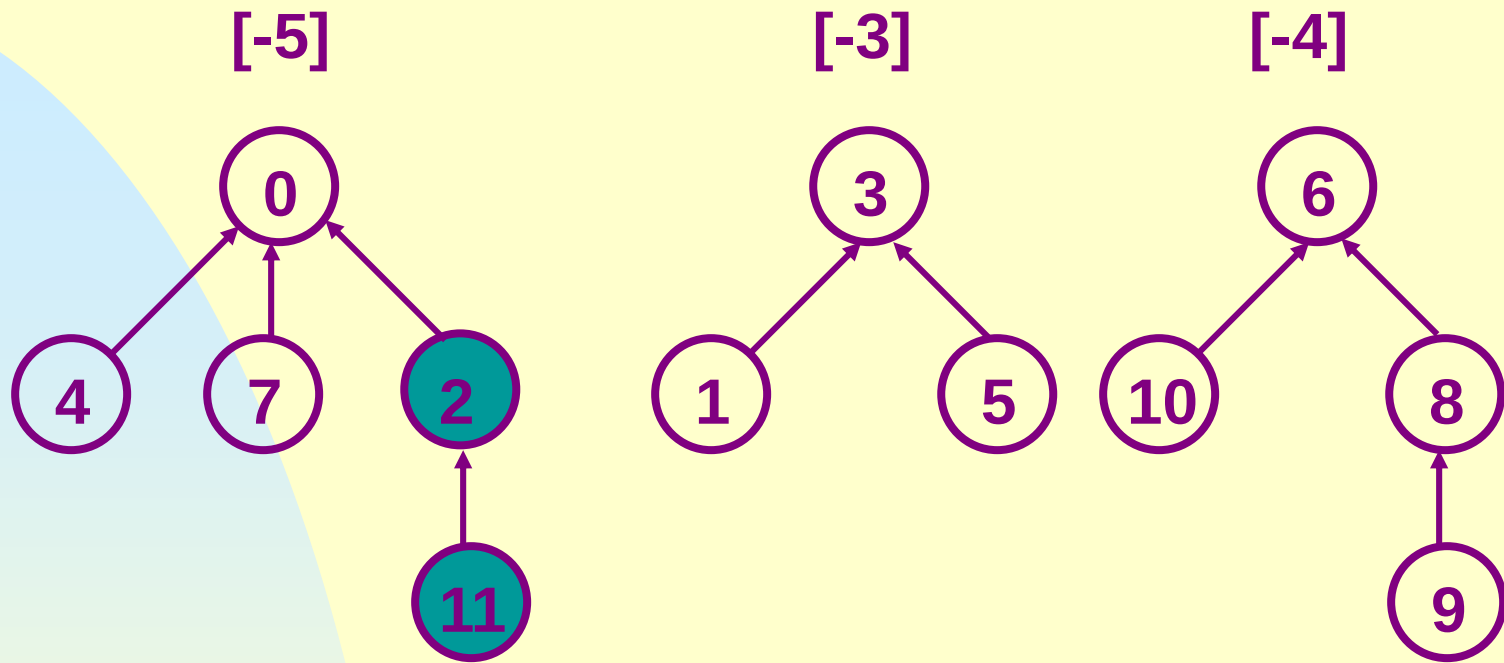
(a) Initial trees



(b) After processing $0 \equiv 4$, $3 \equiv 1$, $6 \equiv 10$, and $8 \equiv 9$



(c) After processing $7 \equiv 4$, $6 \equiv 8$, $3 \equiv 5$, and $2 \equiv 11$



(d) After processing $11 \equiv 0$

Exercise(s) and Experiment(s)

Exercise(s):

- **P258:4; P296:1,2; P277:1; P287:3; P301:5;
P323:2,4**

Experiment(s):

- **P267:10; P272:1,2; P316:3**